Chương 9

Prompting

Prompting (*prompting*) (kỹ thuật prompt) là quá trình cung cấp cho một LLM một đầu vào hoặc một gợi ý cụ thể để nó sinh ra đầu ra mong muốn hoặc thực hiện một nhiệm vụ. Ví dụ, nếu muốn LLM dịch một câu từ tiếng Anh sang tiếng Trung, ta có thể "prompt" nó như sau

> Dịch văn bản sau từ tiếng Anh sang tiếng Trung.
> Văn bản: The early bird catches the worm.
> Bản dịch: ___

Chú giải của người dịch: Prompt là gì?

Prompt (giữ nguyên tiếng Anh, có thể hiểu là "lời nhắc", "câu lệnh đầu vào") là đoạn văn bản ta gõ vào để "bảo" mô hình làm việc gì đó — giống như đề bài ta đưa cho một học sinh. Mô hình ngôn ngữ lớn (LLM) như ChatGPT đọc đoạn văn bản này rồi viết tiếp phần trả lời. Trong ví dụ trên, câu tiếng Anh *The early bird catches the worm* nghĩa đen là "Con chim dậy sớm bắt được sâu" (tục ngữ: ai chịu dậy sớm, bắt tay vào việc sớm thì sẽ được lợi). Dòng "Bản dịch:" để trống chính là chỗ ta chờ LLM viết tiếp. Trong prompt gốc, câu tiếng Anh được giữ nguyên vì đó là văn bản cần dịch.

Prompting có vai trò then chốt đối với LLM vì nó ảnh hưởng trực tiếp đến mức độ hiệu quả mà các mô hình này hiểu và đáp lại các truy vấn của người dùng. Một prompt được soạn kỹ có thể dẫn dắt LLM sinh ra những câu trả lời chính xác hơn, liên quan hơn và phù hợp với ngữ cảnh hơn. Hơn nữa, quá trình này có thể được tinh chỉnh lặp đi lặp lại. Bằng cách phân tích các câu trả lời của LLM, người dùng có thể điều chỉnh prompt

để sát hơn với nhu cầu cụ thể của mình. Do tầm quan trọng của prompting trong việc ứng dụng LLM, việc thiết kế prompt đã trở thành một kỹ năng thiết yếu đối với người dùng và nhà phát triển làm việc với LLM. Tầm quan trọng ngày càng tăng của việc thiết kế prompt đã làm nảy sinh một lĩnh vực nghiên cứu sôi động gọi là thiết kế prompt (*prompt engineering*), trong đó chúng ta thiết kế các prompt hiệu quả để tận dụng LLM tốt hơn và nâng cao tính hữu dụng của chúng trong các ứng dụng thực tế.

Một khái niệm quan trọng liên quan đến prompting là học trong ngữ cảnh (*in-context learning*). Khi prompt một LLM, ta có thể thêm thông tin mới vào ngữ cảnh, chẳng hạn các ví dụ minh họa cách giải quyết vấn đề. Điều này cho phép LLM học từ ngữ cảnh đó cách giải bài toán. Dưới đây là một ví dụ prompt LLM bằng vài ví dụ minh họa về cách phân loại văn bản theo cực cảm xúc (*sentiment polarity*).

> Dưới đây là một số ví dụ về phân loại văn bản.
> Ví dụ 1: Chúng tôi đã có một bữa tối thật vui vẻ cùng nhau. → Nhãn: Tích cực
> Ví dụ 2: Tôi bực mình vì những lần trì hoãn. → Nhãn: Tiêu cực
> Nhãn của câu "Lời nhận xét đó khá là tổn thương." là gì?
> Nhãn: ___

Học trong ngữ cảnh thường được xem là một năng lực đột sinh (*emergent ability*) của LLM, xuất hiện sau quá trình tiền huấn luyện (*pre-training*). Mặc dù LLM có thể được huấn luyện hoặc tinh chỉnh (*fine-tuning*) để thực hiện nhiệm vụ mới, học trong ngữ cảnh cung cấp một cách rất hiệu quả để thích nghi các mô hình này mà không cần bất kỳ công sức huấn luyện hay tinh chỉnh nào. Có lẽ đây là một trong những đặc điểm nổi bật nhất của LLM: chúng thực sự học được tri thức tổng quát về thế giới và về ngôn ngữ trong quá trình tiền huấn luyện, và ta có thể dễ dàng áp dụng tri thức đó vào những thử thách mới. Hơn nữa, học trong ngữ cảnh phản ánh xu hướng rộng hơn là làm cho các hệ thống AI tổng quát hơn và thân thiện hơn với người dùng. Thay vì cần các kỹ sư chuyên môn tinh chỉnh mô hình cho từng nhiệm vụ riêng, người dùng có thể tương tác với LLM theo cách trực quan hơn, chỉ cần đưa ví dụ hoặc điều chỉnh ngữ cảnh khi cần.

Chú giải của người dịch: Học trong ngữ cảnh khác gì với "huấn luyện"?

Khi *huấn luyện* hay *tinh chỉnh*, ta thay đổi các tham số (hàng tỉ con số bên trong mô hình) để mô hình "nhớ" cách làm một việc mới — tốn nhiều máy tính và thời gian. Còn *học trong ngữ cảnh* thì không đụng đến tham số nào: ta chỉ đặt vài ví dụ mẫu ngay trong prompt, mô hình đọc chúng rồi bắt chước theo khi trả lời câu hỏi mới. Giống như khi làm bài kiểm tra, thầy cô cho sẵn hai bài mẫu đã giải ở đầu đề, ta nhìn cách giải mẫu để làm bài tương tự — ta không cần "học lại" cả chương, và khi sang đề khác thì các bài mẫu ấy cũng không còn. Ở ví dụ phân loại phía trên, LLM nhìn hai ví dụ có nhãn và sẽ đoán câu "Lời nhận xét đó khá là tổn thương" có nhãn *Tiêu cực*.
"Năng lực đột sinh" là năng lực hầu như không thấy ở các mô hình nhỏ nhưng "bỗng nhiên" xuất hiện khi mô hình đủ lớn và được tiền huấn luyện trên đủ nhiều dữ liệu.

Trong chương này, chúng ta tập trung vào các kỹ thuật prompting cho LLM. Ta bắt đầu bằng việc xem xét một số thiết kế prompt thú vị thường được dùng trong thiết kế prompt. Sau đó, ta bàn về một loạt cải tiến cho các phương pháp này. Cuối cùng, ta khám phá các cách tiếp cận nhằm tự động hóa việc thiết kế prompt.

9.1 Thiết kế prompt tổng quát

Mục này trình bày các khái niệm cơ bản trong thiết kế prompt, cùng với các ví dụ về cách prompt LLM cho nhiều nhiệm vụ NLP khác nhau. Vì hiệu quả của prompting phụ thuộc rất nhiều vào LLM được sử dụng, prompt thường khác nhau giữa các LLM, khiến ta khó đưa ra một danh sách đầy đủ các prompt cho mọi LLM và mọi nhiệm vụ hạ nguồn (*downstream task*). Vì vậy, phần thảo luận này không tập trung vào một LLM cụ thể nào. Thay vào đó, mục tiêu là đưa ra các nguyên tắc định hướng cho việc thiết kế prompt.

9.1.1 Những điều cơ bản

Thuật ngữ *prompt* được dùng theo nhiều cách khác nhau. Trong chương này, ta định nghĩa prompt là văn bản đầu vào của một LLM, ký hiệu là $\mathbf{x}$.

LLM sinh ra đầu ra $\mathbf{y}$ theo xác suất có điều kiện (*conditional probability*) $\Pr(\mathbf{y}|\mathbf{x})$. Trong quá trình sinh này, prompt đóng vai trò là điều kiện để đưa ra dự đoán, và nó có thể chứa bất kỳ thông tin nào giúp mô tả và giải quyết vấn đề.

Chú giải của người dịch: Đọc ký hiệu $\Pr(\mathbf{y}|\mathbf{x})$

$\Pr(\mathbf{y}|\mathbf{x})$ đọc là "xác suất của $\mathbf{y}$ khi biết $\mathbf{x}$". Ở đây $\mathbf{x}$ là prompt ta đưa vào, $\mathbf{y}$ là một câu trả lời có thể có. Với cùng một câu hỏi, mô hình "thấy" có rất nhiều câu trả lời khả dĩ, mỗi câu có một xác suất; mô hình sẽ ưu tiên sinh ra những câu trả lời có xác suất cao. Thay đổi prompt $\mathbf{x}$ tức là thay đổi điều kiện, nên phân phối xác suất trên các câu trả lời cũng thay đổi theo — đó là lý do một prompt tốt có thể dẫn tới câu trả lời tốt hơn.

Một prompt có thể được tạo ra bằng một **khuôn mẫu prompt** (*prompt template*) (gọi tắt là **khuôn mẫu** (*template*)) [Liu và cộng sự, 2023b]. Khuôn mẫu là một đoạn văn bản chứa các chỗ trống (*placeholder*) hay biến, trong đó mỗi chỗ trống có thể được điền bằng một thông tin cụ thể. Sau đây là hai khuôn mẫu để hỏi LLM về gợi ý cho cuối tuần.

> Hãy cho tôi vài gợi ý để có một cuối tuần vui vẻ.

> Nếu {∗premise∗}, bạn có gợi ý gì cho một cuối tuần vui vẻ.

Ở khuôn mẫu thứ nhất, ta chỉ đơn giản yêu cầu LLM đưa ra vài gợi ý. Vì vậy khuôn mẫu này chỉ là một đoạn văn bản không có biến nào. Ở khuôn mẫu thứ hai, biến {∗premise∗} (*tiền đề*) cần được người dùng chỉ định để cung cấp một tiền đề cho việc đưa ra gợi ý. Ví dụ, nếu ta nhập

$$\text{premise} = \text{thời tiết cuối tuần này đẹp}, \tag{9.1}$$

thì ta có thể sinh ra prompt

> Nếu thời tiết cuối tuần này đẹp,
> bạn sẽ gợi ý gì cho một cuối tuần vui vẻ?

Ta cũng có thể thiết kế một khuôn mẫu có nhiều biến. Dưới đây là một ví dụ trong đó ta so sánh độ tương đồng ngữ nghĩa giữa hai câu.

Đây là một câu
{∗sentence1∗}
Đây là một câu khác
{∗sentence2∗}
Hãy tính độ tương đồng ngữ nghĩa giữa hai câu

Chú giải của người dịch: Khuôn mẫu giống như "mẫu đơn" điền thông tin

Khuôn mẫu prompt giống một tờ đơn in sẵn: phần chữ cố định đã viết, còn các ô trống như {∗sentence1∗}, {∗sentence2∗} (câu 1, câu 2) thì mỗi lần dùng ta điền nội dung khác nhau. Nhờ đó, một khuôn mẫu có thể tạo ra hàng nghìn prompt cho hàng nghìn cặp câu khác nhau mà không phải viết lại từ đầu. Ký hiệu {∗…∗} trong sách chỉ đánh dấu chỗ trống, không phải là chữ thực sự đưa vào mô hình.

Một cách phổ biến để định dạng prompt là viết mỗi đầu vào hoặc đầu ra theo kiểu "tên: nội dung". Ví dụ, ta có thể mô tả một cuộc hội thoại giữa hai người tên John và David, rồi dùng LLM để viết tiếp cuộc hội thoại. Sau đây là một khuôn mẫu cho những prompt như vậy.

John: {∗utterance1∗}
David: {∗utterance2∗}
John: {∗utterance3∗}
David: {∗utterance4∗}
John: {∗utterance5∗}
David: {∗utterance6∗}
John: {∗utterance7∗}
David: ___

(*utterance* nghĩa là "lượt lời".)

Định dạng "tên: nội dung" có thể được dùng để định nghĩa nhiệm vụ mà ta muốn LLM thực hiện. Ví dụ, vì "Q" và "A" là các chữ viết tắt thường dùng của "Question" (câu hỏi) và "Answer" (câu trả lời), ta có thể dùng khuôn mẫu sau để thực hiện nhiệm vụ hỏi–đáp.

Q: {∗question∗}
A: ___

Định dạng này có thể được dùng để mô tả những nhiệm vụ phức tạp hơn. Chẳng hạn, sau đây là một ví dụ cung cấp đặc tả cho một nhiệm vụ dịch:

> Nhiệm vụ: Dịch
> Ngôn ngữ nguồn: Tiếng Anh
> Ngôn ngữ đích: Tiếng Trung
> Văn phong: Văn bản trang trọng
> Khuôn mẫu: Dịch câu sau: {∗sentence∗}

Trong các hệ thống thực tế, người ta thường biểu diễn và lưu trữ những dữ liệu như vậy dưới dạng các cặp khóa–giá trị, chẳng hạn JSON[1].

Chú giải của người dịch: Cặp khóa–giá trị và JSON

Một *cặp khóa–giá trị* gồm một tên trường (*khóa*) và nội dung của trường đó (*giá trị*), ví dụ khóa "Ngôn ngữ đích" có giá trị "Tiếng Trung" — giống như các dòng "Họ và tên: …", "Ngày sinh: …" trong sơ yếu lý lịch. JSON là một định dạng văn bản rất phổ biến trong lập trình để lưu những cặp như vậy, theo kiểu `"khóa": "giá trị"` đặt trong cặp ngoặc nhọn.

Khi vấn đề khó mô tả theo kiểu liệt kê thuộc tính như trên, người ta thường hướng dẫn LLM bằng một mô tả rõ ràng và chi tiết. Có nhiều cách để làm điều này. Một ví dụ là gán cho LLM một vai trò và cung cấp đủ ngữ cảnh. Sau đây là một khuôn mẫu hướng dẫn LLM đóng vai một chuyên gia và trả lời câu hỏi của trẻ em.

> Bạn là một nhà khoa học máy tính có kiến thức sâu rộng trong lĩnh vực học sâu.
> Hãy giải thích khái niệm liên quan đến máy tính sau đây cho một em bé

[1] Biểu diễn JSON là

```
{
   "Task": "Translation",
   "Source language": "English",
   "Target language": "Chinese",
   "Style": "Formal text",
   "Template": "Translate the following sentence: {∗sentence∗}",
}
```

(Các trường lần lượt là: nhiệm vụ, ngôn ngữ nguồn, ngôn ngữ đích, văn phong, khuôn mẫu.)

khoảng 10 tuổi, dùng các ví dụ đơn giản bất cứ khi nào có thể.
{∗concept∗}
___

Đoạn văn bản "Bạn là một nhà khoa học máy tính … học sâu." đôi khi được gọi là prompt hệ thống (*system prompt*) (hay *thông tin hệ thống*), và được cung cấp để giúp LLM hiểu ngữ cảnh hoặc các ràng buộc của nhiệm vụ mà nó được yêu cầu thực hiện.

Chú giải của người dịch: Prompt hệ thống

Trong các ứng dụng trò chuyện như ChatGPT, prompt hệ thống thường là phần "lời dặn" đặt trước, do nhà phát triển viết sẵn (người dùng thường không nhìn thấy), ví dụ: "Bạn là trợ lý học tập, luôn trả lời ngắn gọn bằng tiếng Việt". Phần câu hỏi người dùng gõ vào sau đó mới là *prompt người dùng*. Prompt hệ thống giống như việc giáo viên dặn trước "Hôm nay em đóng vai hướng dẫn viên du lịch" rồi mới đặt câu hỏi.

9.1.2 Học trong ngữ cảnh

Việc học có thể diễn ra ngay trong lúc suy luận (*inference*) ở các mô hình ngôn ngữ lớn. Học trong ngữ cảnh là một phương pháp như vậy, trong đó prompt chứa các ví dụ minh họa cách giải quyết vấn đề, và LLM có thể học cách giải các bài toán mới từ những minh họa này. Vì ta không cập nhật tham số của mô hình trong quá trình này, học trong ngữ cảnh có thể được xem là một cách kích hoạt hiệu quả tri thức đã học trong giai đoạn tiền huấn luyện mà không cần huấn luyện hay tinh chỉnh thêm. Điều này cho phép LLM nhanh chóng thích nghi với các bài toán mới và mở rộng năng lực của các mô hình tiền huấn luyện mà không cần điều chỉnh riêng cho từng nhiệm vụ.

Học trong ngữ cảnh có thể được minh họa bằng cách so sánh ba phương pháp: học zero-shot (*zero-shot learning*), học one-shot (*one-shot learning*) và học few-shot (*few-shot learning*). Học zero-shot, đúng như tên gọi, không bao gồm một quá trình "học" theo nghĩa truyền thống. Thay vào đó, nó dùng trực tiếp LLM để giải quyết các bài toán mới chưa từng được quan sát trong quá trình huấn luyện. Trong thực tế, ta có thể điều chỉnh prompt lặp đi lặp lại để dẫn dắt LLM sinh ra câu trả lời tốt

hơn, mà không cần minh họa các bước giải hay đưa ra ví dụ. Hãy xem ví dụ sau. Giả sử ta muốn dùng LLM làm trợ lý giúp sửa các câu tiếng Anh. Một prompt học zero-shot như sau

SYSTEM	Bạn là một trợ lý hữu ích, và rất giỏi sửa lỗi ngữ pháp.
USER	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: She don't like going to the park. Output: ___

Ở đây các chữ màu xám được dùng để chỉ các trường khác nhau của prompt.

Chú giải của người dịch: Zero-shot, one-shot, few-shot

"Shot" ở đây có thể hiểu là "ví dụ mẫu". *Zero-shot*: không cho ví dụ nào, chỉ mô tả nhiệm vụ. *One-shot*: cho đúng một ví dụ mẫu. *Few-shot*: cho một vài ví dụ mẫu. Các nhãn xám trong hộp: SYSTEM (prompt hệ thống), USER (phần của người dùng), DEMO (*demonstration* — ví dụ minh họa). Các câu tiếng Anh được giữ nguyên vì nhiệm vụ chính là sửa ngữ pháp tiếng Anh; "Input" là câu vào, "Output" là câu ra. Câu *She don't like going to the park* ("Cô ấy không thích đi công viên") sai vì chủ ngữ ngôi thứ ba số ít *she* phải đi với *doesn't*; câu đúng là *She doesn't like going to the park*.

Trong học one-shot, ta mở rộng prompt này bằng cách thêm một ví dụ minh họa cách sửa câu, nhờ đó cho phép LLM học từ kinh nghiệm mới được bổ sung này.

SYSTEM	Bạn là một trợ lý hữu ích, và rất giỏi sửa lỗi ngữ pháp.
DEMO	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: There is many reasons to celebrate. Output: There are many reasons to celebrate.
USER	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: She don't like going to the park. Output: ___

Hơn nữa, ta có thể thêm nhiều ví dụ minh họa hơn để thực hiện học few-shot.

SYSTEM	Bạn là một trợ lý hữu ích, và rất giỏi sửa lỗi ngữ pháp.
DEMO1	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: There is many reasons to celebrate. Output: There are many reasons to celebrate.
DEMO2	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: Me and my friend goes to the gym every day. Output: My friend and I go to the gym every day.
USER	Bạn sẽ được cung cấp một câu tiếng Anh. Nhiệm vụ là xuất ra câu đúng. Input: She don't like going to the park. Output: ___

(Nghĩa các câu ví dụ: "Có nhiều lý do để ăn mừng"; "Tôi và bạn tôi đến phòng tập thể dục mỗi ngày".)

Trong học few-shot, về bản chất ta cung cấp một khuôn mẫu ánh xạ một số đầu vào sang các đầu ra tương ứng. LLM cố gắng làm theo khuôn mẫu này khi đưa ra dự đoán, miễn là prompt chứa đủ số lượng ví dụ minh họa (dù số lượng này thường nhỏ). Ta cũng có thể dùng những khuôn mẫu đơn giản hơn để đạt được điều này. Ví dụ, có thể dùng prompt học few-shot sau để dịch các từ từ tiếng Trung sang tiếng Anh.

DEMO	现在	→	now
	来	→	come
	去	→	go
	男孩	→	boy
USER	女孩	→	___

(Nghĩa tiếng Việt: 现在 – now – "bây giờ"; 来 – come – "đến"; 去 – go – "đi"; 男孩 – boy – "bé trai"; 女孩 – "bé gái", đáp án mong đợi là *girl*.)

Nếu LLM đủ mạnh, học few-shot có thể giúp nó giải quyết các bài toán phức tạp, chẳng hạn lập luận toán học. Ví dụ, hãy xét nhiệm vụ sau:

cộng hai số rồi chia tổng cho tích của chúng.

DEMO	12 5	→	$(12+5)/(12\times 5) = 0.283$
	3 1	→	$(3+1)/(3\times 1) = 1.33$
	−9 4	→	$(-9+4)/(-9\times 4) = 0.138$
	15 15	→	$(15+15)/(15\times 15) = 0.133$
USER	19 73	→	___

Chú giải của người dịch: Mô hình phải tự "đoán ra" quy tắc

Trong prompt trên, ta không hề nói "hãy cộng rồi chia cho tích"; mô hình phải tự nhận ra quy tắc từ bốn ví dụ. Chẳng hạn $(-9+4)/(-9\times 4) = (-5)/(-36) \approx 0.138$. Câu trả lời mong đợi cho cặp 19 và 73 là $(19+73)/(19\times 73) = 92/1387 \approx 0.0663$. Ở đây dấu chấm là dấu thập phân (viết theo kiểu quốc tế như sách gốc), tức 0.283 chính là $0{,}283$ theo cách viết ở Việt Nam.

Trong nhiều ứng dụng thực tế, hiệu quả của học trong ngữ cảnh phụ thuộc nhiều vào chất lượng của prompt và năng lực nền tảng của LLM đã tiền huấn luyện. Một mặt, cần nhiều công sức thiết kế prompt để xây dựng được những prompt phù hợp giúp LLM học hiệu quả hơn từ các ví dụ minh họa. Mặt khác, những LLM mạnh hơn có thể tận dụng học trong ngữ cảnh tốt hơn để thực hiện nhiệm vụ mới. Ví dụ, giả sử ta muốn dùng một LLM để dịch các từ từ tiếng Inuktitut sang tiếng Anh. Nếu LLM không được tiền huấn luyện trên dữ liệu tiếng Inuktitut, hiểu biết của nó về tiếng Inuktitut sẽ yếu, và mô hình sẽ khó dịch tốt cho dù ta prompt nó như thế nào. Trong trường hợp này, ta cần tiếp tục huấn luyện LLM với nhiều dữ liệu tiếng Inuktitut hơn, thay vì cố tìm những prompt tốt hơn. [2]

Một câu hỏi thú vị nữa là học trong ngữ cảnh hình thành như thế nào trong quá trình tiền huấn luyện và vì sao nó hoạt động được lúc suy luận. Một cách giải thích trực quan là: dù LLM đã thu được tri thức giải quyết vấn đề trong quá trình tiền huấn luyện, không gian đầu ra quá lớn khiến mô hình khó chọn được dự đoán đúng cho các nhiệm vụ mới lạ. Việc cung cấp ví dụ minh họa sẽ dẫn LLM đi theo những con đường "đúng". Hơn nữa, một số nhà nghiên cứu đã cố gắng diễn giải học trong ngữ cảnh từ nhiều

[2] **[ND]** Inuktitut là ngôn ngữ của người Inuit ở vùng Bắc Cực Canada; đây là một ngôn ngữ có rất ít dữ liệu văn bản trên Internet.

góc nhìn khác nhau, bao gồm suy luận Bayes (*Bayesian inference*) [Xie và cộng sự, 2022], hạ gradient (*gradient descent*) [Dai và cộng sự, 2023; Von Oswald và cộng sự, 2023], hồi quy tuyến tính (*linear regression*) [Akyürek và cộng sự, 2023], học cách học (*meta-learning*) [Garg và cộng sự, 2022], v.v.

9.1.3 Các chiến lược thiết kế prompt

Thiết kế prompt mang tính thực nghiệm rất cao. Nói chung, có nhiều cách để prompt một LLM thực hiện cùng một nhiệm vụ, và ta thường phải thử–sai nhiều lần mới tìm được một prompt vừa ý. Để viết prompt tốt hiệu quả hơn, ta có thể tuân theo một số chiến lược. Ví dụ về các nguyên tắc prompting phổ biến gồm:

- **Mô tả nhiệm vụ càng rõ ràng càng tốt.** Khi áp dụng LLM để giải quyết một vấn đề, ta cần mô tả vấn đề một cách chính xác, cụ thể, rõ ràng và hướng dẫn LLM hành xử như ta mong muốn. Điều này đặc biệt quan trọng khi ta muốn đầu ra của LLM đáp ứng những kỳ vọng nhất định. Ví dụ, giả sử ta tò mò về biến đổi khí hậu. Một prompt đơn giản để yêu cầu LLM cung cấp thông tin là

  > Hãy kể cho tôi về biến đổi khí hậu.

  Vì chỉ dẫn này quá chung chung, LLM có thể sinh ra câu trả lời đề cập đến bất kỳ khía cạnh nào của biến đổi khí hậu, có thể không khớp với mối quan tâm cụ thể của ta. Trong trường hợp này, ta có thể dùng những prompt cụ thể và chi tiết hơn. Một ví dụ như vậy là

  > Hãy giải thích chi tiết các nguyên nhân và hệ quả của biến đổi khí hậu, bao gồm tác động lên nhiệt độ toàn cầu, các kiểu thời tiết và mực nước biển. Đồng thời, hãy thảo luận các giải pháp khả thi và những hành động đang được thực hiện để giảm nhẹ các tác động này.

  Bây giờ giả sử ta định giải thích về biến đổi khí hậu cho một em bé 10 tuổi. Ta có thể điều chỉnh thêm prompt ở trên.

> Hãy giải thích các nguyên nhân và hệ quả của biến đổi khí hậu cho một em bé 10 tuổi. Nói về việc nó ảnh hưởng thế nào đến thời tiết, mực nước biển và nhiệt độ. Đồng thời, hãy nhắc đến một số việc mọi người đang làm để giúp đỡ. Cố gắng giải thích bằng ngôn ngữ đơn giản và không vượt quá 500 từ.

- **Dẫn dắt LLM suy nghĩ.** LLM đã thể hiện một khả năng "suy nghĩ" mạnh đến bất ngờ. Một ví dụ thường gặp là các LLM được phát triển tốt đã đạt hiệu năng ấn tượng trong các nhiệm vụ lập luận toán học, vốn được xem là khó. Trong thiết kế prompt, khả năng suy nghĩ của LLM cần được kích hoạt bằng cách prompt phù hợp, đặc biệt với các bài toán đòi hỏi nhiều công sức suy luận. Trong nhiều trường hợp, một LLM được yêu cầu "suy nghĩ" có thể cho ra kết quả hoàn toàn khác so với khi yêu cầu chính mô hình đó thực hiện nhiệm vụ trực tiếp. Ví dụ, Kojima và cộng sự [2022] phát hiện rằng chỉ cần thêm câu "Let's think step by step" ("Hãy suy nghĩ từng bước một") vào cuối mỗi prompt là có thể cải thiện hiệu năng của LLM trên nhiều nhiệm vụ lập luận. Có nhiều cách để prompt LLM "suy nghĩ". Một phương pháp là yêu cầu LLM sinh ra các bước lập luận về bài toán trước khi đi đến câu trả lời cuối cùng. Ví dụ, hãy xét nhiệm vụ giải toán. Sau đây là một prompt đơn giản cho nhiệm vụ này.

> Bạn là một nhà toán học. Bạn sẽ được cung cấp một bài toán. Hãy giải bài toán đó.

Vì giải toán đòi hỏi một quá trình lập luận chi tiết, LLM có lẽ sẽ mắc lỗi nếu cố đưa ra đáp án ngay. Vì vậy, ta có thể yêu cầu tường minh LLM tuân theo một quy trình suy luận cho trước trước khi đi đến kết luận.

> Bạn là một nhà toán học. Bạn sẽ tuân theo các bước lập luận chi tiết sau khi giải toán.
>
> **Bước 1: Diễn giải bài toán.**
> Nhà toán học lắng nghe kỹ câu hỏi của bạn và hiểu các chi tiết phức tạp của thử thách toán học mà bạn đưa ra.
>
> **Bước 2: Xây dựng chiến lược.**

Dựa trên vốn kiến thức sâu rộng của mình, nhà toán học chọn chiến lược hiệu quả nhất phù hợp với loại bài toán, dù đó là đại số, giải tích hay hình học.

Bước 3: Tính toán chi tiết.
Với sự chính xác và chuyên môn, nhà toán học thực hiện các phép tính cần thiết từng bước một, tuân thủ mọi nguyên tắc toán học.

Bước 4: Rà soát lời giải.
Trước khi đưa ra đáp án cuối cùng, nhà toán học kiểm tra tỉ mỉ các phép tính để đảm bảo độ chính xác và đưa ra lời giải thích hoặc lập luận ngắn gọn cho lời giải.

Bạn sẽ được cung cấp một bài toán. Hãy giải bài toán đó.
{∗problem∗}

Một phương pháp khác để dẫn dắt LLM "suy nghĩ" là thông qua nhiều lượt tương tác với LLM. Ví dụ, ở bước đầu tiên, ta có thể yêu cầu LLM giải bài toán trực tiếp

Bạn sẽ được cung cấp một bài toán. Hãy giải bài toán đó.
{∗problem∗}

Bây giờ ta đã có một đáp án ban đầu cho bài toán. Ở bước thứ hai, ta prompt LLM đánh giá tính đúng đắn của đáp án và, nếu cần, làm lại để tìm ra lời giải tốt hơn.

Bạn sẽ được cung cấp một bài toán, cùng với một lời giải. Hãy đánh giá tính đúng đắn của lời giải này, và chỉ ra các lỗi nếu có. Sau đó, hãy tự giải bài toán theo cách của bạn.
Bài toán: {∗problem∗}
Lời giải: {∗solution∗}

Các prompt được trình bày ở đây liên quan chặt chẽ đến một dòng nghiên cứu dài về các bài toán lập luận (*reasoning*) trong LLM. Không thể thảo luận đầy đủ mọi vấn đề liên quan vì chủ đề này bao trùm một họ phương pháp rất lớn. Nhưng ta sẽ thấy một thảo luận tương đối chi tiết hơn về cách cải thiện prompting thông qua lập luận nhiều hơn ở Mục 9.2.

Chú giải của người dịch: Vì sao "suy nghĩ từng bước" lại giúp ích?

LLM sinh văn bản lần lượt từng token, token sau dựa vào các token đã viết trước đó. Nếu bắt mô hình viết ngay đáp số, nó phải "nhẩm" toàn bộ lời giải trong một bước — giống như học sinh làm bài toán nhiều bước mà chỉ được ghi đáp số. Khi được yêu cầu viết ra các bước trung gian, mỗi bước viết ra lại trở thành ngữ cảnh giúp bước sau dễ đúng hơn, tương tự như ta làm nháp. Cách "hai lượt" (giải trước, rồi tự kiểm tra) thì giống thói quen dò lại bài trước khi nộp.

- **Cung cấp thông tin tham khảo.** Như đã thảo luận ở mục trước, ta có thể đưa các ví dụ minh họa vào prompt và cho phép LLM học trong ngữ cảnh từ những minh họa này cách thực hiện nhiệm vụ. Thực ra, với khả năng hiểu ngôn ngữ đáng kể của LLM, ta có thể thêm bất kỳ loại văn bản nào vào prompt để các mô hình này dự đoán dựa trên ngữ cảnh phong phú hơn. Trong nhiều ứng dụng, ta có nhiều loại thông tin liên quan đến truy vấn của người dùng. Thay vì dùng LLM để dự đoán không ràng buộc, ta thường muốn LLM tạo ra đầu ra gói gọn trong phạm vi văn bản liên quan. Một ví dụ như vậy là RAG[3], trong đó văn bản liên quan đến truy vấn của người dùng được truy xuất bằng một hệ thống truy xuất thông tin (*information retrieval, IR*), rồi ta prompt LLM sinh ra câu trả lời dựa trên văn bản liên quan được cung cấp này. Prompt sau đây cho thấy một ví dụ.

> Bạn là một chuyên gia có thể sinh câu trả lời cho các truy vấn đầu vào. Bây giờ bạn được cung cấp một truy vấn và thông tin ngữ cảnh tương ứng. Hãy sinh câu trả lời dựa trên thông tin ngữ cảnh này. Lưu ý rằng bạn cần đưa ra câu trả lời bằng lời lẽ của chính mình, không chỉ sao chép từ ngữ cảnh được cung cấp.
> Thông tin ngữ cảnh: {∗IR-result∗}
> Truy vấn: {∗query∗}

  Nếu thông tin ngữ cảnh rất đáng tin cậy, ta thậm chí có thể giới hạn LLM chỉ được trả lời bằng văn bản được cung cấp. Một prompt ví dụ như sau

[3] **[ND]** RAG: sinh văn bản tăng cường truy xuất (*retrieval-augmented generation*).

Bạn là một chuyên gia có nhiệm vụ sinh câu trả lời từ các truy vấn đầu vào. Bạn được cung cấp một truy vấn và thông tin ngữ cảnh tương ứng, được sắp xếp thành một bảng trong đó mỗi hàng là một bản ghi hữu ích. Hãy sinh câu trả lời chỉ dùng thông tin ngữ cảnh này. Hãy đảm bảo bạn đưa ra câu trả lời bằng lời lẽ của chính mình.
Thông tin ngữ cảnh: {∗table∗}
Truy vấn: {∗query∗}

Chú giải của người dịch: RAG — "làm bài có tài liệu"

RAG giống như một bài kiểm tra được mở sách: trước khi trả lời, hệ thống tìm kiếm (giống một máy tìm kiếm như Google) lấy ra vài đoạn văn bản liên quan đến câu hỏi ({∗IR-result∗}), rồi nhét chúng vào prompt. LLM đọc tài liệu đó và trả lời dựa trên nó, thay vì chỉ dựa vào "trí nhớ" từ lúc tiền huấn luyện — nhờ vậy câu trả lời cập nhật hơn và ít bịa đặt hơn.

Khi xử lý các bài toán thực tế, ta thường có tri thức tiên nghiệm và thông tin bổ sung về bài toán giúp tạo ra câu trả lời tốt hơn. Việc đưa những thông tin như vậy vào prompt nói chung có ích cho việc cải thiện kết quả.

- **Chú ý đến định dạng của prompt.** Nói chung, hiệu năng của LLM rất nhạy cảm với prompt mà ta đưa vào. Đôi khi một thay đổi nhỏ trong prompt có thể dẫn đến thay đổi lớn ở đầu ra của mô hình. Một ví dụ thú vị là việc thay đổi thứ tự các câu trong prompt có thể khiến LLM sinh ra kết quả khác. Để prompt dễ đọc và giảm sự mơ hồ, người ta thường định dạng chúng sao cho rõ ràng. Một ví dụ là ta định nghĩa vài trường cho prompt và điền thông tin khác nhau vào mỗi trường. Một ví dụ khác là ta có thể dùng prompt theo kiểu mã lệnh cho những LLM có thể hiểu và sinh ra cả ngôn ngữ tự nhiên lẫn mã. Xem prompt kiểu mã lệnh sau đây thực hiện việc dịch, trong đó có một ví dụ minh họa.

[English] = [I have an apple.]
[German] = [Ich habe einen Apfel.]
[English] = [I have an orange.]
[German] = ___

(English: tiếng Anh; German: tiếng Đức. Hai câu tiếng Anh nghĩa là "Tôi có một quả táo" và "Tôi có một quả cam".)

LLM có thể nhận văn bản ở nhiều định dạng khác nhau. Điều này cho phép ta dùng các ký tự điều khiển, thẻ XML, và các định dạng đặc thù để biểu diễn dữ liệu phức tạp. Hơn nữa, sẽ hữu ích nếu ta chỉ rõ đầu vào và đầu ra nên được định dạng hay cấu trúc như thế nào. Ví dụ, ta có thể phân tách các phần văn bản bằng dấu ngoặc kép và prompt LLM cho phù hợp (chẳng hạn thêm vào prompt một câu như "văn bản đầu vào được đặt giữa hai dấu ngoặc kép").

Ở trên, ta mới bàn về một vài chiến lược để viết prompt tốt. Tất nhiên còn rất nhiều phương pháp như vậy, và mỗi người cần tự phát triển phương pháp của mình qua thực hành. Bạn đọc quan tâm có thể tham khảo nhiều tài liệu trực tuyến để biết thêm thông tin, chẳng hạn hướng dẫn của OpenAI về các mô hình dòng GPT[4].

9.1.4 Thêm các ví dụ

Trong tiểu mục này, ta xét thêm các ví dụ về prompt LLM để thực hiện nhiều nhiệm vụ NLP khác nhau. Mục đích ở đây không phải là đưa ra các prompt chuẩn cho các nhiệm vụ này, mà là dùng những ví dụ đơn giản để minh họa cách prompt LLM nhằm giải quyết các bài toán NLP.

1. Phân loại văn bản

Phân loại văn bản (*text classification*) có lẽ là một trong những bài toán phổ biến nhất trong NLP. Nhiều nhiệm vụ có thể được xếp chung vào loại gán các nhãn định sẵn cho một văn bản cho trước. Ở đây, ta xét bài toán phân loại cực cảm xúc trong phân tích cảm xúc (*sentiment analysis*). Ta chọn phân loại cực cảm xúc để minh họa vì đây là một trong những nhiệm vụ phân loại văn bản phổ biến nhất và được định nghĩa rõ ràng nhất. Trong một thiết lập tổng quát cho phân loại cực cảm xúc, mục tiêu là xếp một văn bản cho trước vào một trong ba loại: tiêu cực, tích cực hoặc trung tính. Dưới đây là một prompt đơn giản cho nhiệm vụ này (để dễ đọc, chúng tôi tô sáng phần mô tả nhiệm vụ trong prompt).

[4] Xem `https://platform.openai.com/docs/guides/prompt-engineering/six-strategies-for-getting-better-results`.

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản có thể được phân loại là tiêu cực.

Để đầy đủ, chúng tôi đưa ra cả câu trả lời do LLM sinh ra (phần chữ được gạch dưới).

Mặc dù câu trả lời là đúng, LLM lại tạo ra một câu mô tả thay vì một nhãn rời rạc. Khó khăn này nảy sinh vì về bản chất LLM được thiết kế để sinh văn bản chứ không phải để gán nhãn rời rạc. Do đó, phân loại được xem như một bài toán sinh văn bản. Kết quả là ta cần một hệ thống khác để ánh xạ đầu ra của LLM vào không gian nhãn (một quá trình gọi là **ánh xạ nhãn** (*label mapping*)). Chẳng hạn, ta trích ra "tiêu cực" từ câu "Cực cảm xúc của văn bản có thể được phân loại là tiêu cực". Việc này đơn giản trong hầu hết các trường hợp vì ta có thể nhận ra các từ nhãn bằng những quy tắc kinh nghiệm (*heuristics*) đơn giản. Nhưng thỉnh thoảng LLM có thể không diễn đạt kết quả phân loại bằng các từ nhãn này. Khi đó bài toán trở nên phức tạp hơn, vì ta cần một ánh xạ từ văn bản được sinh ra sang các nhãn định sẵn.

Một phương pháp để thu được nhãn đầu ra từ LLM là phát biểu lại bài toán thành một nhiệm vụ điền vào chỗ trống (*cloze task*). Ví dụ, sau đây là một prompt kiểu điền vào chỗ trống cho phân loại cực cảm xúc.

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản là tiêu cực

Ta có thể dùng LLM để hoàn thành văn bản và điền vào chỗ trống từ phù hợp nhất. Lý tưởng nhất, từ được điền phải là positive (tích cực), negative (tiêu cực) hoặc neutral (trung tính). Tuy nhiên, không có gì bảo đảm LLM sẽ sinh ra đúng các từ nhãn này. Một phương pháp để giải quyết vấn đề này là giới hạn không gian đầu ra vào tập các từ nhãn và chọn từ

có xác suất cao nhất. Khi đó, nhãn đầu ra được cho bởi

$$\text{label} = \arg\max_{y \in Y} \Pr(y|\mathbf{x}), \tag{9.2}$$

trong đó y ký hiệu token (hoặc từ) được điền vào chỗ trống, và Y ký hiệu tập các từ nhãn $\{\text{positive}, \text{negative}, \text{neutral}\}$.

Chú giải của người dịch: Đọc Phương trình (9.2)

$\arg\max_{y \in Y}$ nghĩa là "chọn phần tử y trong tập Y làm cho biểu thức phía sau lớn nhất". Ví dụ, sau khi đọc prompt $\mathbf{x}$, giả sử mô hình gán xác suất cho từ tiếp theo là: "tiêu cực" 0.62, "trung tính" 0.20, "tích cực" 0.03, còn lại 0.15 dành cho mọi từ khác (như "khá", "hơi",…). Ta bỏ qua các từ khác, chỉ so sánh ba từ nhãn, và chọn "tiêu cực" vì nó có xác suất lớn nhất. Nhờ vậy đầu ra luôn là một trong ba nhãn hợp lệ.

Một cách tiếp cận khác để sinh nhãn bằng LLM là ràng buộc đầu ra bằng prompt. Ví dụ, ta có thể prompt LLM dự đoán từ một tập nhãn định sẵn. Đây là một ví dụ.

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản là gì?
Chỉ trả lời: tích cực, tiêu cực, hoặc trung tính.

Tiêu cực

Phân tích cảm xúc là một bài toán NLP phổ biến mà LLM nhìn chung đã "hiểu" khá tốt thông qua tiền huấn luyện hoặc tinh chỉnh. Do đó, ta có thể prompt LLM bằng những chỉ dẫn đơn giản để thực hiện nhiệm vụ. Tuy nhiên, với những bài toán phân loại mới, có thể cần cung cấp thêm chi tiết về nhiệm vụ, chẳng hạn các tiêu chuẩn phân loại, để LLM có thể làm đúng. Để làm điều này, ta có thể thêm một mô tả nhiệm vụ chi tiết hơn và/hoặc minh họa các ví dụ phân loại trong prompt. Để minh họa, hãy xét ví dụ sau.

Hãy phân tích cực cảm xúc của văn bản sau và phân loại nó là tích cực, tiêu cực hoặc trung tính. Dưới đây là ý nghĩa của từng loại:
Tích cực: Văn bản truyền tải một cảm xúc hoặc thái độ tích cực. Ví dụ, các văn bản thể hiện niềm vui, sự hài lòng, sự hào hứng hoặc sự ngưỡng mộ được coi là tích cực.
Tiêu cực: Văn bản thể hiện một cảm xúc hoặc thái độ tiêu cực. Nó bao gồm các cảm xúc buồn bã, tức giận, bực bội hoặc chỉ trích.
Trung tính: Cảm xúc trung tính được dùng để mô tả những văn bản không thể hiện rõ cảm xúc tích cực hay tiêu cực mà mang giọng điệu cung cấp thông tin, nêu sự thật, hoặc dửng dưng.

Văn bản:
Dịch vụ ở nhà hàng chậm hơn dự kiến, điều này khiến tôi hơi bực mình.

Cực cảm xúc của văn bản là gì?

Tiêu cực

Mặc dù có vẻ dùng LLM cho các bài toán phân loại là đơn giản, vẫn còn những vấn đề chưa được giải quyết tốt. Ví dụ, khi phải xử lý một số lượng lớn các lớp, việc prompt LLM một cách hiệu quả vẫn còn là thách thức. Lưu ý rằng nếu ta gặp một bài toán phân loại rất khó và có một lượng dữ liệu được gán nhãn nhất định, thì việc tinh chỉnh LLM hoặc dùng các kiến trúc kiểu "BERT + bộ phân loại" cũng là lựa chọn đáng cân nhắc.

2. Trích xuất thông tin

Nhiều nhiệm vụ NLP có thể được phát biểu thành các bài toán trích xuất thông tin (*information extraction*), tức là xác định và trích ra những mẩu thông tin cụ thể từ văn bản phi cấu trúc. Những thông tin này bao gồm các thực thể có tên (*named entity*), các quan hệ, các sự kiện và các điểm dữ liệu liên quan khác. Mục tiêu của trích xuất thông tin là chuyển dữ liệu thô thành một định dạng có cấu trúc để có thể dễ dàng phân tích và sử dụng trong nhiều ứng dụng hạ nguồn.

Vì trích xuất thông tin bao trùm rất nhiều bài toán, ta không thể bàn hết ở đây. Thay vào đó, ta bắt đầu với nhiệm vụ nhận dạng thực thể có tên (*named entity recognition*) — một nhiệm vụ từ lâu đã là trọng tâm trong NLP. Nhận dạng thực thể có tên là nhiệm vụ xác định và phân loại các thực thể có tên trong văn bản. Những thực thể này thường bao gồm danh từ riêng và các biểu thức khác như ngày tháng, giá trị tiền tệ và tỉ lệ phần

trăm. Trước tiên, hãy xét một ví dụ đơn giản về trích xuất tên người từ một văn bản cho trước[5].

Hãy xác định tất cả tên người trong văn bản được cung cấp.

Văn bản:
Liệu nước Anh có thực sự đang làm ăn tệ đến vậy, hay xu hướng du lịch đã thay đổi sau đại dịch? Theo Tom Jenkins, CEO của European Tourism Organisation, thì là điều thứ hai. "Tôi nghĩ UK vẫn đang làm rất tốt nhưng chúng ta sẽ thấy nhiều người đi đến châu Âu (Europe) hơn," ông nói về năm 2024, đồng thời cho biết "nghe nói nhu cầu của người Mỹ đối với UK đang hơi chững lại."

Tên người được xác định trong văn bản là:
Tom Jenkins

Chú giải của người dịch: Thực thể có tên

Thực thể có tên là những "đối tượng cụ thể có tên gọi riêng": người (Tom Jenkins), địa điểm (UK — Vương quốc Anh, Europe — châu Âu), tổ chức (European Tourism Organisation — Tổ chức Du lịch châu Âu), thời gian (2024)… Nhận dạng thực thể có tên giống như việc dùng bút dạ quang tô các tên riêng trong một bài báo rồi ghi bên cạnh đó là người, nơi chốn hay tổ chức. Trong bản dịch các ví dụ, chúng tôi giữ nguyên tên riêng tiếng Anh để khớp với kết quả mà LLM trả về trong sách gốc.

Tiếp theo, ta có thể mở rộng phạm vi để trích xuất mọi thực thể có tên.

Hãy xác định và phân loại tất cả các thực thể có tên trong văn bản được cung cấp vào các loại như tên người, địa điểm, ngày tháng và tổ chức. Liệt kê mỗi thực thể cùng với loại của nó trên một dòng.

Văn bản:
Liệu nước Anh có thực sự đang làm ăn tệ đến vậy, hay xu hướng du lịch đã thay đổi sau đại dịch? Theo Tom Jenkins, CEO của European Tourism Organisation, thì là điều thứ hai. "Tôi nghĩ UK vẫn đang làm rất tốt nhưng chúng ta sẽ thấy nhiều người đi đến châu Âu (Europe) hơn," ông nói về năm 2024, đồng thời cho biết "nghe nói nhu cầu của người Mỹ đối với UK đang hơi chững lại."

[5] Văn bản lấy từ `https://edition.cnn.com/travel`.

1. Tom Jenkins – Tên người
2. European Tourism Organisation – Tổ chức
3. UK – Địa điểm
4. Europe – Địa điểm
5. 2024 – Ngày tháng

Với các thực thể có tên đã xác định được, ta có thể áp dụng các nhiệm vụ trích xuất thông tin khác lên văn bản. Ví dụ, ta có thể xác định quan hệ giữa các thực thể có tên này (gọi là trích xuất quan hệ (*relation extraction*)). Prompt tương ứng được cho dưới đây.

Cho một văn bản và danh sách các thực thể có tên đã được xác định trong đó, hãy phân tích và mô tả các quan hệ giữa những thực thể này. Giải thích mỗi thực thể liên quan đến các thực thể khác như thế nào trong ngữ cảnh.

Văn bản:
Liệu nước Anh có thực sự đang làm ăn tệ đến vậy, hay xu hướng du lịch đã thay đổi sau đại dịch? Theo Tom Jenkins, CEO của European Tourism Organisation, thì là điều thứ hai. "Tôi nghĩ UK vẫn đang làm rất tốt nhưng chúng ta sẽ thấy nhiều người đi đến châu Âu (Europe) hơn," ông nói về năm 2024, đồng thời cho biết "nghe nói nhu cầu của người Mỹ đối với UK đang hơi chững lại."

Thực thể có tên:
Tom Jenkins, European Tourism Organisation, UK, Europe, 2024

Tom Jenkins – European Tourism Organisation: Tom Jenkins là CEO của European Tourism Organisation. Điều này cho thấy một mối liên hệ nghề nghiệp, trong đó Tom Jenkins có lẽ giữ vai trò quan trọng trong việc điều hành các hoạt động của tổ chức, đặc biệt là những hoạt động liên quan đến du lịch ở châu Âu.
…

Nếu một LLM đã được tinh chỉnh để làm theo chỉ dẫn, việc điều chỉnh nó cho các nhiệm vụ trích xuất thông tin khác nhau là tương đối đơn giản. Ví dụ, sau đây là một khuôn mẫu prompt tổng quát cho trích xuất thông tin.

Bạn sẽ được cung cấp một văn bản. Nhiệm vụ của bạn là {∗task-description∗}

Văn bản: {∗text∗}

___

Ở đây task-description (mô tả nhiệm vụ) là chỉ dẫn cho nhiệm vụ. Ví dụ về các chỉ dẫn như vậy gồm

> *trích xuất các từ khóa từ văn bản.*
> *trích xuất các sự kiện chính từ văn bản.*
> *phát hiện các liên kết đồng tham chiếu và hồi chỉ giữa các thực thể trong văn bản.*

Thông thường, cung cấp một mô tả nhiệm vụ chi tiết hơn sẽ giúp LLM đưa ra câu trả lời chính xác hơn.

Chú giải của người dịch: Đồng tham chiếu và hồi chỉ

Hai cách diễn đạt *đồng tham chiếu* (*coreference*) khi chúng cùng chỉ một đối tượng. Trong văn bản ví dụ ở trên, "Tom Jenkins" và "ông" (trong "ông nói về năm 2024") cùng chỉ một người. *Hồi chỉ* (*anaphora*) là trường hợp một từ (thường là đại từ như "ông", "nó", "họ") quay lại chỉ một đối tượng đã được nhắc đến trước đó.

3. Sinh văn bản

Trong NLP, thuật ngữ *sinh văn bản* (*text generation*) được dùng theo nghĩa rộng, bao trùm nhiều loại bài toán, nhiều trong số đó tập trung vào việc sinh nội dung mới dựa trên một đầu vào. Có thể tạm chia các bài toán này thành hai nhóm: 1) viết tiếp từ một văn bản đầu vào ban đầu (gọi là **hoàn thành văn bản** (*text completion*)), và 2) chuyển văn bản đầu vào thành một văn bản khác (gọi là **biến đổi văn bản** (*text transformation*)). Ở đây, ta trình bày các ví dụ prompt cho nhóm thứ nhất và sẽ bàn về nhóm thứ hai ở phần sau.

Một ví dụ đơn giản của hoàn thành văn bản là viết tiếp một câu chuyện bắt đầu từ vài câu mở đầu.

Bạn là một nhà văn rất giỏi viết truyện. Bạn được cung cấp vài câu mở đầu. Hãy viết tiếp câu chuyện từ những câu đã cho.

Mở đầu câu chuyện:
Một buổi chiều nắng đẹp, Lily phát hiện một chiếc chìa khóa nhỏ xíu bằng vàng giấu dưới một hòn đá trong sân sau nhà mình. Tò mò, cô bé nhặt nó lên và tự hỏi nó có thể mở được thứ gì. Đúng lúc đó, chú mèo Whiskers của cô bắt đầu kêu meo meo đầy phấn khích, dẫn cô về phía cây sồi già ở rìa khu vườn.

Lily đi theo Whiskers, tim đập rộn ràng vì phấn khích xen chút hồi hộp. Cây sồi già lúc nào cũng có vẻ bí ẩn đối với cô, những cành cây xương xẩu của nó đổ những chiếc bóng dài ngay cả vào những ngày nắng nhất. Khi họ đến bên gốc cây, Whiskers lao lên trước và bắt đầu cào vào một thứ gì đó bị vùi dưới đống lá mùa thu …

Tương tự, ta có thể hoàn thành một cuộc hội thoại khi biết ngữ cảnh hội thoại.

Bạn được cung cấp một cuộc hội thoại giữa một du khách và một tài xế taxi. Hãy hoàn thành cuộc hội thoại.

Du khách: Anh có thể chở tôi đến bảo tàng ở trung tâm thành phố được không?
Tài xế taxi: Tất nhiên rồi! Lần đầu anh đến thành phố này à?
Du khách: Vâng, quanh đây có chỗ nào nhất định phải ghé không?
Tài xế taxi: Nhất định anh nên ghé vườn bách thảo sau khi đi bảo tàng. Nó ở gần đó và khá đẹp.

Du khách: Tuyệt quá, cảm ơn anh đã mách nhé!

Tài xế taxi: Không có gì. Chúc anh tham quan vui vẻ!

Ngoài việc viết tiếp đơn thuần, mô hình (khuôn khổ) này có thể được mở rộng cho các nhiệm vụ sinh văn bản theo chỉ dẫn, trong đó mô hình "hoàn thành" một câu trả lời dựa trên yêu cầu của người dùng. Những yêu cầu này có thể quy định văn phong, giọng điệu, độ dài và nội dung mong muốn. Chẳng hạn, người dùng có thể yêu cầu một bản báo cáo trang trọng, một câu chuyện sáng tạo, hoặc một đoạn mã chương trình. Dưới đây là một ví dụ prompt để sáng tác một bài thơ dựa trên các ràng buộc cho trước.

Bạn là một nhà thơ. Hãy viết một bài thơ theo các yêu cầu sau.

Chủ đề: Sự hồi sinh của mùa xuân
Cấu trúc: Thơ bốn câu với nhịp điệu đều đặn
Cảm xúc: Truyền tải cảm giác hạnh phúc và đổi mới qua những hình ảnh mùa xuân sống động.

Xuân thức dậy dưới nắng vàng rực rỡ
Hoa tươi non nhảy múa giữa ban mai
Đồng xanh thì thầm khúc ca vui khẽ
Hy vọng mới bừng lên theo gió xuân

(Bài thơ trong gốc là thơ tiếng Anh do LLM sáng tác; ở đây chúng tôi dịch nghĩa sang tiếng Việt.)

Hơn nữa, khi một LLM được tiền huấn luyện hoặc tinh chỉnh trên một kho ngữ liệu bao gồm cả ngôn ngữ tự nhiên lẫn ngôn ngữ lập trình, ta có thể prompt nó thực hiện các nhiệm vụ sinh mã từ văn bản. Trong trường hợp này, mô hình đơn giản là "dịch" các chỉ dẫn bằng ngôn ngữ tự nhiên thành mã chạy được. Ví dụ sau cho thấy cách prompt LLM viết một hàm Python.

Hãy viết một hàm Python để tính trung bình cộng của một danh sách các số.

```
def calculate_average(numbers):
    if numbers:  # Kiểm tra danh sách có rỗng không
        return sum(numbers) / len(numbers)
    else:
        return 0  # Trả về 0 nếu danh sách rỗng
```

Chú giải của người dịch: Đọc đoạn mã Python

Hàm `calculate_average` ("tính trung bình") nhận vào một danh sách số `numbers`. Nếu danh sách không rỗng, nó trả về `sum(numbers) / len(numbers)`, tức tổng các số chia cho số lượng số — đúng công thức trung bình cộng ở THPT; ví dụ với $[2, 4, 9]$ kết quả là $15/3 = 5$. Nếu danh sách rỗng thì trả về 0 để tránh phép chia cho 0. Phần sau dấu # là lời chú thích cho người đọc, máy tính bỏ qua (trong gốc chú thích viết bằng tiếng

Anh).

4. Biến đổi văn bản

Các nhiệm vụ biến đổi văn bản điển hình gồm dịch máy (*machine translation*), tóm tắt (*summarization*), chuyển đổi văn phong (*text style transfer*), v.v. Các prompt dùng để thực hiện những nhiệm vụ này không khác biệt đáng kể so với các prompt đã trình bày trước đó. Ở đây, ta đưa ra vài ví dụ để minh họa các prompt này.

Hãy dịch văn bản sau từ tiếng Anh sang tiếng Tây Ban Nha.

Văn bản:
The quick brown fox jumps over the lazy dog.

El rápido zorro marrón salta sobre el perro perezoso.

(Nghĩa: "Con cáo nâu nhanh nhẹn nhảy qua con chó lười biếng." Câu tiếng Anh và câu tiếng Tây Ban Nha được giữ nguyên vì đây là ví dụ dịch giữa hai ngôn ngữ này.)

Hãy tóm tắt bài viết sau trong không quá 50 từ:

Bài viết:
Trong những năm gần đây, các khu vực đô thị đang phải đối mặt với những thách thức đáng kể liên quan đến tính bền vững và quản lý môi trường. Các thành phố đang chịu mức ô nhiễm cao hơn, tình trạng tắc nghẽn giao thông gia tăng và nhu cầu lớn hơn về cơ sở hạ tầng. Sự tăng trưởng này đã dẫn đến nhiều vấn đề môi trường, bao gồm mức độ ô nhiễm không khí và nước tăng cao, lượng rác thải gia tăng, và các dịch vụ công bị quá tải …

Các khu vực đô thị đang vật lộn với những thách thức về tính bền vững, như ô nhiễm gia tăng, tắc nghẽn giao thông và nhu cầu về cơ sở hạ tầng …

Hãy viết lại văn bản này với giọng điệu trang trọng.

Văn bản:
Ê, dạo này sao rồi? Lâu quá không gặp!

Xin chào, dạo này bạn thế nào?

Đã khá lâu rồi kể từ lần cuối chúng ta gặp nhau!

5. Hỏi đáp

Trong hỏi đáp (*question answering*), một hệ thống nhận một câu hỏi và được kỳ vọng đưa ra một câu trả lời phù hợp. Các khuôn mẫu prompt cho hỏi đáp đa mục đích có thể là:

{∗question∗}

___

Câu hỏi: {∗question∗}
Trả lời: ___

Q: {∗question∗}
A: ___

Hỏi đáp quan trọng trong NLP vì nhiều bài toán có thể được phát biểu dưới dạng nhiệm vụ hỏi đáp. Đặc biệt, nhiều nhiệm vụ lập luận gần đây được định nghĩa dưới dạng hỏi đáp. Ví dụ, trong bộ đánh giá chuẩn MMLU [Hendrycks và cộng sự, 2021a], mỗi mẫu gồm một câu hỏi trắc nghiệm, và LLM được yêu cầu chọn đáp án đúng. Sau đây là một prompt ví dụ để trả lời một câu hỏi trong tập dữ liệu này.

$(\mathbb{Z}, *)$ là một nhóm với $a * b = a + b + 1$ với mọi a, b thuộc $\mathbb{Z}$. Phần tử nghịch đảo của a là
(A) 0
(B) -2
(C) $a - 2$
(D) $(2 + a) * -1$

D

Chú giải của người dịch: Câu hỏi MMLU ở trên nói gì?

Đây là một câu hỏi đại số trình độ đại học. $\mathbb{Z}$ là tập số nguyên, và ta định nghĩa một "phép toán mới" $a * b = a + b + 1$ (ví dụ $2 * 3 = 6$). "Phần tử

đơn vị" e là số thỏa $a * e = a$, tức $a + e + 1 = a$, nên $e = -1$. "Phần tử nghịch đảo" của a là số b thỏa $a * b = e = -1$, tức $a + b + 1 = -1$, suy ra $b = -a - 2$. Đáp án (D): $(2 + a) * (-1) = (2 + a) + (-1) + 1 = 2 + a$; nếu hiểu biểu thức là $(2 + a) \cdot (-1)$ (nhân thông thường) thì được $-a - 2$, khớp với kết quả trên — đó là cách hiểu cho đáp án D mà đề này dùng. Bạn không cần nắm phần toán này để theo dõi chương; điều quan trọng là LLM chỉ cần trả lời bằng một chữ cái.

Một bộ đánh giá chuẩn khác được dùng rộng rãi là tập dữ liệu GSM8K [Cobbe và cộng sự, 2021]. Nó gồm hàng nghìn bài toán đố (có lời văn) trình độ tiểu học. Với mỗi bài toán, ta dùng LLM để sinh ra lời giải bằng ngôn ngữ tự nhiên. Ví dụ, sau đây là một prompt dùng để giải các bài toán trong GSM8K.

DEMO

Q: Hai cô con gái của Jerry chơi bóng mềm ở hai đội khác nhau. Mùa giải này mỗi đội có 8 trận đấu. Mỗi đội tập luyện 4 giờ cho mỗi trận đấu. Nếu mỗi trận kéo dài 2 giờ, thì tổng cộng Jerry sẽ dành bao nhiêu giờ ở sân để xem các con gái thi đấu và tập luyện?

A: Jerry sẽ dành 8 trận * 2 giờ mỗi trận $=\ll 8 * 2 = 16 \gg$ 16 giờ để xem một cô con gái thi đấu. Ông sẽ dành $16 * 2 =\ll 16 * 2 = 32 \gg$ 32 giờ để xem cả hai con gái thi đấu. Ông sẽ dành 8 trận * 4 giờ tập $=\ll 8 * 4 = 32 \gg$ 32 giờ để xem một cô con gái tập luyện. Ông sẽ dành $32 * 2 =\ll 32 * 2 = 64 \gg$ 64 giờ để xem cả hai con gái tập luyện. Ông sẽ dành tổng cộng 32 giờ xem thi đấu + 64 giờ xem tập luyện $=\ll 32 + 64 = 96 \gg$ 96 giờ. #### 96

Q: Mary mua sáu quả táo ở cửa hàng. Từ số táo đã mua, cứ mỗi quả Mary ăn, cô lại trồng hai cây từ những quả còn lại. Mary đã ăn bao nhiêu quả táo?

A: Cô đã trồng tám cây. Điều này nghĩa là cô đã dùng một nửa số đó là táo, tức là 8 cây $/2$ cây/quả táo $=\ll 8/2 = 4 \gg$ 4 quả táo. Nghĩa là cô đã trồng bốn trong sáu quả táo đã mua, chỉ còn lại 6 quả táo $-$ 4 quả táo $=\ll 6 - 4 = 2 \gg$ 2 quả táo để ăn. #### 2

USER Q: Boris có 100 quả táo. Beck có ít hơn Boris 23 quả táo. Nếu Boris cho Beck 10 quả táo, thì bây giờ Beck có ít hơn Boris bao nhiêu quả táo?

A: Boris bắt đầu với 100 quả táo và cho Beck 10 quả táo, còn lại 100 quả táo -10 quả táo $=\ll 100-10=90\gg$ 90 quả táo. Ban đầu, Beck có ít hơn Boris 23 quả táo, nên số táo ban đầu của Beck là $100-23=\ll 100-23=77\gg$ 77 quả táo. Sau khi nhận 10 quả táo từ Boris, tổng số táo của Beck là 77 quả táo $+10$ quả táo $=\ll 77+10=87\gg$ 87 quả táo. Do đó, bây giờ Beck có ít hơn Boris $90-87=\ll 90-87=3\gg$ 3 quả táo. #### 3

Ở đây, một prompt few-shot được sử dụng. Từ các ví dụ minh họa gồm những cặp bài toán–lời giải này, LLM không chỉ học cách giải bài toán mà còn học cả cách định dạng đầu ra. Ví dụ, kết quả cuối cùng của phép tính xuất hiện sau token ####, và $\ll \cdots \gg$ chú thích các bước tính toán chi tiết (gọi là chú thích tính toán (*calculation annotation*))[6].

Chú giải của người dịch: Ví dụ few-shot vừa dạy cách giải, vừa dạy cách trình bày

Hai bài mẫu (DEMO) cho mô hình thấy một "khuôn trình bày" cố định: viết lời giải từng bước, mỗi phép tính được bọc trong $\ll \gg$, và đáp số cuối cùng đặt sau ####. Nhờ khuôn cố định này, một chương trình đơn giản có thể tự động lấy đáp số (chỉ cần đọc phần sau ####) để chấm điểm. Còn các phép tính trong $\ll \gg$ có thể được giao cho máy tính tính hộ thay vì để LLM "tính nhẩm" — vì LLM vốn dự đoán chữ chứ không thực sự làm phép tính, nên đôi khi tính sai những phép tính đơn giản. Ở câu hỏi của USER, phần gạch dưới là lời giải do LLM sinh ra, theo đúng khuôn của hai bài mẫu.

[6]Trong quá trình dự đoán, một máy tính bỏ túi (chương trình tính toán) được dùng khi ta gặp $\ll \cdots \gg$. Cụ thể hơn, mỗi khi LLM gặp dấu "=" bên trong một $\ll \cdots \gg$, máy tính sẽ tính biểu thức ở vế trái của "=". Phương pháp này giúp giảm các lỗi tính toán do LLM gây ra.

9.2 Các phương pháp prompting nâng cao

Cho đến đây trong chương này, chúng ta đã giới thiệu các khái niệm cơ bản liên quan đến việc prompt LLM và trình bày một số prompt cho các nhiệm vụ NLP. Giờ ta xem xét một vài kỹ thuật giúp nâng cao hiệu quả của prompting.

9.2.1 Chuỗi suy nghĩ

Chúng ta đã nhiều lần gặp khái niệm **chuỗi suy nghĩ** (*chain of thought*) (**CoT**) trong chương này và các chương trước [Wei và cộng sự, 2022c; Chowdhery và cộng sự, 2022]. Các phương pháp CoT cung cấp một cách đơn giản để prompt LLM sinh ra quá trình lập luận từng bước cho các bài toán phức tạp, nhờ đó tiếp cận nhiệm vụ theo cách giống con người hơn. Thay vì đi thẳng tới kết luận, các phương pháp CoT hướng dẫn LLM sinh ra các bước lập luận, hoặc học từ các ví dụ minh họa quá trình lập luận chi tiết. Để minh họa CoT, ta xét bài toán tính toán đại số, như thường được mô tả trong các tài liệu. Giả sử ta có một bài toán đại số

Tính trung bình cộng của các số 2, 4 và 9.

Ta có thể đặt nó thành một câu hỏi để prompt LLM.

> Q: Hãy tính trung bình cộng của các số 2, 4 và 9.
> A: Đáp án là 6.

Với những nhiệm vụ lập luận không tầm thường, việc prompt trực tiếp một LLM không phải lúc nào cũng cho đáp án đúng. Một cải tiến đơn giản là thêm vào prompt các ví dụ minh họa của những bài toán tương tự, nhờ đó LLM có thể học từ các ví dụ này.

> Q: Hãy tính trung bình cộng của các số 1, 3, 5 và 7.
> A: Đáp án là 4.
>
> Q: Hãy tính trung bình cộng của các số 2, 4 và 9.
> A: Đáp án là 7.

Vấn đề ở đây là mặc dù ta đã cho thấy một cặp câu hỏi–đáp án tương tự, LLM vẫn khó lập luận ra đáp án đúng. Trong CoT, LLM không chỉ học từ sự tương ứng giữa câu hỏi và đáp án mà còn được lợi từ các bước giải chi tiết dùng để đi tới đáp án. Để làm điều này, ta có thể đưa một số bước lập luận vào prompt để có được một *prompt CoT*.

Q: Hãy tính trung bình bình phương của các số 1, 3, 5 và 7.
A: Tính bình phương của từng số: $1^2 = 1$, $3^2 = 9$, $5^2 = 25$ và $7^2 = 49$. Cộng các bình phương lại, $1 + 9 + 25 + 49 = 84$. Có tất cả 4 số. Chia tổng cho số phần tử, $84/4 = 21$. Đáp án là 21.

Q: Hãy tính trung bình cộng của các số 2, 4 và 9.
A: Tính $2 + 4 + 9$, kết quả bằng 15. Có ba số. Chia tổng cho số lượng các số, được $15/3 = 5$. Đáp án là 5.

Ở đây các bước lập luận được tô màu xanh lá. Nhờ được cung cấp các ví dụ minh họa lập luận chi tiết, LLM được khuyến khích sinh ra các bước trung gian dẫn tới đáp án đúng.

Chú giải của người dịch: Vì sao thêm các bước giải lại giúp LLM?

Ở prompt thứ hai, LLM chỉ thấy "câu hỏi → đáp số" nên nó phải "đoán" đáp số trong một bước (và đoán sai: 7). Trung bình cộng đúng là $(2+4+9)/3 = 15/3 = 5$. Khi bài mẫu trình bày cả cách làm (cộng lại, đếm số phần tử, chia), LLM bắt chước viết ra từng bước, và mỗi bước nhỏ thì dễ làm đúng hơn nhiều. Vì LLM sinh văn bản lần lượt từng token, các bước đã viết ra trở thành một phần ngữ cảnh giúp nó dự đoán đáp số cuối cùng — giống như học sinh làm nháp ra giấy thay vì tính nhẩm. Lưu ý rằng bài mẫu (trung bình bình phương) không giống hệt bài cần giải; mô hình học *kiểu lập luận* chứ không chép đáp số.

Việc dùng prompting CoT có một số lợi ích. Thứ nhất, CoT cho phép LLM phân rã các bài toán phức tạp thành những bước lập luận nhỏ hơn, nối tiếp nhau. Điều này giống cách con người giải quyết bài toán phức tạp từng bước một, nên nó đặc biệt hiệu quả với những nhiệm vụ đòi hỏi suy luận chi tiết, nhiều bước. Thứ hai, CoT làm cho quá trình lập luận minh bạch và dễ diễn giải hơn. Vì các bước lập luận trung gian đều hiện ra, ta có thể hiểu rõ hơn kết luận đã được rút ra như thế nào. Thứ ba, nếu người

dùng theo dõi được logic đằng sau quá trình lập luận, họ sẽ có xu hướng tin tưởng các dự đoán của LLM hơn. Điều này đặc biệt quan trọng khi áp dụng LLM trong các lĩnh vực như y tế, giáo dục và tài chính. Thứ tư, CoT là một cách tiếp cận học trong ngữ cảnh, vì vậy nó áp dụng được cho hầu hết các LLM có sẵn đã được huấn luyện tốt. Hơn nữa, CoT cung cấp những cách hiệu quả để thích nghi LLM với các loại bài toán khác nhau. Nó thậm chí có thể gợi mở những lời giải sáng tạo hơn bằng cách khám phá nhiều đường lập luận thay thế khác nhau, những đường mà có thể không hiển nhiên khi đi thẳng tới kết luận.

Phương pháp mô tả ở trên đòi hỏi cung cấp một hoặc nhiều ví dụ lập luận CoT, thường được gọi là phương pháp *CoT few-shot*. Ngược lại, phương pháp *CoT zero-shot* không cần các ví dụ như vậy. Thay vào đó, nó prompt LLM lập luận từng bước bằng cách đưa các chỉ dẫn cụ thể vào prompt. Ví dụ, dưới đây là một prompt CoT zero-shot.

Q: Hãy tính trung bình cộng của các số 2, 4 và 9.

A: Hãy suy nghĩ từng bước một.

Ta có ba số: 2, 4 và 9. Cộng các số này lại, $2 + 4 + 9 = 15$. Xác định có bao nhiêu số, trong trường hợp này là ba. Trung bình cộng được tính bằng cách chia tổng cho số phần tử. Thực hiện phép chia được $15/3 = 5$. Vậy đáp án là 5.

Theo chỉ dẫn "Hãy suy nghĩ từng bước một" (*Let's think step by step*), LLM được prompt để sinh ra các bước lập luận chi tiết. Như đã bàn trong công trình của Kojima và cộng sự [2022], prompt bằng những chỉ dẫn như vậy có thể khiến LLM chỉ sinh ra các bước lập luận mà không có kết luận rõ ràng. Trong trường hợp này, có thể dùng một vòng prompting thứ hai để trích xuất đáp án từ các bước lập luận đó. Chẳng hạn, Kojima và cộng sự [2022] tạo một prompt thứ hai kết hợp cả đầu vào lẫn đầu ra của vòng prompting thứ nhất. Dùng đầu vào kết hợp này, LLM có thể tiếp tục quá trình lập luận rồi sinh ra đáp án đúng. Hơn nữa, có thể prompt LLM lập luận bằng những chỉ dẫn khác ngoài "Hãy suy nghĩ từng bước một", chẳng hạn "Hãy suy nghĩ một cách logic" và "Hãy cho tôi xem các bước suy nghĩ của bạn trước".

Mặc dù ta đã minh họa các phương pháp CoT bằng một bài toán lập luận đại số, các phương pháp này có thể áp dụng cho nhiều loại bài toán. Các ứng dụng điển hình của CoT gồm lập luận toán học, lập luận logic,

lập luận thường thức, lập luận ký hiệu, sinh mã, v.v. Xem Hình 9.1 để có thêm các ví dụ về việc áp dụng CoT trong nhiều nhiệm vụ khác nhau.

CoT là một trong những chủ đề nghiên cứu sôi động nhất của thiết kế prompt. Nó không chỉ cải thiện hiệu năng của việc prompt LLM mà còn thúc đẩy rất nhiều nghiên cứu về đánh giá và kiểm chứng năng lực lập luận của LLM. Mặc dù trong mục này ta tập trung vào ý tưởng cơ bản của CoT, nó có thể được cải tiến theo nhiều cách. Chẳng hạn, ta có thể xem quá trình lập luận là bài toán tìm kiếm qua nhiều đường đi khả dĩ, mỗi đường có thể gồm nhiều trạng thái trung gian (tức các bước lập luận). Nói chung, ta muốn không gian tìm kiếm được định nghĩa rõ ràng và đủ lớn, để có nhiều khả năng tìm được kết quả tối ưu hơn. Vì lý do này, một hướng nghiên cứu LLM hiện nay nhằm thiết kế các cấu trúc tốt hơn để biểu diễn quá trình lập luận, cho phép LLM giải quyết những thử thách lập luận phức tạp hơn. Các cấu trúc này gồm cấu trúc dạng cây [Yao và cộng sự, 2024], cấu trúc dạng đồ thị [Besta và cộng sự, 2024], v.v. Nhờ dùng các biểu diễn gọn này cho các đường lập luận, LLM có thể khám phá một phạm vi rộng hơn các đường ra quyết định, tương tự như tư duy Hệ thống 2[7]. Một hướng nghiên cứu khác tập trung vào việc prompt LLM bằng các tương tác nhiều vòng. Điều này bao gồm phân rã bài toán phức tạp thành các bài toán con, kiểm chứng và tinh chỉnh đầu ra của mô hình, sử dụng kết hợp nhiều mô hình (*model ensembling*), v.v. Lưu ý rằng các phương pháp này và những vấn đề liên quan không chỉ giới hạn ở CoT. Thực tế, chúng thường được dùng như những cách tiếp cận tổng quát hơn để cải thiện LLM, còn CoT có thể được xem là một cách để kiểm tra năng lực của LLM. Ta sẽ thấy phần thảo luận về một số vấn đề này trong các tiểu mục tiếp theo.

Chú giải của người dịch: Suy luận như một bài toán tìm kiếm

Hãy hình dung việc giải một bài toán như đi trong mê cung: mỗi bước lập luận là một ngã rẽ, và một lời giải là một đường đi từ cửa vào (đề bài) tới cửa ra (đáp án). CoT thông thường chỉ đi *một* đường thẳng từ đầu đến cuối. Cấu

[7] Tư duy Hệ thống 1 và Hệ thống 2, như Kahneman [2011] mô tả, đại diện cho hai chế độ xử lý nhận thức khác nhau. Hệ thống 1 nhanh, tự động, trực giác và cảm tính. Chế độ tư duy này vận hành không tốn sức và nhanh chóng, thường là thứ dẫn dắt các quyết định, phán đoán và ấn tượng hằng ngày của chúng ta. Hệ thống 2 thì chậm, có chủ ý và mang tính phân tích. Nó được kích hoạt khi ta cần thực hiện những tính toán phức tạp.

trúc dạng *cây* cho phép tại mỗi bước thử vài khả năng khác nhau (rẽ nhánh) rồi bỏ các nhánh kém; cấu trúc dạng *đồ thị* còn cho phép gộp các nhánh lại với nhau. Cách làm này giống "Hệ thống 2" trong tâm lý học: khi giải một bài hình khó, ta không viết ngay đáp số theo cảm giác (Hệ thống 1) mà thử vài hướng, kiểm tra, quay lại khi đi vào ngõ cụt.

Trước khi kết thúc phần thảo luận về CoT, ta nên xét đến những hạn chế thực tế của nó. Một hạn chế lớn là trong các tình huống CoT few-shot cần có các ví dụ minh họa lập luận chi tiết, nhiều bước, vốn có thể khó thu thập, dù là tự động hay thủ công. Ngoài ra, không có phương pháp chuẩn nào để chia nhỏ bài toán phức tạp thành các bước giải đơn giản hơn. Việc này thường phụ thuộc nhiều vào kinh nghiệm của người dùng. Thêm nữa, lỗi ở các bước trung gian cũng có thể ảnh hưởng đến độ chính xác của kết luận cuối cùng. Để tìm hiểu thêm về ưu và nhược điểm của CoT, bạn đọc quan tâm có thể tham khảo các bài tổng quan gần đây về chủ đề này [Chu và cộng sự, 2023; Yu và cộng sự, 2023b; Zhang và cộng sự, 2023a].

Chú giải của người dịch: Bốn nhiệm vụ trong Hình 9.1

CSQA (*CommonsenseQA*) và StrategyQA là các bộ câu hỏi về *thường thức*: câu trả lời không nằm sẵn trong đề mà cần hiểu biết đời thường (tivi cần cáp, hamster là con mồi…). *Ngôn ngữ Dyck* là các chuỗi dấu ngoặc đóng mở đúng cách, như ([ ]); lời giải dùng một *ngăn xếp* (*stack*) — giống chồng đĩa: ngoặc mở nào vào sau thì phải được đóng trước. Với đầu vào "< [[", ngăn xếp là < [[, nên phải đóng lần lượt]] >. *Ghép chữ cái cuối* là nhiệm vụ lập luận ký hiệu: lấy chữ cái cuối của mỗi từ rồi nối lại; các tên riêng được giữ nguyên vì nhiệm vụ phụ thuộc vào chính chữ cái của chúng.

9.2.2 Phân rã bài toán

Ta đã thấy LLM có thể được lợi khi giải một bài toán phức tạp bằng cách chia nhỏ nó thành những bài toán con đơn giản hơn. Cách tiếp cận như vậy có thể xem là một ví dụ của một mô thức rộng hơn gọi là **phân rã bài toán** (*problem decomposition*), vốn đã được khám phá và bàn luận rộng rãi trong tâm lý học và khoa học máy tính. Dưới góc độ tâm lý học, giải quyết vấn đề phức tạp là quá trình xử lý một vấn đề bằng tri thức giúp vượt

Q: Thiết bị giải trí gia đình nào cần dùng cáp?
Các lựa chọn: (a) cửa hàng radio (b) trạm biến áp (c) tivi (d) tủ
A: Đáp án phải là thứ cần dùng cáp. Trong các lựa chọn trên, chỉ có tivi cần cáp. Vậy đáp án là (c).

Q: Kiểu người nào thường mắc bệnh?
Các lựa chọn: (a) bệnh viện (b) cái đầu (c) người ốm (d) người cao tuổi (e) phòng khám bác sĩ
A: Đáp án phải là một người dễ mắc bệnh. Trong các lựa chọn, "người cao tuổi" là đúng vì họ thường có hệ miễn dịch yếu hơn. Vậy đáp án là (d).

(a) CSQA

Q: Chuột hamster có là thức ăn cho loài động vật nào không?
A: Hamster là động vật bị săn mồi. Con mồi là thức ăn cho thú săn mồi. Do đó, hamster là thức ăn cho một số loài động vật. Vậy đáp án là có.

Q: Có thể dùng Microsoft Office mà không có internet không?
A: Có, bạn có thể dùng các ứng dụng Microsoft Office, mặc dù những tính năng như lưu trữ đám mây và cộng tác trực tuyến cần internet. Vậy đáp án là có.

(b) StrategyQA

Q: Hãy hoàn thành phần còn lại của chuỗi, bảo đảm các dấu ngoặc được đóng đúng cách. Đầu vào: [{
A: Hãy suy nghĩ từng bước một.
0: ngăn xếp rỗng
1: [; ngăn xếp: [
2: { ; ngăn xếp: [{
Vậy đáp án là }].

Q: Hãy hoàn thành phần còn lại của chuỗi, bảo đảm các dấu ngoặc được đóng đúng cách. Đầu vào: < [[
A: Hãy suy nghĩ từng bước một.
0: ngăn xếp rỗng
1: < ; ngăn xếp: <
2: [; ngăn xếp: < [
3: [; ngăn xếp: < [[
Vậy đáp án là]] >.

(c) Ngôn ngữ Dyck

Q: Lấy các chữ cái cuối của các từ trong "Elon Musk" rồi ghép chúng lại.
A: Chữ cái cuối của "Elon" là "n". Chữ cái cuối của "Musk" là "k". Ghép chúng lại được "nk". Đáp án là nk.

Q: Lấy các chữ cái cuối của từng từ trong "Renata Mariela Mona Kristin" rồi ghép chúng lại.
A: Chữ cái cuối của "Renata" là "a". Chữ cái cuối của "Mariela" là "a". Chữ cái cuối của "Mona" là "a". Chữ cái cuối của "Kristin" là "n". Ghép chúng lại được "aaan". Đáp án là aaan.

(d) Ghép chữ cái cuối

Hình 9.1: CoT trong bốn nhiệm vụ lập luận khác nhau, gồm CSQA, StrategyQA, ngôn ngữ Dyck (*Dyck languages*) và ghép chữ cái cuối (*Last Letter Concatenation*). Các phần CoT được tô màu xanh lá.

qua các rào cản của vấn đề đó[8]. Thường không có con đường rõ ràng nào để giải một vấn đề phức tạp. Tuy nhiên, việc dùng các chiến lược phân rã vấn đề thường có lợi, vì chúng giúp xử lý các bài toán con tương ứng dễ dàng hơn và ít tốn công hơn. Chẳng hạn, hãy xét việc viết một bài blog về các rủi ro của AI. Nếu ta chỉ đơn giản prompt LLM bằng chỉ dẫn "Hãy viết một bài blog về các rủi ro của AI", LLM có thể sinh ra một bài blog với cấu trúc và văn phong tùy ý. Một cách tốt hơn là lập dàn ý cho bài blog và cung cấp thông tin chi tiết hơn về từng phần. Hãy xét prompt sau

Bạn là một người viết blog. Hãy làm theo dàn ý được cung cấp dưới đây để viết một bài blog về các rủi ro của AI.

- Mở đầu
  Giới thiệu về AI, tầm quan trọng của nó, và tầm quan trọng của việc hiểu các rủi ro của nó đối với giới trẻ.
- Lo ngại về quyền riêng tư
  Bàn về việc AI có thể xâm phạm quyền riêng tư cá nhân như thế nào thông qua các tương tác trực tuyến.
- Thông tin sai lệch
  Tìm hiểu vai trò của AI trong việc lan truyền thông tin sai lệch và tác động đến các quyết định của người trẻ.
- Bắt nạt trên mạng
  Làm nổi bật cách các công cụ AI có thể bị lợi dụng để bắt nạt trên mạng và tác động của việc này đến sức khỏe tinh thần.
- Lời khuyên để dùng AI an toàn
  Đưa ra các hướng dẫn sử dụng AI có trách nhiệm và khuyến khích tư duy phản biện.
- Kết luận
  Tóm tắt các ý chính và khuyến khích chủ động quan tâm đến đạo đức AI.

—

Ở đây ta đưa ra tiêu đề và các ý chính cho từng phần. Sau đó, LLM có thể dùng cấu trúc này để chia nhỏ nhiệm vụ viết bằng cách điền nội dung cho các phần ấy. Lưu ý rằng cách cấu trúc bài blog có thể do con người

[8]Một định nghĩa tương đối hình thức có thể tìm thấy trong sách của Frensch và Funke [2014]: *giải quyết vấn đề phức tạp diễn ra nhằm vượt qua các rào cản giữa một trạng thái cho trước và một trạng thái mục tiêu mong muốn thông qua các hoạt động hành vi và/hoặc nhận thức gồm nhiều bước.*

cung cấp, hoặc thậm chí được sinh tự động. Ví dụ, ta có thể dùng LLM để sinh dàn ý trước, rồi yêu cầu nó làm theo dàn ý đó để hoàn thành bài viết.

Trong khoa học máy tính, phân rã bài toán phức tạp là một chiến lược thường dùng trong thiết kế hệ thống phần mềm và phần cứng. Một ví dụ nổi tiếng là mô thức chia để trị (*divide-and-conquer*), thường được dùng để thiết kế thuật toán cho những bài toán tính toán có thể quy về các bài toán đơn giản hơn, dễ xử lý hơn. Ví dụ, xét bài toán xác định xem một tài liệu có bàn về các rủi ro của AI hay không. Ta có thể chỉ dẫn LLM bằng prompt sau.

> Bạn được cung cấp một văn bản. Hãy xác định xem văn bản đó có bàn về các rủi ro của AI hay không.
>
> {∗document∗}
>
> ___

Nếu tài liệu dài, việc suy luận có thể trở nên tốn kém về tính toán. Thay vào đó, ta có thể chia tài liệu thành các đoạn tương đối ngắn và thực hiện cùng một nhiệm vụ trên từng đoạn. Các đoạn này có thể được xử lý song song để giảm thêm chi phí tính toán. Tiếp theo, ta xác định mức độ liên quan của từng đoạn tới chủ đề rủi ro của AI. Đầu ra cuối cùng sau đó được sinh ra bằng một prompt khác.

> Nhiệm vụ của bạn là xác định xem một văn bản có bàn về các rủi ro của AI hay không. Văn bản này đã được chia thành các đoạn, và bạn đã có mức độ liên quan của từng đoạn tới chủ đề rủi ro của AI. Dựa vào đó, hãy đưa ra kết quả cuối cùng của bạn.
>
> Đoạn 1: {∗relevance-to-the-topic1∗}
> Đoạn 2: {∗relevance-to-the-topic2∗}
> Đoạn 3: {∗relevance-to-the-topic3∗}
> …
>
> ___

Chú giải của người dịch: Chia để trị

"Chia để trị" là một ý tưởng quen thuộc: muốn làm một việc lớn, ta chia nó thành nhiều việc nhỏ cùng loại, làm từng việc nhỏ rồi ghép kết quả lại. Ví

dụ, muốn biết một cuốn sách 300 trang có nói về rủi ro của AI hay không, ta có thể nhờ 10 bạn mỗi người đọc 30 trang (làm *song song*, cùng lúc), mỗi bạn báo lại "phần của mình có liên quan hay không"; cuối cùng chỉ cần đọc 10 bản báo cáo ngắn để kết luận. Với LLM, chi phí xử lý tăng nhanh theo độ dài văn bản đầu vào (với Transformer, chi phí tự chú ý tăng theo bình phương độ dài), nên chia nhỏ văn bản thường rẻ hơn nhiều. Trong prompt thứ hai, {∗relevance-to-the-topic1∗} (*mức độ liên quan tới chủ đề* của đoạn 1) là chỗ trống sẽ được điền bằng kết quả của bước trước.

Giờ hãy quay lại phần thảo luận tổng quát hơn về phân rã bài toán trong prompting. Mặc dù phân rã bài toán có thể áp dụng cho nhiều bài toán NLP khác nhau, gần đây nó được thảo luận và thử nghiệm rộng rãi hơn cả trong các nhiệm vụ lập luận. Với các nhiệm vụ lập luận phức tạp, ta thường cần một đường lập luận nhiều bước để đi tới kết luận đúng. Ta có thể dùng LLM để thực hiện lập luận như vậy theo ba cách khác nhau. Thứ nhất, LLM có thể đi thẳng tới kết luận. Nói cách khác, chúng có thể dự đoán mà không có quá trình lập luận tường minh, và quá trình lập luận bên dưới vẫn ẩn và không diễn giải được. Thứ hai, LLM được prompt để sinh ra một đường lập luận nhiều bước dẫn tới kết luận, giống như CoT. Tuy nhiên, ta chỉ chạy LLM một lần, và mọi bước trung gian của quá trình lập luận được sinh ra trong một lần dự đoán duy nhất. Thứ ba, ta chia bài toán ban đầu thành một số bài toán con, mỗi bài toán con được xử lý trong các lần chạy LLM riêng biệt hoặc được giải bằng các hệ thống khác. Ở đây ta tập trung vào cách thứ ba, vốn liên quan chặt chẽ đến phân rã bài toán. Tuy nhiên cần lưu ý rằng một thảo luận đầy đủ hơn có thể bao quát cả ba cách này, còn hai cách đầu đã được bàn tới ở mức độ nào đó trong chương này.

Một khung tổng quát cho phân rã bài toán gồm hai thành phần.

- **Sinh bài toán con** (*Sub-problem Generation*). Thành phần này phân rã bài toán đầu vào thành một số bài toán con.

- **Giải bài toán con** (*Sub-problem Solving*). Thành phần này giải từng bài toán con và rút ra các kết luận trung gian cũng như kết luận cuối cùng thông qua suy luận.

Hai vấn đề này có thể được mô hình hóa theo những cách khác nhau, dẫn tới nhiều phương pháp phân rã bài toán khác nhau. Một cách tiếp cận

là coi chúng là hai bước riêng biệt trong một quy trình hai bước. Chẳng hạn, xét nhiệm vụ viết blog đã mô tả ở đầu tiểu mục này. Ở bước thứ nhất, ta phân rã toàn bộ bài toán thành các bài toán con cùng một lúc (tức là lập dàn ý cho bài blog). Ở bước thứ hai, ta giải các bài toán con theo thứ tự lần lượt hoặc theo một thứ tự khác (tức là điền nội dung cho từng phần khi cần). Đầu ra cuối cùng của quy trình này kết hợp các kết quả giải từng bài toán con. Mặc dù phương pháp này đơn giản và dễ hiểu, nó giả định rằng bài toán có tính *kết hợp* (*compositional*) — tức lời giải của cả bài được ghép từ lời giải của các phần — nên nó phù hợp hơn với những nhiệm vụ như viết văn và sinh mã.

Tuy nhiên, nhiều bài toán trong thực tế đòi hỏi lập luận phức tạp. Một đặc điểm then chốt của những bài toán này là đường lập luận của chúng không được cố định trước và có thể phụ thuộc vào các kết quả trung gian thu được trong quá trình lập luận. Trong những trường hợp đó, việc sinh trước các bài toán con một cách cố định là không mong muốn. Thay vào đó, các bài toán con nên được sinh ra một cách động dựa trên bài toán đầu vào, và nếu có thể, được sinh ngay trong lúc suy luận. Điều này khiến phân rã bài toán khó khăn hơn so với thiết kế các thuật toán chia để trị. Lý tưởng nhất, ta muốn thiết kế đồng thời cả hai hệ thống sinh bài toán con và giải bài toán con. Nhưng một cách tiếp cận thực tế hơn và được dùng rộng rãi là dùng các mô hình riêng cho các nhiệm vụ này. Một cách đơn giản để làm được điều đó là thích nghi một LLM cho các nhiệm vụ này bằng cách prompt hoặc tinh chỉnh mô hình.

Ở đây ta xét một phương pháp dựa trên ý tưởng trên, gọi là **prompting từ dễ đến khó** (*least-to-most prompting*) [Zhou và cộng sự, 2023b]. Động lực của phương pháp này xuất phát từ khó khăn khi giải các bài toán lập luận khó — những bài không thể giải quyết chỉ bằng cách khái quát hóa từ một vài ví dụ. Với những bài toán này, một chiến lược giải hiệu quả hơn là đi theo một chuỗi các bài toán con tăng dần, dẫn tới kết luận một cách có hệ thống. Cụ thể hơn, trong phương pháp prompting từ dễ đến khó, việc sinh bài toán con được thực hiện bằng cách prompt LLM với các chỉ dẫn và/hoặc ví dụ minh họa. Chẳng hạn, dưới đây là một prompt 2-shot dùng để sinh bài toán con trong prompting từ dễ đến khó.

```
TASK  Nhiệm vụ của bạn là phân rã một bài toán thành vài bài toán con.
      Bạn sẽ được cho vài ví dụ để minh họa cách làm điều này.
```

DEMO Q: Trong một cộng đồng, 5% dân số là trẻ sơ sinh, 15% là trẻ em, 40% là người trưởng thành và 40% là người cao tuổi. Nhóm nào chiếm tỉ lệ dân số lớn nhất?

A: Để trả lời câu hỏi "Nhóm nào chiếm tỉ lệ dân số lớn nhất?", ta cần biết: "Trẻ sơ sinh chiếm bao nhiêu phần trăm?", "Trẻ em chiếm bao nhiêu phần trăm?", "Người trưởng thành chiếm bao nhiêu phần trăm?", "Người cao tuổi chiếm bao nhiêu phần trăm?".

Q: Alice, Bob và Charlie mang hạt cườm đến cho dự án nhóm trong giờ thủ công. Alice có số hạt cườm gấp đôi Bob, và Bob có số hạt cườm gấp năm lần Charlie. Nếu Charlie có 6 hạt cườm, thì họ có thể dùng bao nhiêu hạt cườm cho dự án thủ công của mình?

A: Để trả lời câu hỏi "Họ có thể dùng bao nhiêu hạt cườm cho dự án thủ công của mình?", ta cần biết: "Bob có bao nhiêu hạt cườm?", "Alice có bao nhiêu hạt cườm?".

USER Q: Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C. Nghiên cứu môi trường này kéo dài bao lâu?

A: Để trả lời câu hỏi "Nghiên cứu môi trường này kéo dài bao lâu?", ta cần biết: "Nghiên cứu môi trường bắt đầu khi nào?", "Nghiên cứu môi trường kết thúc khi nào?".

Bằng cách học từ các ví dụ, LLM có thể sinh ra hai bài toán con để trả lời bài toán mới "Nghiên cứu môi trường này kéo dài bao lâu?" (được tô màu xanh dương và cam). Có các bài toán con này rồi, ta giải chúng lần lượt. Với mỗi bài toán con, ta lấy tất cả các cặp hỏi–đáp đã sinh trước đó làm ngữ cảnh, rồi đưa ra câu trả lời. Với ví dụ trên, ta cần trả lời bài toán con thứ nhất bằng cách prompt LLM như sau

Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C.

SUB-PROB1 Q: Nghiên cứu môi trường bắt đầu khi nào?
A: Nghiên cứu môi trường bắt đầu vào năm 2015.

Khi đã có câu trả lời cho bài toán con thứ nhất, ta chuyển sang bài toán con thứ hai. Lần này, ta đưa cả bài toán con thứ nhất và câu trả lời tương ứng của nó vào đầu vào.

Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C.

SUB-PROB1 Q: Nghiên cứu môi trường bắt đầu khi nào?
A: Nghiên cứu môi trường bắt đầu vào năm 2015.

SUB-PROB2 Q: Nghiên cứu môi trường kết thúc khi nào?
A: Nghiên cứu môi trường kết thúc vào năm 2020.

Cuối cùng, ta dùng LLM để giải bài toán ban đầu khi đã có câu trả lời cho tất cả các bài toán con.

Nghiên cứu môi trường được tiến hành từ năm 2015 đến năm 2020 cho thấy nhiệt độ trung bình trong khu vực đã tăng 2.3 độ C.

SUB-PROB1 Q: Nghiên cứu môi trường bắt đầu khi nào?
A: Nghiên cứu môi trường bắt đầu vào năm 2015.

SUB-PROB2 Q: Nghiên cứu môi trường kết thúc khi nào?
A: Nghiên cứu môi trường kết thúc vào năm 2020.

FINAL Q: Nghiên cứu môi trường này kéo dài bao lâu?
A: Nghiên cứu môi trường kéo dài 5 năm.

Chú giải của người dịch: Prompting "từ dễ đến khó" hoạt động thế nào?

Tên gọi *least-to-most* nghĩa là "từ (bài) ít khó nhất đến (bài) khó nhất". Quy trình gồm hai giai đoạn: (1) nhờ LLM *liệt kê* các câu hỏi nhỏ cần trả lời trước (bắt đầu khi nào? kết thúc khi nào?); (2) lần lượt nhờ LLM trả lời từng câu hỏi nhỏ, mỗi lần đều "mang theo" các câu hỏi và câu trả lời trước đó trong prompt, rồi cuối cùng trả lời câu hỏi gốc: $2020 - 2015 = 5$ năm. Mỗi lần gọi LLM chỉ phải làm một việc nhỏ và dễ, nên ít sai hơn so với đòi nó làm tất cả trong một lần. Các màu (xanh dương, cam, đỏ) trong các hộp trên giúp ta theo dõi mỗi câu hỏi con và câu trả lời của nó đi qua các prompt như thế nào.

Phương pháp từ dễ đến khó cung cấp một cách tiếp cận cơ bản để prompt LLM sinh ra và giải các bài toán con một cách riêng rẽ. Ta có thể cải tiến nó theo nhiều cách. Một cải tiến đơn giản là áp dụng các kỹ thuật prompting nâng cao khác nhau, vốn không đòi hỏi thay đổi khung phân rã bài toán. Ví dụ, ta có thể đưa CoT vào prompt để nâng cao hiệu năng lập luận khi sinh và giải các bài toán con.

Một cải tiến khác là tìm hiểu các phương pháp phân rã bài toán và tổ chức các đường giải tốt hơn. Để mô tả những cách tiếp cận này, ta sẽ dùng ký hiệu p_0 để chỉ bài toán đầu vào, và dùng các ký hiệu $\{p_1, \cdots, p_n\}$ để chỉ các bài toán con tương ứng với p_0. Với prompting từ dễ đến khó, ta phân rã p_0 thành $\{p_1, \cdots, p_n\}$, cho bởi

$$\{p_1, \cdots, p_n\} \;=\; G(p_0), \tag{9.3}$$

trong đó $G(\cdot)$ là hàm sinh bài toán con. Sau đó, ta giải lần lượt các bài toán con $\{p_1, \cdots, p_n\}$, thu được một dãy câu trả lời $\{a_1, \cdots, a_n\}$. Để trả lời bài toán con thứ i là p_i, ta đưa cả bài toán ban đầu p_0 lẫn tất cả các cặp bài toán–câu trả lời đã sinh trước đó vào ngữ cảnh để dự đoán. Câu trả lời a_i được cho bởi

$$a_i \;=\; S_i(p_i, \{p_0, p_{<i}, a_{<i}\}), \tag{9.4}$$

trong đó $p_{<i} = \{p_1, \cdots, p_{i-1}\}$ và $a_{<i} = \{a_1, \cdots, a_{i-1}\}$. $S_i(\cdot)$ là hàm giải bài toán con p_i khi biết ngữ cảnh $\{p_0, p_{<i}, a_{<i}\}$. Bước cuối cùng là sinh câu trả lời cho bài toán ban đầu p_0, có thể được biểu diễn tương tự như Phương trình (9.4).

$$a_0 \;=\; S_0(p_0, \{p_{\leq n}, a_{\leq n}\}). \tag{9.5}$$

Chú giải của người dịch: Đọc các ký hiệu trong Phương trình (9.3–9.5)

Áp vào ví dụ nghiên cứu môi trường: p_0 = "Nghiên cứu kéo dài bao lâu?"; $G(p_0)$ trả về p_1 = "Bắt đầu khi nào?" và p_2 = "Kết thúc khi nào?" (ở đây $n = 2$). Khi giải p_2, ngữ cảnh gồm p_0 và các cặp đã có trước, tức $p_{<2} = \{p_1\}$, $a_{<2} = \{a_1\}$ với a_1 = "2015"; kết quả a_2 = "2020". Cuối cùng S_0 nhận p_0 cùng tất cả các cặp $p_{\leq 2}, a_{\leq 2}$ và trả lời a_0 = "5 năm". Ký hiệu "$< i$" nghĩa là "mọi chỉ số nhỏ hơn i", còn "$\leq n$" là "mọi chỉ số từ 1 đến n". Các "hàm" G và S_i ở đây thực chất là những lần gọi LLM với prompt phù hợp.

Một cách để cải tiến mô hình này là sửa đổi hàm $G(\cdot)$ để mô hình có thể sinh câu trả lời một cách động. Thay vì sinh tất cả các bài toán con cùng một lúc, ta có thể sinh từng bài toán con trong quá trình giải [Dua và cộng sự, 2022]. Để làm điều này, ta có thể thay Phương trình (9.3) bằng

$$p_i \;=\; G_i(p_0, \{p_{<i}, a_{<i}\}). \tag{9.6}$$

Nhờ vậy ta có một mô hình sinh bài toán con hoạt động theo kiểu từng bước. Ở bước i, đầu tiên ta prompt LLM bằng bài toán ban đầu p_0 cùng với lịch sử suy luận trước đó $\{p_{<i}, a_{<i}\}$ để sinh ra một bài toán con mới p_i. Sau đó ta giải bài toán con này bằng chính LLM đó hoặc một LLM khác, với điều kiện là cùng ngữ cảnh ấy (xem Phương trình (9.4)). Phương pháp này mở rộng hiệu quả năng lực lập luận của LLM bằng cách cho phép chúng sinh và giải các bài toán con một cách động trong các bước lập luận trung gian. Kết quả là các đường lập luận không bị cố định trước, và mô hình có thể lựa chọn và điều chỉnh chiến lược lập luận của mình trong quá trình giải.

Một cách khác để cải tiến mô hình trên là tập trung phát triển các bộ giải bài toán con tốt hơn. Trong phần thảo luận trước, ta giới hạn $S_i(\cdot)$ là các LLM được prompt để giải bài toán con p_i. Thực ra, ta có thể mở rộng hàm này thành bất kỳ hệ thống nào có khả năng xử lý bài toán con. Chẳng hạn, $S_i(\cdot)$ có thể gọi tới các hệ thống truy xuất thông tin (*information retrieval, IR*), nhờ đó cho phép ta truy cập một phạm vi dữ liệu rộng hơn để giải bài toán. Một ví dụ khác là dùng $S_i(\cdot)$ như một máy tính bỏ túi để tính chính xác kết quả khi giải toán. Nếu bài toán con p_i phức tạp và cần nhiều bước giải trung gian, ta cũng có thể phân rã tiếp p_i thành các bài toán con nhỏ hơn. Ví dụ, $S_i(\cdot)$ có thể được định nghĩa là một chương trình đệ quy sinh ra và giải các bài toán con. Điều này đưa đệ quy vào việc giải

bài toán và cho phép ta xử lý bài toán bằng cách phân rã lặp đi lặp lại. Kết quả là ta có thể định nghĩa một cấu trúc phân cấp cho việc giải bài toán [Khot và cộng sự, 2023].

Chú giải của người dịch: Đệ quy

Một chương trình *đệ quy* là chương trình tự gọi lại chính nó trên một phiên bản nhỏ hơn của bài toán, cho tới khi bài toán đủ nhỏ để giải trực tiếp. Ví dụ quen thuộc: $n! = n \cdot (n-1)!$, với $1! = 1$. Ở đây, nếu bài toán con p_i vẫn còn khó, bộ giải S_i lại phân rã nó thành các bài toán con nhỏ hơn nữa, tạo thành một "cây" bài toán nhiều tầng (cấu trúc phân cấp).

Một bước tổng quát hóa xa hơn của cách phát biểu này là xem nó như một bài toán học tăng cường (*reinforcement learning*). Một phương pháp điển hình là mô hình hóa quá trình giải bài toán như một quá trình ra quyết định. Ở mỗi bước của quá trình này, một hành động được thực hiện dựa trên trạng thái hiện tại. Các hành động này có thể bao gồm tất cả các hàm sinh và giải bài toán con (tức $G_i(\cdot)$ và $S_i(\cdot)$). Như vậy, dãy hành động tương ứng với một đường giải bài toán. Vì việc thảo luận các bài toán học tăng cường nằm ngoài phạm vi chương này, ta bỏ qua mô tả chính xác của nhiệm vụ học này. Tuy vậy, phát triển một tác tử (*agent*) hay bộ điều khiển để quyết định khi nào và làm thế nào sinh và giải một bài toán con cũng là một lựa chọn tự nhiên.

Trong NLP, phân rã bài toán có liên quan tới một dòng nghiên cứu lâu đời về hỏi đáp đa bước (*multi-hop question answering*) [Mavi và cộng sự, 2024]. Nhiệm vụ này đòi hỏi hệ thống thu thập và kết hợp thông tin từ nhiều đoạn văn bản để đưa ra câu trả lời chính xác cho một câu hỏi phức tạp. Ví dụ, để trả lời câu hỏi "Thủ đô của quốc gia nơi Albert Einstein sinh ra là gì?", ta cần biết "Albert Einstein sinh ra ở đâu?" và "Thủ đô của Đức là gì?". Các công trình trước đây trong lĩnh vực này và các lĩnh vực liên quan đã nghiên cứu vấn đề phân rã bài toán, dù các phương pháp có thể không dựa trên LLM. Ví dụ, một phương pháp phổ biến là phát triển thêm một mô hình nơ-ron để sinh ra các câu hỏi đơn giản hơn, mỗi câu hỏi đề cập một khía cạnh khác nhau của câu hỏi ban đầu [Andreas và cộng sự, 2016; Talmor và Berant, 2018; Min và cộng sự, 2019]. Bộ sinh câu hỏi này có thể tạo các câu hỏi theo lô (cùng một lúc) hoặc tuần tự.

Nói rộng hơn, phân rã bài toán cũng liên quan tới vấn đề *tính kết hợp* (*compositionality*) trong NLP [Drozdov và cộng sự, 2022; Press và cộng

sự, 2023]. Chẳng hạn, trong phân tích ngữ nghĩa (*semantic parsing*), ta ánh xạ các câu ngôn ngữ tự nhiên thành các biểu diễn ý nghĩa có cấu trúc bằng cách chia nhỏ chúng thành các thành phần cấu tạo và hiểu câu dựa trên ý nghĩa của các thành phần đó cùng các quy tắc dùng để kết hợp chúng. Trong những nghiên cứu ban đầu của lĩnh vực này, các câu có tính kết hợp cao được coi là những trường hợp kiểm tra dễ hơn để đánh giá hệ thống, vì việc phân rã những câu như vậy và kết hợp ý nghĩa các phần của chúng là tương đối đơn giản. Tuy nhiên, nhiệm vụ trở nên khó hơn nhiều khi cần khái quát hóa nhiều hơn để mô hình hóa tính kết hợp trên dữ liệu mới. Trong trường hợp này, ta muốn hệ thống có năng lực **khái quát hóa kết hợp** (*compositional generalization*) tốt hơn. Trong các nghiên cứu gần đây hơn về LLM, vấn đề này thường được bàn tới trong các nhiệm vụ lập luận kết hợp, như SCAN[9], vì nó được coi là một khía cạnh quan trọng để kiểm tra năng lực hiểu ngôn ngữ và lập luận của LLM. Điều này cũng đặt ra những nhiệm vụ mới để phát triển và kiểm nghiệm các phương pháp phân rã bài toán.

Chú giải của người dịch: Tính kết hợp và ví dụ SCAN

Tính kết hợp nghĩa là ý nghĩa của cả câu được ghép từ ý nghĩa của các phần theo những quy tắc nhất định. Trong SCAN, nếu hệ thống hiểu "jump" = JUMP (nhảy), "walk" = WALK (đi), "thrice" = lặp ba lần, "opposite left" = quay trái hai lần (LTURN LTURN, tức quay ngược lại), thì nó có thể tự ghép ra "LTURN LTURN JUMP WALK WALK WALK" cho một mệnh lệnh chưa từng thấy. *Khái quát hóa kết hợp* chính là khả năng xử lý đúng những tổ hợp mới như vậy — giống như khi đã biết nghĩa của "ba" và của "quả táo", ta hiểu ngay "ba quả táo" dù chưa gặp cụm này.

Trong LLM, một ứng dụng thú vị của phân rã bài toán là *sử dụng công cụ*. Trong một số trường hợp, cần tích hợp các công cụ bên ngoài vào LLM để truy cập dữ liệu chính xác không có sẵn trong lúc huấn luyện hay tinh chỉnh. Ví dụ, LLM có thể tích hợp với các API để lấy dữ liệu thời gian thực như cập nhật thời tiết, giá chứng khoán hay tin tức, nhờ đó đưa ra các câu trả lời cập nhật cho truy vấn của người dùng. Khi dùng công cụ, dự

[9]Các nhiệm vụ SCAN (phiên bản đơn giản hóa của các nhiệm vụ CommAI Navigation) được thiết kế để đánh giá khả năng khái quát hóa kết hợp của LLM [Lake và Baroni, 2018]. Chúng yêu cầu dịch các mệnh lệnh ngôn ngữ tự nhiên thành một dãy hành động. Ví dụ, mệnh lệnh "jump opposite left and walk thrice" (*nhảy về phía đối diện bên trái rồi đi bộ ba lần*) có thể được dịch thành dãy hành động "LTURN LTURN JUMP WALK WALK WALK".

đoán của LLM có thể chứa các dấu hiệu chỉ ra chỗ nào và cách nào để gọi API bên ngoài. Điều này đòi hỏi phân rã bài toán thành các bài toán con, một số do LLM xử lý và số khác do các công cụ bên ngoài xử lý. Thảo luận chi tiết hơn về chủ đề này sẽ được trình bày ở Mục 9.2.5.

9.2.3 Tự tinh chỉnh

Trong nhiều trường hợp, dự đoán của LLM có thể không chính xác hoặc có sai sót. Vì các LLM hiện nay có khả năng thực hiện những nhiệm vụ như chỉnh sửa và sửa lỗi, việc tìm hiểu các phương pháp cho phép các mô hình này *tự tinh chỉnh* (*self-refine*) đầu ra của chính chúng là rất có giá trị. Tự tinh chỉnh là một hiện tượng phổ biến trong quá trình nhận thức và hành vi hằng ngày của con người. Chẳng hạn, khi thiết kế một sản phẩm, nhà thiết kế có thể tạo một nguyên mẫu cơ bản trước, rồi tinh chỉnh thiết kế sau khi đánh giá và thử nghiệm để nâng cao trải nghiệm người dùng và tính năng. Quá trình tinh chỉnh này có thể lặp lại nhiều lần cho tới khi thiết kế đạt yêu cầu. Mô thức *dự đoán rồi tinh chỉnh* có lịch sử lâu dài trong NLP. Một ví dụ sớm là bộ gán nhãn của Brill [Brill, 1992], trong đó kết quả gán nhãn từ loại (*part-of-speech tagging, POS tagging*) ban đầu của một câu có thể được tinh chỉnh lặp đi lặp lại bằng một hệ thống dựa trên luật. Trong kỷ nguyên học sâu, rất nhiều công trình về các bài toán chuỗi-sang-chuỗi, như sửa lỗi ngữ pháp và viết lại văn bản, cũng có thể được xem là ví dụ của mô thức này.

Ta có thể prompt LLM để tự tinh chỉnh. Hãy xét một ví dụ đơn giản, trong đó đầu tiên ta dùng một LLM để dịch một văn bản từ tiếng Trung sang tiếng Anh, rồi dùng chính mô hình đó để tinh chỉnh bản dịch. Đây là cách ta có thể prompt mô hình để dịch (văn bản tiếng Trung và tiếng Anh được giữ nguyên vì đây là ví dụ dịch máy; nghĩa tiếng Việt được ghi ở chú giải phía dưới):

> Hãy dịch văn bản sau từ tiếng Trung sang tiếng Anh:
>
> Tiếng Trung:
> 一系列考古发现奠定红山文化在中华文明起源研究中的重要地位。最新公布的研究成果认为，大约从距今5800年开始，中华大地上各个区域相继出现较为明显的社会分化，进入文明起源的加速阶段。
>
> Tiếng Anh:

A series of discoveries have cemented the significant role of the Hongshan culture in studies on the origins of Chinese civilization. The latest research findings suggest that, starting from around 5800 years ago, many regions across China began to experience noticeable social differentiations, entering an accelerated phase in the origin of civilization.

Sau đó, ta có thể prompt nó tinh chỉnh bản dịch:

Bạn được cung cấp một văn bản tiếng Trung cùng với bản dịch tiếng Anh của nó. Hãy tinh chỉnh bản dịch để cải thiện độ chính xác và độ trôi chảy nếu cần.

Tiếng Trung:
一系列考古发现奠定红山文化在中华文明起源研究中的重要地位。最新公布的研究成果认为，大约从距今5800年开始，中华大地上各个区域相继出现较为明显的社会分化，进入文明起源的加速阶段。

Tiếng Anh:
A series of discoveries have cemented the significant role of the Hongshan culture in studies on the origins of Chinese civilization. The latest research findings suggest that, starting from around 5800 years ago, many regions across China began to experience noticeable social differentiations, entering an accelerated phase in the origin of civilization.

Bản tinh chỉnh:
A series of archaeological discoveries have cemented the significant role of the Hongshan culture in studies on the origins of Chinese civilization. The latest research findings suggest that, starting from around 5800 years ago, various regions across China began to experience noticeable social differentiations, entering an accelerated phase in the origin of civilization.

Ở đây những chỗ sửa đổi so với bản dịch đầu vào được tô màu xanh lá. Quá trình này có thể lặp lại: ở mỗi bước, ta dùng bản dịch của bước trước làm đầu vào và prompt LLM tạo ra một bản dịch có thể tốt hơn.

Chú giải của người dịch: Nghĩa của đoạn văn trong ví dụ

Đoạn văn tiếng Trung (và bản dịch tiếng Anh) có nghĩa: "Một loạt phát hiện khảo cổ đã khẳng định vị trí quan trọng của văn hóa Hồng Sơn trong nghiên cứu về nguồn gốc văn minh Trung Hoa. Các kết quả nghiên cứu mới công bố cho rằng, bắt đầu từ khoảng 5800 năm trước, nhiều khu vực trên đất Trung Hoa lần lượt xuất hiện sự phân hóa xã hội tương đối rõ rệt, bước vào giai

đoạn tăng tốc của quá trình hình thành văn minh." Bản dịch đầu tiên bỏ sót chữ 考古 ("khảo cổ"); bản tinh chỉnh bổ sung *archaeological* (khảo cổ), và đổi *many* (nhiều) thành *various* (các, khác nhau) cho sát với 各个 ("từng/các") hơn.

Trong ví dụ trên, việc tinh chỉnh đầu ra ban đầu của mô hình được thực hiện bằng cách prompt LLM với một chỉ dẫn đơn giản như "Hãy tinh chỉnh nó!". Tuy nhiên, việc tinh chỉnh chỉ dựa vào khả năng làm theo chỉ dẫn của LLM, và không có hướng dẫn hay giám sát nào về việc cải thiện đầu ra của mô hình như thế nào và ở đâu. Một cách tiếp cận hiệu quả hơn là xét đến *phản hồi* về những khía cạnh cụ thể cần tinh chỉnh. Chẳng hạn, ta có thể prompt LLM bằng "Hãy sửa tất cả các lỗi ngữ pháp trong bản dịch", để mô hình tập trung hơn vào việc sửa lỗi ngữ pháp trong lúc tinh chỉnh.

Một khung tổng quát cho việc tự tinh chỉnh bằng LLM gồm ba bước [Madaan và cộng sự, 2024].

- **Dự đoán** (*Prediction*). Ta dùng một LLM để tạo ra đầu ra ban đầu của mô hình.
- **Thu thập phản hồi** (*Feedback Collection*). Ta thu được phản hồi về đầu ra của mô hình.
- **Tinh chỉnh** (*Refinement*). Ta dùng LLM để tinh chỉnh đầu ra của mô hình dựa trên phản hồi.

Hai bước cuối có thể được lặp lại nhiều lần, dẫn tới một quá trình tự tinh chỉnh lặp. Trong khung này, việc nhận được phản hồi chính xác và chi tiết là then chốt vì nó định hướng cho bước tinh chỉnh tiếp theo. Nói chung, phản hồi có thể thu được theo nhiều cách [Fernandes và cộng sự, 2023; Pan và cộng sự, 2024]. Ta có thể tự kiểm tra thủ công các dự đoán của mô hình để phát hiện vấn đề. Ta cũng có thể dùng một *mô hình phản hồi* để tự động sinh phản hồi. Chẳng hạn, một mô hình phần thưởng (*reward model*) được huấn luyện trên một ít dữ liệu có gán nhãn có thể giúp đánh giá đầu ra của mô hình và trả về một điểm số cho biết chất lượng của nó.

Vì trọng tâm của chương này là prompting LLM, ta xét việc dùng LLM để đưa ra phản hồi. Để minh họa, hãy xét một ví dụ sinh câu trả lời tốt cho các câu hỏi của người dùng:

- Đầu tiên, ta prompt một LLM trả lời câu hỏi đầu vào, như thường lệ.

> Hãy viết câu trả lời cho câu hỏi: "Hiện nay có những vấn đề môi trường lớn nào?"
>
> Câu trả lời:
> Những vấn đề môi trường lớn hiện nay gồm biến đổi khí hậu, ô nhiễm không khí và nước, phá rừng, suy giảm đa dạng sinh học và suy thoái đại dương.

- Sau đó, bằng prompt dưới đây, ta lại prompt chính LLM đó để nhận phản hồi về câu trả lời đã sinh ra.

> Bạn sẽ được cung cấp một câu hỏi và câu trả lời tương ứng. Hãy đánh giá độ chính xác và độ sâu của câu trả lời này, đồng thời chỉ ra mọi chỗ sai về sự kiện hoặc những chỗ mà việc bổ sung chi tiết có thể giúp hiểu rõ hơn.
>
> Câu hỏi:
> Hiện nay có những vấn đề môi trường lớn nào?
>
> Câu trả lời:
> Những vấn đề môi trường lớn hiện nay gồm biến đổi khí hậu, ô nhiễm không khí và nước, phá rừng, suy giảm đa dạng sinh học và suy thoái đại dương.
>
> Phản hồi:
> Câu trả lời được đưa ra là chính xác và nêu bật các vấn đề môi trường then chốt. Tuy nhiên, nó sẽ tốt hơn nếu có mô tả cụ thể và chi tiết hơn về những vấn đề này. Dưới đây là các gợi ý cải thiện:
> 1. Nêu các ví dụ cụ thể về các vấn đề môi trường.
> 2. Bổ sung thông tin về các chất gây ô nhiễm chính.
> 3. Chỉ rõ những khu vực bị ảnh hưởng nặng nhất.

- Với phản hồi mà LLM tạo ra, ta prompt nó tinh chỉnh câu trả lời.

> Bạn sẽ được cung cấp một câu hỏi, câu trả lời tương ứng và một số phản hồi về câu trả lời này.
>
> Câu hỏi:
> Hiện nay có những vấn đề môi trường lớn nào?
>
> Câu trả lời:

Những vấn đề môi trường lớn hiện nay gồm biến đổi khí hậu, ô nhiễm không khí và nước, phá rừng, suy giảm đa dạng sinh học và suy thoái đại dương.

Phản hồi:
Câu trả lời được đưa ra là chính xác và nêu bật các vấn đề môi trường then chốt. Tuy nhiên, nó sẽ tốt hơn nếu có mô tả cụ thể và chi tiết hơn về những vấn đề này. Dưới đây là các gợi ý cải thiện:

1. Nêu các ví dụ cụ thể về các vấn đề môi trường.
2. Bổ sung thông tin về các chất gây ô nhiễm chính.
3. Chỉ rõ những khu vực bị ảnh hưởng nặng nhất.

Bây giờ hãy tinh chỉnh câu trả lời dựa trên phản hồi đã cho.

Câu trả lời đã tinh chỉnh:
Các vấn đề môi trường then chốt gồm biến đổi khí hậu, thể hiện qua nhiệt độ toàn cầu tăng và các kiểu thời tiết khắc nghiệt; ô nhiễm không khí và nước nghiêm trọng ở các khu đô thị và công nghiệp; nạn phá rừng tràn lan ở những vùng như Amazon, gây suy giảm đa dạng sinh học; và suy thoái đại dương, nổi bật là hiện tượng san hô bị tẩy trắng và đánh bắt cá quá mức trên diện rộng.

Lý tưởng nhất, nếu dùng một LLM mạnh, ta muốn nó thực hiện cả ba bước mà không cần huấn luyện thêm. Mặt khác, nếu có đủ dữ liệu gán nhãn cho nhiệm vụ quan tâm, ta có thể nâng cao hiệu năng của LLM bằng học có giám sát. Ví dụ, ta có thể tinh chỉnh LLM để nó thích nghi tốt hơn với các nhiệm vụ tinh chỉnh đầu ra, hoặc dùng các mô hình chuyên cho nhiệm vụ, vốn không nhất thiết phải dựa trên LLM [Welleck và cộng sự, 2023; Schick và cộng sự, 2023]. Theo nghĩa rộng hơn, việc cải thiện LLM cho các nhiệm vụ tự tinh chỉnh có thể được xem là một vấn đề căn chỉnh (*alignment*). Chẳng hạn, người ta đã thấy rằng một số năng lực tự sửa lỗi có thể được kích hoạt thông qua RLHF [Ganguli và cộng sự, 2023]. Tuy nhiên, việc thảo luận những vấn đề này nằm ngoài phạm vi chương này. Thảo luận thêm có thể tìm thấy ở Chương 10.

Chú giải của người dịch: Tự tinh chỉnh giống việc tự chấm và sửa bài

Ba bước *dự đoán – thu thập phản hồi – tinh chỉnh* giống cách ta làm bài tập làm văn: viết bản nháp, nhờ thầy cô (hoặc tự mình) đọc lại và ghi nhận xét "thiếu dẫn chứng, cần ví dụ cụ thể", rồi viết lại theo nhận xét. Điểm đặc biệt

ở đây là cả ba vai (người viết, người nhận xét, người sửa) đều do *cùng một* LLM đảm nhận, chỉ khác nhau ở prompt. Lời nhận xét cụ thể ("thêm ví dụ, nêu chất gây ô nhiễm, chỉ rõ khu vực") hữu ích hơn nhiều so với một yêu cầu chung chung "hãy viết hay hơn". (RLHF — học tăng cường từ phản hồi của con người — là một kỹ thuật căn chỉnh sẽ được trình bày ở Chương 10.)

Trong bối cảnh LLM, tự tinh chỉnh liên quan tới một số khái niệm mô phỏng năng lực lập luận nâng cao, như khả năng *tự phản tư* (*self-reflection*). Một quan điểm cho rằng việc trang bị khả năng tự phản tư cho LLM giúp chúng sinh ra các dự đoán chính xác hơn và thể hiện khả năng tự sửa lỗi. Năng lực phản tư này có thể được kích hoạt theo nhiều cách, chẳng hạn bằng cách prompt LLM tham gia vào quá trình lập luận có cân nhắc, từng bước một, hoặc bằng cách cung cấp các ví dụ trong ngữ cảnh để mô hình học hỏi và điều chỉnh. Để minh họa, ta xem xét phương pháp **cân nhắc rồi sinh** (*deliberate-then-generate*) (**DTG**) do Li và cộng sự [2023a] đề xuất, phương pháp này prompt LLM một cách tường minh để cân nhắc. Trong khung DTG, cho một đầu ra ban đầu của mô hình, có thể có sai sót, LLM được prompt để xác định các loại lỗi cụ thể trước khi đưa ra một bản sửa tốt hơn. Dưới đây là một khuôn mẫu prompt DTG cho nhiệm vụ dịch từ tiếng Trung sang tiếng Anh.

Cho câu tiếng Trung: {∗source∗}
Bản dịch tiếng Anh là: {∗target∗}
Hãy phát hiện loại lỗi trước, rồi tinh chỉnh bản dịch.
Loại lỗi: _____

_____

Ta muốn mô hình dự đoán loại lỗi trước (ô màu đỏ), rồi tạo ra một bản dịch đã tinh chỉnh (ô màu xanh dương). Quá trình cân nhắc này được dẫn dắt bởi chỉ dẫn "Hãy phát hiện loại lỗi trước, rồi tinh chỉnh bản dịch". Nó khuyến khích LLM ban đầu phân tích thấu đáo rồi mới đưa ra kết quả tốt hơn. Vì việc dự đoán loại lỗi và việc tinh chỉnh được thực hiện trong một lần chạy LLM, phương pháp này gộp cả hai bước phản hồi và tinh chỉnh vào một quá trình.

Trong các prompt trên, ta giả định rằng LLM được dùng có khả năng

xem xét bản dịch đầu vào và xác định đúng các loại lỗi của nó. Tuy nhiên, điều này đặt ra những khó khăn mới vì mô hình có thể không giỏi tìm lỗi trong bản dịch. Điều đó lại dẫn tới công sức tinh chỉnh hoặc thiết kế prompt bổ sung. Vì vậy, một phương pháp đơn giản hơn là giảm gánh nặng xác định lỗi và chỉ dùng LLM để cân nhắc. Để làm điều này, ta có thể thay bản dịch đầu vào bằng một bản dịch ngẫu nhiên và gán một loại lỗi mặc định. Một ví dụ về prompt như vậy được trình bày dưới đây (câu tiếng Trung và bản dịch tiếng Anh được giữ nguyên).

Cho câu tiếng Trung:
一系列考古发现奠定红山文化在中华文明起源研究中的重要地位。
Bản dịch tiếng Anh là:
A variety of innovative techniques have redefined the importance of modern art in contemporary cultural studies.
Hãy phát hiện loại lỗi trước, rồi tinh chỉnh bản dịch.
Loại lỗi: Dịch sai

_____

Chú giải của người dịch: Vì sao lại cố ý đưa vào một bản dịch sai?

Câu tiếng Trung nghĩa là "Một loạt phát hiện khảo cổ đã khẳng định vị trí quan trọng của văn hóa Hồng Sơn trong nghiên cứu nguồn gốc văn minh Trung Hoa", còn câu tiếng Anh tô xanh lại nghĩa là "Nhiều kỹ thuật đổi mới đã định nghĩa lại tầm quan trọng của nghệ thuật hiện đại trong nghiên cứu văn hóa đương đại" — hoàn toàn không liên quan. Vì ta *biết chắc* bản dịch này sai, ta có thể điền sẵn loại lỗi là "Dịch sai" mà không cần mô hình tự tìm lỗi. Mô hình chỉ cần làm việc còn lại: nhìn câu gốc, nhận ra một ví dụ "sai", và viết một bản dịch đúng vào ô xanh dương.

Trong ví dụ này, bản dịch đầu vào không phải do LLM sinh ra mà được lấy mẫu ngẫu nhiên từ tập dữ liệu. Vì vậy, nó đơn giản chỉ là một bản dịch sai của câu nguồn, và ta có thể đặt loại lỗi tương ứng. LLM sau đó sinh ra một bản dịch mới bằng cách lấy cả câu nguồn lẫn bản dịch sai làm đầu vào. Thiết kế prompt này cũng có thể được xem là kích hoạt năng lực học của LLM thông qua "bằng chứng phủ định" (*negative evidence*) [Marcus, 1993], nhờ đó cho phép chúng phản tư và tạo ra kết quả tốt hơn thông qua

phân tích đối chiếu. Tuy nhiên, phương pháp này không dựa vào bất kỳ phản hồi nào và có thể nâng cao hiệu năng của một lần dự đoán duy nhất của LLM chỉ bằng prompting đơn giản.

Lưu ý rằng mặc dù DTG không lặp, việc học và tinh chỉnh lặp lại được dùng phổ biến trong NLP. Một ưu điểm của các cách tiếp cận lặp là chúng mô phỏng cách con người học và giải quyết vấn đề, trong đó phản hồi và điều chỉnh liên tục dẫn tới kết quả ngày càng tốt hơn. Các phương pháp lặp có thể áp dụng cho nhiều bài toán prompting LLM. Ví dụ, trong phân rã bài toán, ta có thể đưa các bài toán con mới và lời giải của chúng vào ngữ cảnh ở mỗi bước, nhờ đó LLM có thể dần dần tiến tới lời giải của bài toán ban đầu. Mặt khác, các phương pháp lặp nảy sinh một số vấn đề không có ở các phương pháp không lặp. Chẳng hạn, lỗi ở các bước trước có thể ảnh hưởng xấu tới việc giải ở các bước sau, và việc xác định khi nào dừng lặp thường đòi hỏi thêm công sức kỹ thuật.

9.2.4 Kết hợp nhiều mô hình

Việc kết hợp mô hình (*model ensembling*) cho sinh văn bản đã được bàn luận rộng rãi trong các tài liệu NLP. Ý tưởng là kết hợp các dự đoán của hai hay nhiều mô hình để sinh ra một đầu ra cuối cùng tốt hơn. Kỹ thuật này có thể áp dụng trực tiếp cho LLM. Ví dụ, ta có thể tập hợp một nhóm LLM và chạy từng mô hình trên cùng một đầu vào. Đầu ra cuối cùng thu được bằng cách kết hợp các dự đoán của những mô hình này.

Với prompting LLM, cũng có thể cải thiện hiệu năng bằng cách kết hợp các dự đoán dựa trên những prompt khác nhau. Giả sử ta có một LLM và một tập các prompt cùng giải quyết một nhiệm vụ. Ta có thể chạy LLM này với từng prompt rồi kết hợp các dự đoán. Ví dụ, dưới đây là ba khuôn mẫu prompt khác nhau cho nhiệm vụ đơn giản hóa văn bản (*text simplification*).

```
Hãy làm cho văn bản này đơn giản hơn.
{*text*}

___
```

Hãy rút gọn và đơn giản hóa văn bản này.
{∗text∗}

___

Hãy viết lại cho dễ đọc.
{∗text∗}

___

Mỗi prompt này sẽ dẫn tới một dự đoán khác nhau, và ta có thể kết hợp ba dự đoán này để tạo ra đầu ra cuối cùng.

Một cách hình thức, gọi $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$ là K prompt dùng để thực hiện cùng một nhiệm vụ. Cho một LLM $\Pr(\cdot|\cdot)$, ta có thể tìm dự đoán có khả năng cao nhất cho mỗi $\mathbf{x}_i$ bằng $\hat{\mathbf{y}}_i = \arg\max_{\mathbf{y}_i} \Pr(\mathbf{y}_i|\mathbf{x}_i)$. Các dự đoán này có thể được kết hợp để tạo thành một dự đoán "mới":

$$\hat{\mathbf{y}} = \text{Combine}(\hat{\mathbf{y}}_1, \cdots, \hat{\mathbf{y}}_K). \tag{9.7}$$

Ở đây $\text{Combine}(\cdot)$ là mô hình kết hợp, có thể được thiết kế theo nhiều cách khác nhau. Chẳng hạn, ta có thể chọn dự đoán tốt nhất bằng bỏ phiếu, hoặc bằng cách tìm dự đoán trùng lặp nhiều nhất với các dự đoán khác. Một cách tiếp cận khác là kết hợp điểm xác suất ở mức token trong quá trình giải mã. Gọi $\hat{y}_j$ là token được dự đoán ở bước thứ j khi kết hợp mô hình. Token dự đoán $\hat{y}_j$ được cho bởi

$$\hat{y}_j = \arg\max_{y_j} \sum_{k=1}^{K} \log \Pr(y_j|\mathbf{x}_k, \hat{y}_1, \cdots, \hat{y}_{j-1}). \tag{9.8}$$

Bạn đọc quan tâm có thể tham khảo Chương 5 để biết thêm chi tiết về các phương pháp này.

Chú giải của người dịch: Hai cách kết hợp trong Phương trình (9.7) và (9.8)

Phương trình (9.7) kết hợp ở mức *toàn bộ câu trả lời*: mỗi prompt cho ra một bản đơn giản hóa hoàn chỉnh, rồi ta chọn bản "được nhiều phiếu nhất" hoặc giống các bản khác nhất. Phương trình (9.8) kết hợp ở mức *từng token*: khi sinh token thứ j, ta hỏi cả K prompt xem mỗi token ứng viên có xác suất bao nhiêu, cộng các log xác suất lại và chọn token có tổng lớn nhất. Ví dụ

với $K = 2$: token A có xác suất 0.6 và 0.5, token B có 0.9 và 0.1. Tổng log của A là $\log 0.6 + \log 0.5 = \log 0.30$, của B là $\log 0.9 + \log 0.1 = \log 0.09$, nên chọn A — token mà *cả hai* prompt đều thấy khá hợp lý, thay vì token chỉ một prompt rất thích. (Cộng các log tương đương với nhân các xác suất.)

Khi kết hợp trong prompting LLM, nói chung nên dùng các prompt đa dạng để sự kết hợp có thể bao quát một phạm vi rộng hơn các câu trả lời tiềm năng. Cách làm này phổ biến trong học kết hợp (*ensemble learning*), vì sự đa dạng giúp "trung bình hóa" những thiên lệch và sai sót có thể chỉ riêng một mô hình hay một cấu hình mắc phải. Từ góc nhìn Bayes, ta có thể coi prompt $\mathbf{x}$ là một biến ẩn (*latent variable*), khi biết bài toán quan tâm p. Điều này cho phép viết phân phối dự đoán của $\mathbf{y}$ khi biết p dưới dạng phân phối $\Pr(\mathbf{y}|\mathbf{x})$ được lấy biên (*marginalized*) trên mọi prompt khả dĩ

$$\Pr(\mathbf{y}|p) \;=\; \int \Pr(\mathbf{y}|\mathbf{x})\,\Pr(\mathbf{x}|p)\;d\mathbf{x}. \tag{9.9}$$

Tích phân này tính xác suất toàn phần của $\mathbf{y}$ bằng cách xét mọi giá trị khả dĩ của $\mathbf{x}$, có trọng số là xác suất của chúng khi biết p. Ở đây $\Pr(\mathbf{y}|\mathbf{x})$ do LLM cho, còn $\Pr(\mathbf{x}|p)$ là phân phối tiên nghiệm (*prior distribution*) của các prompt cho bài toán. Cách phát biểu này có lợi vì tích phân tính đến sự bất định trong việc chọn $\mathbf{x}$. Kết quả là phân phối dự đoán cuối cùng $\Pr(\mathbf{y}|p)$ trở nên vững hơn trước những biến động và thiên lệch của từng prompt riêng lẻ. Tuy nhiên, tính trực tiếp tích phân này có thể bất khả thi về mặt tính toán do không gian của $\mathbf{x}$ có thể vô hạn. Một cách giải quyết vấn đề này là dùng các phương pháp như lấy mẫu Monte Carlo (*Monte Carlo sampling*), xấp xỉ tích phân bằng một số hữu hạn, vừa phải các prompt.

Chú giải của người dịch: Ý nghĩa của Phương trình (9.9)

Đây là "công thức xác suất toàn phần" cho trường hợp có vô số khả năng. Hãy hình dung phiên bản rời rạc: giả sử chỉ có 3 prompt có thể dùng, với xác suất được chọn $\Pr(\mathbf{x}_k|p)$ là 0.5, 0.3, 0.2, và xác suất mỗi prompt dẫn tới đáp án đúng $\mathbf{y}$ là 0.9, 0.6, 0.5. Khi đó $\Pr(\mathbf{y}|p) = 0.5{\cdot}0.9 + 0.3{\cdot}0.6 + 0.2{\cdot}0.5 = 0.73$ — một "trung bình có trọng số" qua các prompt. Dấu tích phân $\int$ chỉ là phiên bản "tổng" khi có vô số prompt. Vì không thể cộng vô số số hạng, *Monte Carlo* chọn ngẫu nhiên một số prompt (theo $\Pr(\mathbf{x}|p)$) rồi lấy trung bình của $\Pr(\mathbf{y}|\mathbf{x})$ trên các prompt đó để ước lượng. Phân phối *tiên nghiệm* là niềm tin "trước khi thấy dữ liệu" về việc prompt nào phù hợp.

Mặc dù cách xử lý theo Bayes được định nghĩa chặt chẽ về mặt toán học, trong NLP người ta thường giả định một tiên nghiệm không mang thông tin hoặc tiên nghiệm đều, và thay vào đó tập trung xây dựng một tập prompt đa dạng. Do đó, đầu ra có thể được tính bằng một mô hình kết hợp đơn giản, như mô tả ở Phương trình (9.7). Vấn đề tạo ra các prompt đa dạng, chất lượng cao đã được nghiên cứu trong CoT và các lĩnh vực học trong ngữ cảnh khác. Phần lớn nghiên cứu tập trung vào việc đưa nhiều ví dụ minh họa khác nhau vào các prompt khác nhau. Dưới đây ta liệt kê một số phương pháp như vậy.

- Với một bài toán, ta tự tạo thủ công một số ví dụ minh họa và dùng các ví dụ khác nhau cho các prompt khác nhau.
- Với một bài toán, ta dùng LLM để tự động sinh ví dụ minh họa và prompt.
- Với một prompt, ta tạo các prompt khác nhau bằng cách thay đổi thứ tự các ví dụ minh họa trong prompt.
- Với một prompt, ta dùng LLM để sinh ra một số prompt tương tự.
- Với một prompt, ta biến đổi nó sang các dạng khác, chẳng hạn dịch nó sang các ngôn ngữ khác.

Tất nhiên, trong thực tế, ta có thể kết hợp các phương pháp này để đạt được sự đa dạng lớn hơn. Một giả định ngầm ở đây là các prompt đa dạng có thể dẫn tới các đầu ra đa dạng của mô hình. Điều này đặc biệt đúng khi xử lý những bài toán tương đối mới lạ và phức tạp. Với những LLM mạnh và vững hơn, mức độ biến thiên của đầu ra với các prompt tương tự có thể không lớn. Trong trường hợp này, lợi ích của việc dùng nhiều prompt có thể khiêm tốn.

Ngoài việc cung cấp các prompt đa dạng cho LLM, một cách tiếp cận khác là tận dụng tính biến thiên vốn có trong đầu ra của LLM. Một cách đơn giản là lấy mẫu nhiều đầu ra từ không gian giả thuyết. Điều này dễ làm với LLM vì các thuật toán tìm kiếm của chúng vốn phù hợp với việc tạo ra nhiều kết quả. Chẳng hạn, nếu dùng tìm kiếm chùm (beam search), ta có thể dễ dàng thu thập tất cả các giả thuyết đã hoàn chỉnh trong chùm để tạo ra nhiều ứng viên đầu ra. Ngoài ra, ta có thể sửa đổi thuật toán tìm kiếm để cài đặt những phương pháp lấy mẫu mạnh hơn và khám phá phạm vi rộng hơn các giả thuyết chất lượng cao.

Câu hỏi tiếp theo là làm thế nào để kết hợp các dự đoán đa dạng của mô hình. Điều này đưa ta trở lại vấn đề kết hợp mô hình như mô tả ở Phương trình (9.7). Ở đây ta xét phương pháp **tự nhất quán** (*self-consistency*), phương pháp này không nhất thiết đưa ra dự đoán có xác suất cao nhất, mà đưa ra dự đoán nhất quán nhất với các dự đoán khác [Wang và cộng sự, 2022d; 2023c]. Đầu tiên, LLM được prompt bằng CoT như thường lệ và sinh ra nhiều đường lập luận bằng cách lấy mẫu. Sau đó, tần suất của mỗi đáp án được đếm trên các đường lập luận này. Đầu ra cuối cùng là đáp án xuất hiện nhiều nhất. Dưới đây là ba dự đoán khác nhau được sinh ra bằng cách prompt một LLM với CoT.

Dự đoán 1 (Đúng):

Q: Ba người bạn chơi một trò chơi tung một đồng xu cân đối. Mỗi người tung đồng xu một lần. Xác suất để có đúng một người tung được mặt sấp là bao nhiêu?

A: Hãy suy nghĩ từng bước một.
Mỗi lần tung đồng xu là một biến cố độc lập với 50% khả năng ra mặt sấp (H) và 50% ra mặt ngửa (T). Xét các kết quả có thể có của ba lần tung: HHH, HHT, HTH, HTT, THH, THT, TTH, TTT. Trong số này, chỉ HHH, HHT, HTH và THH là liên quan. Ta chỉ quan tâm đến các trường hợp có đúng một mặt sấp: HTT, THT và TTH. Có 3 kết quả như vậy trong tổng số 8, cho xác suất là $3/8$.

Dự đoán 2 (Đúng):

Q: Ba người bạn chơi một trò chơi tung một đồng xu cân đối. Mỗi người tung đồng xu một lần. Xác suất để có đúng một người tung được mặt sấp là bao nhiêu?

A: Hãy suy nghĩ từng bước một.
Dùng công thức xác suất nhị thức, trong đó p (thành công, tức mặt sấp) là 0.5, và mỗi lần thử trong 3 lần thử (lần tung) là độc lập, ta tìm trường hợp có đúng một thành công (mặt sấp). Công thức cho đúng k thành công trong n lần thử được cho bởi công thức hệ số nhị thức $\binom{n}{k}p^k(1-p)^{n-k}$. Thay các giá trị vào, ta được $\binom{3}{1}(0.5)^1(0.5)^{3-1} = 3 \cdot 0.5 \cdot 0.25 = 0.375.$

Dự đoán 3 (Sai):

Q: Ba người bạn chơi một trò chơi tung một đồng xu cân đối. Mỗi người tung đồng xu một lần. Xác suất để có đúng một người tung được mặt sấp là bao nhiêu?

A: Hãy suy nghĩ từng bước một.
Vì mỗi lần tung đồng xu có 50% khả năng ra mặt sấp và 50% khả năng ra mặt ngửa, và ta đang tìm trường hợp chỉ một trong ba lần tung ra mặt sấp, thoạt đầu ta có thể nghĩ đây đơn giản là việc chọn lần tung nào ra mặt sấp. Có ba lần tung, nên người ta có thể giả định sai rằng khả năng chọn được một kết quả cụ thể như vậy là 1 trên 3. Do đó, họ có thể kết luận rằng xác suất có đúng một mặt sấp là $1/3 = 33.3\%$.

Dự đoán 1 và 2 xác định đúng ba trường hợp có đúng một mặt sấp, cả hai đều thu được xác suất 37.5%. Lập luận trong Dự đoán 3 không tính đến tổng số kết quả có thể có khi tung đồng xu ba lần, nên cho đáp án sai là 33.3%. Vì vậy, ta chọn 37.5% làm đáp án cuối cùng vì đó là đáp án được đồng thuận.

Chú giải của người dịch: Kiểm tra lại bài toán đồng xu

Ba lần tung có $2^3 = 8$ kết quả đồng khả năng. Kết quả có đúng một mặt sấp (H) là HTT, THT, TTH: 3 kết quả, nên xác suất là $3/8 = 0.375 = 37.5\%$. Cách 2 dùng công thức nhị thức quen thuộc ở lớp 11 (*công thức Bernoulli*): $C_3^1 \cdot 0.5^1 \cdot 0.5^2 = 3 \cdot 0.125 = 0.375$, với $\binom{n}{k} = C_n^k$. Dự đoán 3 sai vì nhầm "chọn 1 trong 3 lần tung" thành xác suất $1/3$, quên rằng mỗi lần tung còn có thể ra sấp hoặc ngửa. (Lưu ý: Dự đoán 1 có một câu lập luận thừa, hơi lủng củng — "chỉ HHH, HHT, HTH và THH là liên quan" — nhưng vẫn đi đến đáp án đúng; LLM thật cũng thường mắc những lỗi nhỏ như vậy.) Tự nhất quán giống như hỏi ý kiến nhiều người rồi chọn đáp án được nhiều người đưa ra nhất: 2 trong 3 lời giải cho 0.375, nên chọn 0.375.

Tự nhất quán cung cấp một tiêu chí để xác định dự đoán tốt nhất trong một tập ứng viên. Vì prompt và mô hình được giữ cố định trong phương pháp này, nó không hẳn là một phương pháp kết hợp prompt. Thay vào đó, nó có thể được xem là một trường hợp của các phương pháp kết hợp đầu ra, còn gọi là các phương pháp *lựa chọn giả thuyết* (*hypothesis selection*), vốn đã được khám phá từ lâu trong NLP, đặc biệt cho các bài toán sinh văn bản [Xiao và cộng sự, 2013]. Trong những phương pháp này, nhiều đầu ra được sinh ra bằng cách thay đổi kiến trúc hoặc tham số của mô hình. Mỗi

đầu ra sau đó được gán một điểm số theo một tiêu chí nào đó, và các đầu ra được xếp hạng lại dựa trên các điểm số này. Có nhiều cách định nghĩa hàm tính điểm, chẳng hạn đo mức độ đồng thuận giữa một đầu ra với các đầu ra khác, hay dùng một mô hình mạnh hơn để chấm lại điểm cho từng đầu ra[10]. Hình 9.2 so sánh các phương pháp kết hợp khác nhau cho LLM.

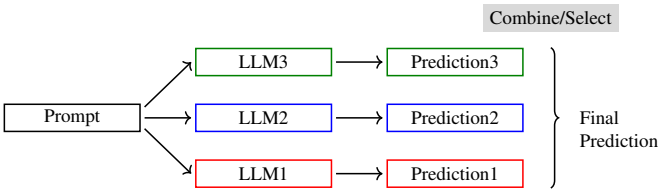
Chú giải của người dịch: Đọc Hình 9.2

Nhãn trong hình: *Prompt* = prompt, *Prediction* = dự đoán, *Combine/Select* = kết hợp/lựa chọn, *Final Prediction* = dự đoán cuối cùng, *Sample* = lấy mẫu; (a) *Model Ensembling* = kết hợp mô hình, (b) *Prompt Ensembling* = kết hợp prompt, (c) *Output Ensembling* = kết hợp đầu ra. Ba sơ đồ khác nhau ở chỗ "cái gì được nhân lên": nhiều mô hình (a), nhiều prompt (b), hay nhiều lần lấy mẫu từ cùng một mô hình với cùng một prompt (c) — tự nhất quán thuộc loại (c).

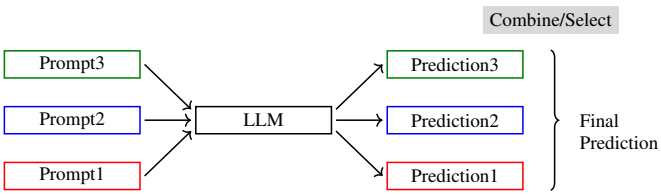
Bây giờ, hãy điểm lại ngắn gọn các phương pháp đã bàn trong mục này, như phân rã bài toán và tự tinh chỉnh. Rõ ràng các phương pháp này cải thiện việc ra quyết định bằng cách đưa thêm nhiều "lựa chọn" vào quá trình lập luận. Ở một mức độ nào đó, tất cả chúng đều liên quan tới việc đánh giá và đưa ra phản hồi về kết quả của LLM. Chẳng hạn, trong tự tinh chỉnh, ta cần đưa ra gợi ý để cải thiện dự đoán của LLM, còn trong kết hợp đầu ra, ta chọn đầu ra tối ưu từ một tập ứng viên. Theo nghĩa này, các phương pháp này thuộc nhóm rộng hơn gồm các cách tiếp cận *dự đoán rồi kiểm chứng*, trong đó dự đoán được đưa ra trước, sau đó được kiểm chứng và tinh chỉnh. Vấn đề cốt lõi ở đây là kiểm chứng và đánh giá kết quả suy luận hoặc các bước trung gian. Vấn đề này phần nào liên quan tới bài toán huấn luyện mô hình phần thưởng trong RLHF, mặc dù RLHF giải quyết một khía cạnh khác. Trên thực tế, việc phát triển các *bộ kiểm chứng* (*verifier*) đã được khám phá và triển khai trong lập luận bằng LLM.

[10]Một cách diễn giải tự nhất quán là xem nó như một quá trình tìm kiếm với *rủi ro Bayes cực tiểu* (*minimum Bayes risk*). Nó tìm đầu ra tốt nhất bằng cách cực tiểu hóa rủi ro Bayes. Cụ thể hơn, một hàm rủi ro $R(\mathbf{y}, \mathbf{y}_r)$ được định nghĩa trên mỗi cặp đầu ra (ký hiệu $(\mathbf{y}, \mathbf{y}_r)$), biểu thị chi phí khi thay $\mathbf{y}$ bằng $\mathbf{y}_r$. Cho một tập đầu ra Ω, rủi ro của một đầu ra $\mathbf{y} \in \Omega$ được cho bởi

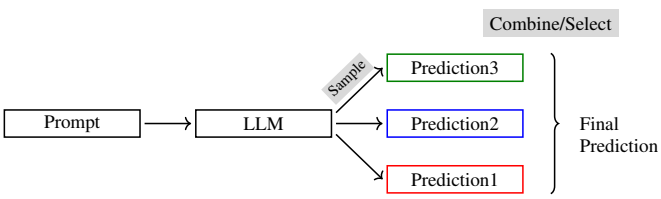
$$\begin{aligned}\text{Risk}(\mathbf{y}) &= \mathbb{E}_{\mathbf{y}_r \sim \Pr(\mathbf{y}_r|\mathbf{x})} R(\mathbf{y}, \mathbf{y}_r) \\ &= \sum_{\mathbf{y}_r \in \Omega} R(\mathbf{y}, \mathbf{y}_r) \cdot \Pr(\mathbf{y}_r|\mathbf{x}).\end{aligned} \tag{9.10}$$



(a) Model Ensembling



(b) Prompt Ensembling



(c) Output Ensembling

Hình 9.2: Minh họa các phương pháp kết hợp cho LLM. Trong kết hợp mô hình tiêu chuẩn (a), nhiều LLM khác nhau về kiến trúc hoặc tham số được sử dụng. Mỗi LLM nhận cùng một prompt và tạo ra một dự đoán. Các dự đoán này được kết hợp để sinh ra dự đoán cuối cùng. Trong kết hợp prompt (b), ta có một LLM và nhiều prompt. LLM tạo ra một dự đoán cho mỗi prompt, và các dự đoán này được kết hợp như thường lệ. Trong kết hợp đầu ra (c), LLM lấy mẫu nhiều dự đoán trên không gian dự đoán với một prompt cho trước. Nó có thể được xem là một phương pháp tăng cường hiệu năng của chính LLM. Lưu ý rằng các phương pháp kết hợp này có thể được phối hợp với nhau để tăng tính đa dạng của dự đoán. Ví dụ, ta có thể dùng cả kết hợp prompt lẫn kết hợp đầu ra để thu được các dự đoán đa dạng hơn.

Thay vì phát triển các thuật toán dựa trên quy tắc kinh nghiệm chạy lúc suy luận, phần lớn công trình tập trung vào việc học các bộ kiểm chứng theo cách có giám sát. Một phương pháp đơn giản là huấn luyện bộ kiểm chứng như các bộ phân loại nhị phân, chẳng hạn phân loại một câu trả lời là đúng hay sai, mặc dù các bộ kiểm chứng này thường được dùng như mô hình chấm điểm. Cho một đường lập luận của một bài toán, bộ kiểm chứng có thể được dùng để chấm điểm hoặc toàn bộ đường đi (gọi là các cách tiếp cận *dựa trên kết quả*, *outcome-based*) [Cobbe và cộng sự, 2021], hoặc từng bước lập luận riêng lẻ (gọi là các cách tiếp cận *dựa trên quá trình*, *process-based*) [Uesato và cộng sự, 2022; Lightman và cộng sự, 2024].

9.2.5 RAG và sử dụng công cụ

RAG thường được dùng khi các LLM tiêu chuẩn thiếu độ chính xác về sự kiện hoặc không truy cập được tri thức cập nhật. Bằng cách lấy thông tin từ các cơ sở dữ liệu và tài liệu bên ngoài, RAG có thể cải thiện đáng kể mức độ phù hợp với ngữ cảnh và tính có căn cứ về sự kiện của câu trả lời. Cách tiếp cận này đặc biệt hữu ích trong các tình huống đòi hỏi độ chính xác cao về sự kiện và thông tin cập nhật, như hỏi đáp phức tạp.

Mặc dù RAG đã được giới thiệu ở các chương trước, ở đây ta tóm tắt ngắn gọn các bước cốt lõi của nó cho đầy đủ.

- Ta chuẩn bị một tập văn bản đóng vai trò nguồn tri thức bên ngoài.
- Ta truy xuất các văn bản liên quan cho một truy vấn cho trước.
- Ta đưa cả các văn bản truy xuất được lẫn truy vấn vào LLM, rồi prompt nó tạo ra dự đoán cuối cùng.

Bước 1 và 2 có thể được thực hiện bằng một hệ thống truy xuất thông tin bên ngoài. Chẳng hạn, ta có thể lưu tập văn bản trong một cơ sở dữ liệu vectơ (*vector database*) rồi truy xuất các văn bản liên quan bằng các kỹ thuật tìm kiếm dựa trên vectơ. Vì truy xuất thông tin không phải trọng tâm của chương này, ta sẽ giả định rằng các hệ thống như vậy có sẵn và dùng trực tiếp chúng.

Ở đây ta trình bày cách prompt LLM để tận dụng các văn bản truy xuất được. Để minh họa, hãy xét một ví dụ dùng LLM để trả lời câu hỏi sau.

Thế vận hội 2028 sẽ được tổ chức ở đâu?

Ta có thể đơn giản nhập câu hỏi này vào một công cụ tìm kiếm trực tuyến. Nó sẽ trả về các đoạn văn bản liên quan tìm thấy trên internet, chẳng hạn,

> (Wikipedia)
> *Thế vận hội Mùa hè 2028, tên chính thức là Đại hội Thể thao Olympic lần thứ XXXIV và thường được gọi là Los Angeles 2028 hay LA28, là một sự kiện thể thao đa môn quốc tế sắp diễn ra, dự kiến tổ chức từ ngày 14 đến 30 tháng 7 năm 2028 tại Hoa Kỳ. …*
>
> (The Sporting News)
> *Năm 2028, Los Angeles sẽ trở thành thành phố thứ ba, sau London và Paris, đăng cai ba kỳ Thế vận hội, sau khi đã tổ chức Thế vận hội Mùa hè vào các năm 1932 và 1984. Đây cũng sẽ là lần đầu tiên Hoa Kỳ đăng cai một kỳ Thế vận hội kể từ Thế vận hội Mùa đông 2002 ở Salt Lake City. …*
>
> …

Ta có thể dùng các văn bản truy xuất được này làm ngữ cảnh bổ sung, và prompt LLM sinh câu trả lời dựa trên các văn bản đó. Dưới đây là một ví dụ prompt RAG.

> Nhiệm vụ của bạn là trả lời câu hỏi sau. Để giúp bạn, các văn bản liên quan được cung cấp kèm theo. Hãy dựa vào các văn bản này để trả lời.
>
> Câu hỏi:
> Thế vận hội 2028 sẽ được tổ chức ở đâu?
>
> Văn bản liên quan 1:
> Thế vận hội Mùa hè 2028, tên chính thức là Đại hội Thể thao Olympic lần thứ XXXIV và thường được gọi là Los Angeles 2028 hay LA28 …
>
> Văn bản liên quan 2:
> Năm 2028, Los Angeles sẽ trở thành thành phố thứ ba, sau London và Paris, đăng cai ba kỳ Thế vận hội sau khi …
>
> …
>
> Thế vận hội 2028 sẽ được tổ chức tại Los Angeles.

Prompt này giả định rằng các văn bản được cung cấp có liên quan tới câu hỏi và mong LLM sinh ra một câu trả lời trung thực dựa trên các văn

bản đó. Tuy nhiên, hệ thống truy xuất thông tin đôi khi có thể cung cấp các văn bản không liên quan hoặc không chính xác, điều này có thể khiến LLM đưa ra câu trả lời sai. Một cách đơn giản để giải quyết vấn đề này là cải thiện độ chính xác của hệ thống truy xuất thông tin. Tuy vậy, như với hầu hết các hệ thống AI, lỗi vẫn có thể xảy ra. Vì vậy, cũng cần tăng cường tính vững của LLM, để nó có thể đưa ra dự đoán hợp lý ngay cả khi đầu vào không chính xác. Dưới đây là một prompt mới giúp LLM trung thành hơn với sự thật, và cho phép nó chọn không trả lời khi thông tin được cung cấp không chính xác.

Nhiệm vụ của bạn là trả lời câu hỏi sau. Để giúp bạn, các văn bản liên quan được cung cấp kèm theo. Hãy dựa vào các văn bản này để trả lời.
Xin lưu ý rằng câu trả lời của bạn cần chính xác nhất có thể và trung thành với sự thật. Nếu thông tin được cung cấp không đủ để trả lời chính xác, bạn có thể chỉ cần xuất ra "Không có câu trả lời!".
Câu hỏi:
Thế vận hội 2028 sẽ được tổ chức ở đâu?
Văn bản liên quan 1:
Thế vận hội Mùa hè 2024, tên chính thức là Đại hội Thể thao Olympic lần thứ XXXIII và mang thương hiệu Paris 2024, là một sự kiện thể thao đa môn quốc tế …
…
Không có câu trả lời!

Trong ví dụ này, LLM từ chối trả lời vì thông tin được cung cấp không đủ và không liên quan tới câu hỏi.

Chú giải của người dịch: RAG giống làm bài "được mở tài liệu"

RAG giống như một bài kiểm tra cho phép mở tài liệu: trước khi trả lời, ta tìm vài trang sách liên quan rồi đặt chúng ngay trong đề bài để mô hình đọc và trả lời dựa vào đó, thay vì chỉ dựa vào "trí nhớ" từ lúc huấn luyện (vốn có thể đã cũ hoặc nhớ sai). Nhưng nếu "tài liệu" tìm được lại nói về chuyện khác — như trong ví dụ thứ hai, đoạn văn nói về Thế vận hội 2024 ở Paris chứ không phải 2028 — thì một học sinh trung thực nên trả lời "không đủ thông tin" thay vì đoán bừa. Dòng chỉ dẫn được tô xanh dạy cho mô hình đúng thái độ đó, giúp giảm hiện tượng ảo giác (*hallucination*) — tức mô hình "bịa" ra

câu trả lời nghe có vẻ hợp lý nhưng sai.

Cả RAG và tinh chỉnh đều là những phương pháp phổ biến để thích nghi LLM bằng dữ liệu riêng của nhiệm vụ. RAG tiêu chuẩn không cần huấn luyện và có thể áp dụng trực tiếp cho LLM. Để cải thiện RAG hơn nữa, cũng có thể tinh chỉnh LLM, dù việc này đòi hỏi một ít công sức huấn luyện. Chẳng hạn, ta có thể tinh chỉnh LLM bằng dữ liệu do con người gán nhãn để giám sát chúng học cách từ chối trả lời. Lưu ý rằng mặc dù các ví dụ ở trên có vẻ đơn giản, RAG không hề tầm thường. Từ góc độ thiết kế prompt, các trường hợp sử dụng khác nhau có thể cần các prompt khác nhau, mặc dù mục tiêu rộng hơn của ta là phát triển các chiến lược prompting có thể khái quát hóa qua nhiều nhiệm vụ. Trong nhiều trường hợp, ta cần kiểm soát mức độ phụ thuộc vào ngữ cảnh truy xuất được khi đưa ra dự đoán. Đôi khi, LLM phải sinh câu trả lời hoàn toàn dựa trên các văn bản được cung cấp, trong khi những lúc khác, chúng có thể cần sinh câu trả lời bằng tri thức sẵn có của mình nếu các văn bản được cung cấp không đủ. Có nhiều khía cạnh của RAG, như cải tiến hệ thống truy xuất, không thể đề cập trong chương này. Bạn đọc quan tâm có thể tham khảo các bài tổng quan về kỹ thuật RAG để biết thêm thông tin [Li và cộng sự, 2022d; Gao và cộng sự, 2023c].

Một lý do ta bàn về RAG ở đây là nó có thể được xem, theo nghĩa rộng, như một trường hợp của khung phân rã bài toán tổng quát (xem Mục 9.2.2). RAG chia việc giải bài toán thành hai bước. Ở bước thứ nhất, ta thu thập thông tin liên quan và hỗ trợ cho một truy vấn cho trước từ nhiều nguồn tri thức khác nhau. Ở bước thứ hai, ta dùng LLM để sinh câu trả lời dựa trên thông tin đã thu thập. Nếu mở rộng khái niệm phân rã bài toán hơn nữa, ta sẽ thấy rằng nhiều nhiệm vụ đòi hỏi dùng các hệ thống hay công cụ bên ngoài có thể được xử lý như những bài toán tương tự. Một ví dụ như vậy là *sử dụng công cụ* (*tool use*) trong LLM. Trong nhiều ứng dụng, LLM cần dùng các cơ sở dữ liệu bên ngoài, các API, và thậm chí cả các công cụ mô phỏng để sinh ra câu trả lời chính xác. Ví dụ, LLM có thể truy cập dữ liệu thời gian thực từ thị trường tài chính để cung cấp thông tin tài chính, hoặc truy cập các cơ sở tri thức y khoa để đưa ra thông tin y tế được cá nhân hóa. Sự tích hợp này mở rộng năng lực của LLM bằng cách cho phép chúng tương tác với các hệ thống bên ngoài, và trong một số bối cảnh, tác động tới hoặc điều khiển các hệ thống đó. Nhờ vậy, LLM

có thể hoạt động như những *tác tử tự hành* (*autonomous agent*) đơn giản chứ không chỉ là những bộ sinh văn bản [Franklin và Graesser, 1996].

Sử dụng công cụ là một lĩnh vực nghiên cứu rộng. Ở đây ta thu hẹp thảo luận vào những nhiệm vụ có thể được hỗ trợ bằng cách gọi các API bên ngoài để giải một số bài toán con [Parisi và cộng sự, 2022; Gao và cộng sự, 2023b]. Hãy xét lại ví dụ yêu cầu LLM trả lời câu hỏi "Thế vận hội 2028 sẽ được tổ chức ở đâu?". Giả sử LLM có thể truy cập một công cụ tìm kiếm web. Khi đó ta có thể prompt LLM trả lời câu hỏi bằng tìm kiếm web như sau:

> Nhiệm vụ của bạn là trả lời câu hỏi sau. Bạn có thể dùng các công cụ bên ngoài, như tìm kiếm web, để hỗ trợ.
>
> Câu hỏi:
> Thế vận hội 2028 sẽ được tổ chức ở đâu?
>
> Thông tin liên quan đến câu hỏi này như sau:
> `{tool: web-search, query: "2028 Olympics"}`
> Vậy câu trả lời là: Los Angeles

Ở đây `tool: web-search, query: "2028 Olympics"` biểu thị một yêu cầu gửi tới hệ thống tìm kiếm web với truy vấn "2028 Olympics" (Thế vận hội 2028). Lệnh gọi công cụ được sinh ra sẽ bị một hệ thống bên ngoài chặn lại; hệ thống này thực thi công cụ tương ứng và nối kết quả trả về vào ngữ cảnh. Ngữ cảnh đã cập nhật sau đó được đưa trở lại LLM cho các bước dự đoán tiếp theo, và nhờ vậy mô hình có thể sinh ra câu trả lời đúng bằng thông tin truy xuất được.

Hãy xét một ví dụ khác, trong đó ta yêu cầu LLM giải một bài toán.

> Bài toán:
> Cần bơm đầy nước vào một bể bơi. Bể dài 10 mét, rộng 4 mét và sâu 2 mét. Hãy tính thể tích của bể theo mét khối, rồi xác định cần bao nhiêu lít nước để làm đầy bể (biết 1 mét khối bằng 1000 lít).
>
> Lời giải:
> Để giải bài toán này, LLM cần tính thể tích của bể trước bằng công thức thể tích hình hộp chữ nhật: Dài $\times$ Rộng $\times$ Sâu. Do đó, thể tích là $10\,\text{m} \times 4\,\text{m} \times 2\,\text{m} =$ `{tool: calculator, expression: 10 * 4 * 2}` m^3. Tiếp theo, để biết cần bao nhiêu lít nước, LLM nhân thể tích tính bằng

mét khối với 1000 (vì 1 mét khối bằng 1000 lít). Như vậy, 80 $\times 1000$ = {tool: calculator, expression: 80*1000} lít.

Ở đây chuỗi `{tool: calculator, expression: 10 * 4 * 2}` kích hoạt việc gọi một trình thông dịch toán học để tính kết quả của biểu thức. Lưu ý rằng kết quả (tức 80) sẽ thay thế `{tool: calculator, expression: 10 * 4 * 2}` và có thể được tham chiếu trong các bước dự đoán token tiếp theo. Chẳng hạn, ở bước cuối của lời giải, 80 được dùng thay cho `{tool: calculator, expression: 10 * 4 * 2}`.

Chú giải của người dịch: LLM "gọi công cụ" như thế nào?

LLM không tự bấm máy tính hay tự lên mạng. Nó chỉ *viết ra* một đoạn văn bản theo khuôn dạng quy ước, như `{tool: calculator, expression: 10 * 4 * 2}`. Một chương trình bên ngoài luôn "theo dõi" văn bản mô hình đang sinh; hễ thấy khuôn dạng này, nó tạm dừng mô hình, tự tính $10 \cdot 4 \cdot 2 = 80$, thay đoạn lệnh bằng số 80, rồi cho mô hình viết tiếp. Vì thế mô hình "thấy" như thể chính nó đã viết ra số 80, và có thể dùng tiếp ở bước sau ($80 \times 1000 = 80\,000$ lít). Cách làm này giống như học sinh được phép dùng máy tính bỏ túi: học sinh lo phần lập luận (dùng công thức nào), còn máy lo phần tính toán cho chính xác.

Một điểm khác biệt chính giữa các ví dụ sử dụng công cụ ở đây và các ví dụ RAG đã bàn trước đó là trong sử dụng công cụ, các hàm bên ngoài có thể được gọi *trong lúc* suy luận. Ngược lại, trong RAG, các văn bản truy xuất được cung cấp *trước khi* quá trình dự đoán bắt đầu. Tuy nhiên, từ góc độ mô hình hóa ngôn ngữ, chúng tương tự nhau: trước khi sinh kết quả cuối cùng, ta dùng các công cụ bên ngoài, một cách thủ công hoặc tự động, để có được ngữ cảnh đầy đủ và liên quan. Một cách diễn giải ở mức cao về các cách tiếp cận này là cả hai đều dựa vào một "tác tử" có thể xác định chỗ nào và cách nào để gọi các hàm bên ngoài nhằm tạo ra ngữ cảnh cần thiết cho việc dự đoán.

Một vấn đề với việc sử dụng công cụ là LLM ban đầu không được huấn luyện để sinh ra các dấu hiệu cần thiết cho việc dùng công cụ. Vì vậy, ta cần tinh chỉnh LLM để thích nghi nó với các nhiệm vụ này [Schick và cộng sự, 2024]. Vì chương này tập trung vào prompting, ta sẽ không trình bày chi tiết quá trình tinh chỉnh này. Nói đơn giản, trước tiên ta cần

gán nhãn dữ liệu. Với mỗi ví dụ tinh chỉnh, ta thay những phần của đầu ra cần dùng công cụ bên ngoài bằng các lệnh hoặc dấu hiệu được định nghĩa trước. Sau đó, ta dùng dữ liệu đã gán nhãn này để tinh chỉnh các tham số của LLM như thường lệ. Kết quả là LLM có thể có được khả năng sinh ra các lệnh gọi công cụ bên ngoài. Trong lúc suy luận, ta có thể thực thi các lệnh dùng công cụ này trong đầu ra của mô hình để nhận trợ giúp từ các công cụ bên ngoài.

9.3 Học cách tạo prompt

Cho đến giờ trong chương này, chúng ta đã xem xét một số chiến lược prompting cơ bản và nhiều cải tiến khác nhau cho chúng. Tuy nhiên, tất cả các prompt mà ta đã bàn đều được thiết kế thủ công. Điều này dẫn đến một số vấn đề. Thứ nhất, thiết kế prompt chất lượng cao là việc khó và thường đòi hỏi thử nghiệm rất nhiều để tìm ra các prompt hiệu quả. Vì các LLM khác nhau có thể nhạy cảm ở mức khác nhau với các phong cách prompt, việc xây dựng những prompt tổng quát hóa tốt trên nhiều mô hình có thể tốn nhiều tài nguyên. Thứ hai, thiết kế prompt thủ công phụ thuộc nặng nề vào chuyên môn của con người, điều này có thể hạn chế sự đa dạng của các cách tiếp cận và bỏ sót những prompt có tiềm năng hiệu quả nhưng con người không nhận ra ngay. Thứ ba, các prompt do con người tạo ra có thể phức tạp và dài dòng, dẫn đến đầu vào dài hơn cho LLM và chi phí tính toán cao hơn.

Trong mục này, chúng ta bàn về các kỹ thuật prompting tự động (*automated prompting*). Các phương pháp này nhằm tự động tạo ra, tối ưu hóa và biểu diễn prompt sao cho các nhiệm vụ hạ nguồn (*downstream task*) có thể được giải quyết hiệu quả hơn (cả về chất lượng lẫn chi phí). Cụ thể, ở đây ta xem xét ba vấn đề.

- Làm thế nào để tự động hóa quá trình thiết kế và tối ưu hóa prompt cho LLM?
- Có những cách biểu diễn prompt nào khác ngoài chuỗi văn bản thuần túy hay không, và làm thế nào để học được những biểu diễn đó?
- Làm thế nào để prompt ngắn gọn và cô đọng hơn, qua đó giảm độ

phức tạp và độ dài của chúng?

Lưu ý rằng có nhiều bối cảnh trong đó ta có thể nghiên cứu các vấn đề này. Ví dụ, prompt có thể được tối ưu hóa cho một LLM cụ thể, hoặc được thiết kế để tổng quát hóa trên nhiều LLM khác nhau. Những bối cảnh này có thể dẫn đến các phương pháp và kịch bản ứng dụng khác nhau, nhưng các phương pháp ấy có thể trùng lặp nhau ở một số khía cạnh. Trong phần thảo luận dưới đây, ta sẽ đề cập đến một vài kịch bản khác nhau và bàn về mối liên hệ giữa các phương pháp.

9.3.1 Tối ưu hóa prompt

Vì thiết kế prompt tốn nhiều công sức, ta mong muốn dùng các mô hình học máy để tìm ra prompt tối ưu cho một nhiệm vụ cụ thể (gọi là **thiết kế prompt tự động** (*automatic prompt design*) hay **tối ưu hóa prompt** (*prompt optimization*)). Cách tiếp cận này, nói rộng ra, có thể được xem là một trường hợp của **học máy tự động** (*automated machine learning*) (**AutoML**), lĩnh vực nhằm giảm bớt hoặc loại bỏ nhu cầu thiết kế mô hình học máy thủ công bởi chuyên gia. Mặc dù trọng tâm của ta ở đây là thiết kế prompt, bản thân prompt là những cấu trúc rời rạc. Do đó, tối ưu hóa prompt tương tự như việc tìm kiếm trên các kiến trúc mô hình rời rạc trong học máy. Một lĩnh vực có liên quan chặt chẽ là **tìm kiếm kiến trúc nơ-ron** (*neural architecture search*) (**NAS**), trong đó các mạng nơ-ron tối ưu được xác định bằng cách khám phá một không gian các mạng nơ-ron khả dĩ [Zoph và Le, 2016; Elsken và cộng sự, 2019a]. Nếu xem tối ưu hóa prompt là một quá trình tìm kiếm, ta có thể mô tả một khung tối ưu hóa prompt tổng quát gồm các thành phần sau:

- **Không gian tìm kiếm prompt** (*Prompt Search Space*). Thành phần này xác định mọi prompt khả dĩ mà thuật toán có thể khám phá. Ví dụ, ta có thể chỉnh sửa một số prompt hạt giống (*seed prompt*) để sinh ra một tập các prompt ứng viên đa dạng.
- **Ước lượng hiệu năng** (*Performance Estimation*). Khi một prompt đã được chọn, nó cần được đánh giá. Ví dụ, một cách đơn giản là đưa nó vào một LLM và đo hiệu năng trên một tập kiểm định (*validation set*).
- **Chiến lược tìm kiếm** (*Search Strategy*). Quá trình tìm kiếm nhìn

chung giống với quá trình được dùng trong nhiều hệ thống AI. Ở mỗi bước, hệ thống khám phá một tập các prompt triển vọng trong không gian tìm kiếm và đánh giá chúng. Quá trình này tiếp diễn khi ngày càng nhiều prompt được khám phá. Kết quả của việc tìm kiếm là prompt có hiệu năng tốt nhất quan sát được cho đến khi việc tìm kiếm dừng lại.

Chú giải của người dịch: Tối ưu hóa prompt giống việc gì quen thuộc?

Hãy tưởng tượng bạn cần viết một "câu đề bài" sao cho một bạn học sinh (ở đây là LLM) làm bài đúng nhiều nhất. Bạn thử viết vài phiên bản đề (*không gian tìm kiếm*), cho bạn ấy làm thử 50 câu đã biết đáp án rồi đếm số câu đúng (*ước lượng hiệu năng*), giữ lại vài phiên bản tốt nhất, sửa chúng thành phiên bản mới rồi thử tiếp (*chiến lược tìm kiếm*). Tối ưu hóa prompt là tự động hóa vòng lặp "thử – chấm điểm – sửa" này bằng máy tính. Gọi prompt là "rời rạc" vì nó là một dãy từ: ta không thể "nhích" một từ đi một lượng nhỏ như nhích một số thực, mà chỉ có thể thay từ này bằng từ khác.

Đây là một khung rất tổng quát, và các hệ thống tối ưu hóa prompt khác nhau có thể khác nhau ở cách thiết kế từng thành phần. Một cách tiếp cận được dùng rộng rãi là lấy LLM làm nền tảng để xây dựng các thành phần này. Ban đầu, một vài prompt được cung cấp. Sau đó, quá trình sau được lặp lại cho đến khi thỏa mãn một tiêu chí dừng: 1) các prompt được đánh giá trên một tập kiểm định; 2) một tập ứng viên (*candidate pool*) được duy trì bằng cách chỉ giữ lại những prompt triển vọng nhất; và 3) các prompt mới được tạo ra bằng cách dùng LLM suy ra những prompt tương tự từ tập ứng viên này. Một lợi ích của cách tiếp cận này là nó cho phép ta dùng các LLM có sẵn để thực hiện các nhiệm vụ nói trên mà không cần phát triển hệ thống phức tạp. Để làm được điều này, ta có thể prompt hoặc tinh chỉnh LLM để thích nghi chúng với các nhiệm vụ này. Ở đây ta xét phương pháp của Zhou và cộng sự [2023c] để minh họa việc tối ưu hóa prompt dựa trên LLM. Phương pháp này gồm các bước sau.

- **Khởi tạo** (*Initialization*). Gọi C là tập ứng viên các prompt cần khám phá. Bước đầu tiên là thêm các prompt ban đầu vào C. Ta có thể làm việc này theo vài cách. Một cách đơn giản là tự tay viết các prompt như vậy cho một nhiệm vụ cho trước. Tuy nhiên, trong nhiều trường hợp con người có hiểu biết hạn chế về cách viết prompt hiệu

quả cho nhiệm vụ, việc xây dựng prompt trở nên khó khăn. Trong những trường hợp đó, ta nên dùng LLM để sinh prompt. Ví dụ, ta có thể trực tiếp yêu cầu LLM tạo ra prompt, kèm theo mô tả về nhiệm vụ.

> Bạn được giao một nhiệm vụ cần hoàn thành bằng LLM. Hãy viết một prompt để hướng dẫn LLM.
> {∗mô-tả-nhiệm-vụ∗}
>
> ___

Phương pháp này đơn giản, nhưng nó vẫn cần một mô tả nhiệm vụ do con người cung cấp. Một phương pháp khác là dùng LLM để sinh prompt khi được cho các ví dụ về đầu vào và đầu ra của nhiệm vụ. Dưới đây là một khuôn mẫu prompt (*prompt template*).

> Bạn được cung cấp một vài cặp đầu vào – đầu ra của một nhiệm vụ. Hãy viết một chỉ dẫn để thực hiện nhiệm vụ này.
> Đầu vào: {∗đầu-vào-1∗} Đầu ra: {∗đầu-ra-1∗}
> Đầu vào: {∗đầu-vào-2∗} Đầu ra: {∗đầu-ra-2∗}
> ...
>
> ___

Bằng cách này, LLM có thể suy ra chỉ dẫn tương ứng cho nhiệm vụ từ các đầu vào và đầu ra được cung cấp.

- **Đánh giá** (*Evaluation*). Khi đã có tập ứng viên C, ta cần đánh giá các prompt trong C. Một cách là đưa từng prompt vào một LLM và đánh giá kết quả trên nhiệm vụ hạ nguồn. Ví dụ, ta có thể đánh giá đầu ra của LLM với một đầu vào cho trước bằng một độ đo (*metric*) định sẵn, hoặc dùng log-hợp lý (*log-likelihood*) của đầu ra làm thước đo chất lượng của prompt.

- **Cắt tỉa** (*Pruning*). Nếu C chứa một số lượng lớn prompt, việc cắt bỏ những prompt kém triển vọng trong đó là hợp lý, nhờ vậy giảm gánh nặng tính toán ở các bước sau. Đây là một bài toán cắt tỉa tiêu chuẩn. Với điểm đánh giá của mỗi prompt, một cách đơn giản là chỉ giữ lại một tỷ lệ phần trăm nhất định các prompt và loại bỏ phần còn lại.

- **Mở rộng** (*Expansion*). Mở rộng là một thao tác then chốt trong các thuật toán tìm kiếm, dùng để khám phá các trạng thái khác nhau trong không gian tìm kiếm. Thao tác mở rộng ở đây có thể được định nghĩa là một hàm:

$$C' = \text{Expand}(C, f), \tag{9.11}$$

trong đó C' là tập các prompt mới được sinh ra từ C bằng mô hình f. Nếu xem f là một LLM, ta có thể thực hiện thao tác mở rộng bằng cách yêu cầu f sinh ra các prompt mới và có liên quan dựa trên C. Dưới đây là một ví dụ.

> Dưới đây là một prompt dành cho LLM. Hãy đưa ra một vài prompt mới để thực hiện cùng nhiệm vụ đó.
> Đầu vào: {∗prompt∗}
>
> ___

Sau đó, ta thay C bằng C'. Các bước đánh giá, cắt tỉa và mở rộng có thể được lặp lại, nhờ đó ta có thể dần dần khám phá một phạm vi prompt rộng hơn.

Chú giải của người dịch: Một vòng lặp tìm kiếm cụ thể

Giả sử nhiệm vụ là phân loại cảm xúc của câu nhận xét. Khởi tạo: C gồm 10 prompt do LLM viết. Đánh giá: mỗi prompt được thử trên 100 câu đã có nhãn, ví dụ prompt A đúng 81 câu, prompt B đúng 74 câu, … Cắt tỉa: giữ 3 prompt điểm cao nhất (giữ 30%). Mở rộng: nhờ LLM viết lại mỗi prompt còn lại thành 3–4 biến thể, được tập C' khoảng 10 prompt mới. Lặp lại vài vòng, cuối cùng chọn prompt có điểm cao nhất. Trong Phương trình (9.11), Expand chỉ là tên của thao tác "sinh prompt mới"; f là mô hình dùng để sinh. *Log-hợp lý* của đầu ra là logarit của xác suất mà LLM gán cho đáp án đúng: xác suất càng gần 1 thì log càng gần 0, tức prompt càng "dẫn" mô hình tới đáp án đúng một cách tự tin.

Trong tối ưu hóa prompt, bước mở rộng đóng vai trò then chốt, vì nó xác định cách ta khám phá không gian tìm kiếm, và mục tiêu của ta là tìm ra kết quả tối ưu với ít công sức nhất. Một cải tiến cho bước này là coi bài toán như một nhiệm vụ diễn đạt lại (*paraphrasing*). Một cách đơn giản là áp dụng các hệ thống diễn đạt lại có sẵn, dựa trên LLM hoặc các mô

hình khác, để biến đổi prompt đầu vào thành các dạng tương đương về ngữ nghĩa [Jiang và cộng sự, 2020]. Ngoài ra, ta có thể định nghĩa các thao tác chỉnh sửa cụ thể, chẳng hạn chèn và sửa đổi, cho mỗi token. Một prompt cho trước có thể được chỉnh sửa thành các prompt mới bằng cách áp dụng các thao tác này [Prasad và cộng sự, 2023]. Đồng thời, có thể áp dụng thêm đánh giá và cắt tỉa để lọc bỏ các prompt chất lượng thấp. Ngoài việc coi việc sinh prompt là một bài toán diễn đạt lại, ta có thể cải thiện chất lượng prompt trong bước mở rộng bằng cách học từ phản hồi (*feedback*) [Pryzant và cộng sự, 2023]. Cách tiếp cận này có phần liên quan đến vấn đề tự tinh chỉnh (*self-refinement*) đã bàn ở Mục 9.2.3. Một LLM có thể được dùng để sinh phản hồi về một prompt đầu vào, và prompt này sau đó được sửa lại dựa trên phản hồi đó. Chu trình phản hồi – sửa đổi này có thể được lặp lại nhiều lần cho đến khi kết quả hội tụ hoặc đạt được kết quả mong muốn.

Một cách tiếp cận khác cho tối ưu hóa prompt là áp dụng các kỹ thuật tối ưu hóa kinh điển. Ví dụ, bài toán có thể được phát biểu thành một bài toán tính toán tiến hóa (*evolutionary computation*), trong đó các prompt được coi là các ứng viên tiến hóa qua từng thế hệ khi quá trình tối ưu hóa diễn ra [Guo và cộng sự, 2024]. Vì nhiều thuật toán tối ưu hóa mạnh đã được phát triển trong các lĩnh vực liên quan, chúng có thể được áp dụng trực tiếp cho bài toán này.

Chú giải của người dịch: Tính toán tiến hóa

Ý tưởng mượn từ thuyết tiến hóa trong sinh học (lớp 12): trong một "quần thể" prompt, các prompt có điểm cao ("thích nghi tốt") được giữ lại làm "bố mẹ"; con cháu được tạo ra bằng cách "lai" (ghép một nửa prompt này với một nửa prompt kia) và "đột biến" (thay vài từ). Qua nhiều thế hệ, chất lượng trung bình của quần thể tăng dần, giống chọn lọc tự nhiên.

Trong thực tế, ta có thể muốn dùng các API LLM sẵn có để hiện thực các bước nói trên. Tuy nhiên, cách làm như vậy phụ thuộc nhiều vào khả năng lập luận và học trong ngữ cảnh của LLM. Nếu các LLM này không đủ mạnh và chưa được thích nghi với các nhiệm vụ, chúng có thể đưa lỗi vào quá trình tìm kiếm, chẳng hạn sinh ra các prompt sai trong bước mở rộng. Trong những trường hợp đó, tốt hơn là huấn luyện các mô hình phù hợp hơn với nhiệm vụ. Một cách tiếp cận theo hướng nghiên cứu này sử dụng học tăng cường (*reinforcement learning*), vốn đã được dùng rộng rãi

để giải các bài toán ra quyết định và tối ưu hóa rời rạc. Ví dụ, Deng và cộng sự [2022] đã phát triển một bộ sinh prompt (*prompt generator*) bằng cách tích hợp một bộ thích nghi (*adaptor*) dựa trên FFN vào một LLM. Bộ sinh prompt được huấn luyện như một mạng chính sách (*policy network*) điển hình, nhưng chỉ các tham số của bộ thích nghi được cập nhật, còn các tham số còn lại của mô hình được giữ nguyên. Trong quá trình huấn luyện, phần thưởng (*reward*) có được bằng cách kiểm tra các prompt sinh ra bằng một LLM khác, tương tự phương pháp đánh giá đã bàn ở trên. Khi huấn luyện xong, bộ sinh prompt được dùng để sinh ra các prompt mới.

Chú giải của người dịch: Học tăng cường ở đây hoạt động thế nào?

Có thể hình dung bộ sinh prompt như một người chơi: mỗi lượt nó "viết" ra một prompt (một chuỗi quyết định chọn từng token), rồi nhận về một điểm thưởng — chẳng hạn tỷ lệ câu trả lời đúng khi đưa prompt đó cho một LLM khác. Prompt nào được điểm cao thì lần sau nó có xu hướng viết giống thế hơn. "Mạng chính sách" là mạng nơ-ron quyết định "nên chọn token nào tiếp theo". Chỉ một phần nhỏ (bộ thích nghi) được học, phần LLM khổng lồ còn lại để nguyên, nên việc huấn luyện rẻ hơn nhiều.

Lưu ý rằng trong thảo luận ở đây, prompt đơn giản được xem như các chuỗi token, và đầu ra của tối ưu hóa prompt là một chuỗi như vậy. Tuy nhiên, nói một cách chặt chẽ, prompt thường gồm nhiều thành phần, chẳng hạn chỉ dẫn, ví dụ minh họa và đầu vào của người dùng. Mặc dù các cách tiếp cận đã bàn phần lớn mang tính tổng quát, nhiều công trình về tối ưu hóa prompt tập trung vào việc học các chỉ dẫn tốt hơn cho prompting. Cụ thể, mục tiêu là sinh ra các chỉ dẫn dẫn dắt LLM một cách hiệu quả dựa trên một nhiệm vụ cho trước. Dĩ nhiên, khái niệm tối ưu hóa prompt cũng có thể được mở rộng sang việc học các phần khác của prompt. Ví dụ, đã có rất nhiều quan tâm nghiên cứu về việc học cách chọn hoặc sinh các ví dụ minh họa trong CoT [Liu và cộng sự, 2022; Rubin và cộng sự, 2022; Zhang và cộng sự, 2023b]. Một trong những khác biệt giữa học chỉ dẫn và học ví dụ minh họa là việc sinh ví dụ minh họa chất lượng cao bằng LLM tương đối dễ, và trọng tâm của học ví dụ minh họa thường là cách lấy mẫu những ví dụ minh họa phù hợp từ một tập ứng viên. Ngược lại, khó khăn trong việc học chỉ dẫn một phần là do các LLM đã tiền huấn luyện không phù hợp để dự đoán chất lượng của chỉ dẫn, và việc kiểm tra

các chỉ dẫn này trên nhiệm vụ hạ nguồn tốn kém về tính toán. Điều này khiến các phương pháp tối ưu hóa tốn kém khi áp dụng, và việc khám phá nhiều loại chỉ dẫn khác nhau đặt ra những thách thức đáng kể.

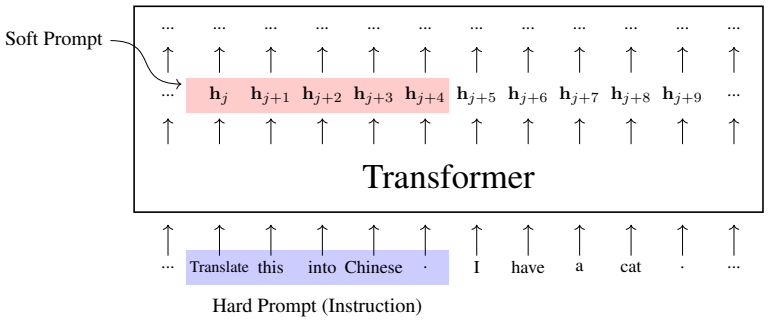
9.3.2 Prompt mềm

Mặc dù xây dựng prompt bằng ngôn ngữ tự nhiên, dù thủ công hay tự động, là một cách tiếp cận đơn giản và được áp dụng rộng rãi, nó vẫn có một số vấn đề. Một vấn đề là prompt bằng ngôn ngữ tự nhiên có thể phức tạp và dài, gây ra gánh nặng tính toán đáng kể khi được LLM xử lý. Trong nhiều ứng dụng, người dùng có thể cần thực hiện một nhiệm vụ lặp đi lặp lại, và việc nhập cùng một prompt dài vào LLM rất nhiều lần rõ ràng là kém hiệu quả. Một vấn đề khác là mặc dù prompt thường được biểu diễn dưới dạng các chuỗi token rời rạc (gọi là **prompt cứng** (*hard prompt*)) trong đầu vào LLM thông thường, LLM lại mã hóa chúng thành các vectơ liên tục. Điều này đặt ra câu hỏi liệu có cách nào gọn hơn và hiệu quả hơn để biểu diễn prompt hay không.

Trong tiểu mục này, chúng ta giới thiệu khái niệm **prompt mềm** (*soft prompt*), có thể được xem là các biểu diễn phân tán (*distributed representation*) ẩn của prompt. Khi prompt một LLM, ta quan tâm đến việc truyền đạt nhiệm vụ hoặc câu hỏi để gợi ra những câu trả lời mong muốn. Ta có thể định nghĩa prompt cứng là các chuỗi văn bản tường minh, định sẵn mà người dùng nhập trực tiếp vào LLM để dẫn dắt câu trả lời. Ngược lại, ta có thể nghĩ về prompt mềm như những khuôn mẫu prompting ngầm, có thể thích nghi, được biểu diễn trong không gian ẩn (*hidden space*) liên tục của LLM. Khác với prompt cứng, vốn được diễn đạt bằng ngôn ngữ tự nhiên và cần dễ hiểu đối với con người, prompt mềm được mã hóa ở một định dạng dễ hiểu đối với mô hình hơn là đối với con người. Để minh họa, hãy xét một prompt đơn giản

> Translate the sentence into Chinese. (Dịch câu này sang tiếng Trung.)
>
> Cứ coi như xong rồi!
>
> ___

Ở đây, chỉ dẫn "Translate the sentence into Chinese" có thể được xem là một prompt cứng, ký hiệu bằng chuỗi token $c_1 \cdots c_5$. Khi đưa các token



Hình 9.3: Minh họa prompt cứng và prompt mềm. Ở đây prompt cứng là chỉ dẫn mà ta đưa vào LLM để thực hiện nhiệm vụ. LLM mã hóa chỉ dẫn này như thường lệ, và các biểu diễn trung gian tương ứng với chỉ dẫn có thể được xem như một dạng prompt mềm.

này vào một LLM, chúng được biến đổi thành một chuỗi các vectơ giá trị thực $\mathbf{h}_1 \cdots \mathbf{h}_5$, mỗi vectơ tương ứng với một token. Ta có thể xem $\mathbf{h}_1 \cdots \mathbf{h}_5$ như một minh họa trực quan cho prompt mềm, như trong Hình 9.3.

Chú giải của người dịch: Đọc Hình 9.3

Hàng dưới cùng là các token đầu vào: phần tô màu "Translate this into Chinese ." (*Dịch câu này sang tiếng Trung.*) là *prompt cứng* (*Hard Prompt (Instruction)* — prompt cứng, dạng chỉ dẫn), tiếp theo là câu cần dịch "I have a cat." (*Tôi có một con mèo.*). Bên trong Transformer, mỗi token được biến thành một vectơ $\mathbf{h}$ — một dãy hàng trăm, hàng nghìn con số. Các vectơ $\mathbf{h}_j, \ldots, \mathbf{h}_{j+4}$ ứng với phần chỉ dẫn (được tô màu, nhãn *Soft Prompt*) chính là một "prompt mềm": cùng mang ý nghĩa "hãy dịch sang tiếng Trung" nhưng ở dạng số mà chỉ máy "đọc" được. Chữ "cứng" (rời rạc, chọn từ một bộ từ vựng hữu hạn) và "mềm" (liên tục, mỗi tọa độ có thể nhận bất kỳ số thực nào) gợi lại sự khác nhau giữa tập số nguyên và tập số thực.

Mặc dù ví dụ trên cho thấy prompt mềm có thể được sinh ra bằng cách biến đổi prompt cứng, giữa chúng không nhất thiết có sự tương ứng trực tiếp. Thực ra, ta thậm chí không cần diễn giải prompt mềm bằng văn bản có nghĩa. Thay vào đó, prompt mềm được biểu diễn dưới dạng các vectơ liên tục trong không gian ẩn của LLM và có thể được tối ưu hóa trực tiếp như những tham số học được. Cách xử lý như vậy cho phép ta khám phá

các phương pháp prompting vượt ra ngoài văn bản. Một lợi ích nữa là prompt mềm cung cấp các biểu diễn dày đặc, học được, để mã hóa cách ta dẫn dắt LLM sinh ra những đầu ra cụ thể. Việc huấn luyện và áp dụng các biểu diễn này đòi hỏi chi phí tính toán thấp hơn đáng kể so với chi phí cần thiết để xử lý các prompt cứng dài. Cách tiếp cận này sẽ có giá trị thực tiễn lớn trong các ứng dụng suy luận LLM mà cùng một prompt được dùng lặp đi lặp lại.

1. Thích nghi LLM với prompt đơn giản hóa

Một cách hiển nhiên để thích nghi một LLM cho một nhiệm vụ cụ thể là đơn giản tinh chỉnh mô hình bằng dữ liệu có nhãn. Điều này dẫn đến nhiều phương pháp căn chỉnh (*alignment*) LLM, chẳng hạn tinh chỉnh có giám sát (*supervised fine-tuning*), vốn cập nhật tham số mô hình bằng cách căn chỉnh câu trả lời cho các prompt cho trước với các tín hiệu giám sát. Các LLM đã tinh chỉnh nhúng thông tin liên quan đến nhiệm vụ vào tham số mô hình, và nhờ vậy các mô hình này có thể trả lời đúng khi gặp những prompt tương tự với các prompt trong quá trình tinh chỉnh.

Nếu đẩy ý tưởng này đi xa hơn, ta có thể kỳ vọng LLM hấp thụ càng nhiều càng tốt tri thức về cách prompt một nhiệm vụ trong quá trình tinh chỉnh. Khi đó, thông tin prompting được nắm bắt một phần trong tham số mô hình, và LLM đã tinh chỉnh có thể thực hiện nhiệm vụ với ít prompting hơn. Ở đây ta xét một dạng prompt đơn giản, chỉ gồm một chỉ dẫn (ký hiệu là $\mathbf{c}$) và một đầu vào của người dùng (ký hiệu là $\mathbf{z}$). Một prompt có thể được biểu diễn bằng bộ sau:

$$\mathbf{x} \;=\; (\mathbf{c}, \mathbf{z}). \tag{9.12}$$

Cho một tập các cặp prompt – câu trả lời $\mathcal{D} = \{(\mathbf{x}, \mathbf{y})\}$, mục tiêu của tinh chỉnh là cực tiểu hóa tổng mất mát trên tập này. Một phương pháp phổ biến là cực tiểu hóa log-hợp lý âm (*negative log-likelihood*) (tức là cực đại hóa log-hợp lý) theo các tham số mô hình θ:

$$\begin{aligned}\hat{\theta} &= \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\mathbf{y}|\mathbf{c},\mathbf{z}),\end{aligned} \tag{9.13}$$

trong đó $\Pr_\theta(\cdot|\cdot)$ là xác suất do một LLM với tham số θ dự đoán[11].

Chú giải của người dịch: Đọc Phương trình (9.13)

$\Pr_\theta(\mathbf{y}|\mathbf{c},\mathbf{z})$ là xác suất mô hình (với bộ tham số θ) sinh ra đúng câu trả lời $\mathbf{y}$ khi nhận chỉ dẫn $\mathbf{c}$ và đầu vào $\mathbf{z}$ — một *xác suất có điều kiện*: "xác suất của $\mathbf{y}$ khi đã biết $\mathbf{c}$ và $\mathbf{z}$". Lấy log rồi cộng qua mọi cặp trong tập dữ liệu $\mathcal{D}$; $\arg\max_\theta$ nghĩa là "chọn bộ tham số θ làm cho tổng này lớn nhất". Vì log là hàm đồng biến, làm tổng log lớn nhất cũng là làm cho mô hình gán xác suất cao cho các câu trả lời đúng. Ví dụ với hai mẫu có xác suất 0.5 và 0.8: tổng $\log = \log 0.5 + \log 0.8 \approx -0.92$ (log tự nhiên); nếu tham số mới cho 0.9 và 0.9 thì tổng ≈ -0.21, lớn hơn, tức tốt hơn. Dấu mũ trong $\hat{\theta}$ chỉ "giá trị tìm được" của θ.

Nói chung, chỉ dẫn trong mỗi ví dụ tinh chỉnh nên tuân theo các nguyên tắc thiết kế prompt; ví dụ, một chỉ dẫn tốt nên rõ ràng nhất có thể và mô tả chi tiết nhiệm vụ. Tuy nhiên, phương pháp mô tả trong phương trình trên không ràng buộc chỉ dẫn phải có dạng cụ thể nào. Sự linh hoạt này cho phép ta ra chỉ dẫn cho LLM theo bất kỳ cách nào ta muốn. Xét một ví dụ trong đó ta muốn chỉ dẫn LLM dịch một câu tiếng Anh sang tiếng Trung. Dĩ nhiên, như đã nói ở phần trước của chương này, ta có thể prompt LLM bằng chỉ dẫn

Dịch câu sau từ tiếng Anh sang tiếng Trung.

Nếu muốn chỉ dẫn đơn giản hơn, ta có thể diễn đạt lại nó ở dạng đơn giản hơn

Dịch câu này sang tiếng Trung.

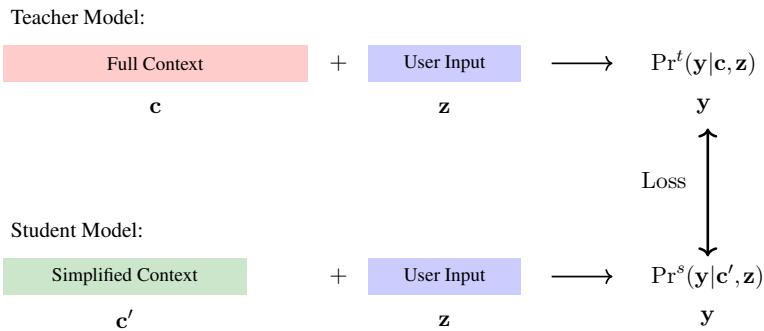
Ta thậm chí có thể định nghĩa chỉ dẫn chỉ là một cụm từ duy nhất

Dịch!

Với tinh chỉnh, ta có thể thích nghi LLM để làm theo bất kỳ chỉ dẫn nào trong số này. Từ góc độ prompting hiệu quả, việc đơn giản hóa chỉ dẫn trong prompting mang lại lợi thế về tính toán. Ví dụ, ta có thể dùng

[11] Trong thực tế, ta khởi tạo θ bằng các tham số thu được từ tiền huấn luyện, rồi điều chỉnh θ một cách vừa phải để đảm bảo kết quả sau tinh chỉnh không lệch quá xa so với kết quả tiền huấn luyện.

những chỉ dẫn đơn giản như "Dịch!" để thực hiện những nhiệm vụ mà thông thường đòi hỏi chỉ dẫn phức tạp và chi tiết hơn. Điều này có thể khiến việc prompting sau đó trong lúc suy luận dễ dàng hơn nhiều. Mặt khác, tinh chỉnh LLM với chỉ dẫn bị đơn giản hóa quá mức có thể gây hại cho khả năng tổng quát hóa của mô hình. Vì chỉ dẫn đơn giản hóa có thể dẫn đến mất mát thông tin, nhiều khả năng LLM sẽ quá khớp (*overfit*) dữ liệu tinh chỉnh và không tổng quát hóa được ra ngoài các chỉ dẫn đó. Vấn đề này càng rõ rệt khi cả chỉ dẫn phức tạp lẫn chỉ dẫn đơn giản hóa cùng được đưa vào quá trình tinh chỉnh, vì dữ liệu có nhãn sẵn có thường không đủ để bao phủ một phạm vi rộng các biến thể của chỉ dẫn.



Hình 9.4: Minh họa chưng cất ngữ cảnh (*context distillation*) [Snell và cộng sự, 2022]. Mô hình thầy là một LLM tiêu chuẩn, nhận cả ngữ cảnh lẫn đầu vào của người dùng làm đầu vào mô hình và đưa ra một dự đoán làm đầu ra. Sau đó, ta đơn giản hóa ngữ cảnh (ví dụ, đơn giản hóa chỉ dẫn trong prompt) và dùng mô hình trò để đưa ra dự đoán dựa trên ngữ cảnh đã đơn giản hóa và đầu vào của người dùng. Mô hình trò được huấn luyện bằng cách cực tiểu hóa mất mát giữa các dự đoán do hai mô hình tạo ra.

Một cách khác để thích nghi LLM với chỉ dẫn đơn giản hóa là thông qua chưng cất tri thức (*knowledge distillation*). Làm ví dụ, ta xét phương pháp chưng cất ngữ cảnh [Snell và cộng sự, 2022]. Mục tiêu của phương pháp này là học một mô hình trò (*student model*) có thể sử dụng chỉ dẫn đơn giản hóa, từ một mô hình thầy (*teacher model*) đã được huấn luyện tốt để làm theo chỉ dẫn. Hình 9.4 minh họa cách tiếp cận này. Mô hình thầy được xây dựng bằng một quá trình tinh chỉnh tiêu chuẩn: trước hết ta thu thập một lượng dữ liệu nhất định gồm chỉ dẫn, đầu vào của người dùng và câu trả lời đúng, rồi tiếp tục huấn luyện một mô hình đã tiền huấn

luyện với tập dữ liệu này. Để xây dựng mô hình trò, ta cần xây dựng một tập dữ liệu mới $\mathcal{D}'$ trong đó mỗi mẫu là một bộ gồm một chỉ dẫn, một chỉ dẫn đơn giản hóa tương ứng và một đầu vào của người dùng, ký hiệu là $\mathbf{x}' = (\mathbf{c}, \mathbf{c}', \mathbf{z})$. Chưng cất tri thức được thực hiện bằng cách cực tiểu hóa một hàm mất mát định nghĩa trên đầu ra của mô hình thầy và mô hình trò:

$$\hat{\theta} \;=\; \arg\min_{\theta} \sum_{\mathbf{x}' \in \mathcal{D}'} \mathrm{Loss}\big(\Pr^{t}(\cdot|\cdot),\, \Pr^{s}_{\theta}(\cdot|\cdot),\, \mathbf{x}'\big), \tag{9.14}$$

trong đó $\Pr^{t}(\cdot|\cdot)$ ký hiệu mô hình thầy đã huấn luyện sẵn, và $\Pr^{s}_{\theta}(\cdot|\cdot)$ ký hiệu mô hình trò với tham số θ. Để ký hiệu đơn giản, ta sẽ viết gọn $\mathrm{Loss}(\Pr^{t}(\cdot|\cdot), \Pr^{s}_{\theta}(\cdot|\cdot), \mathbf{x}')$ thành Loss. Một hàm mất mát thường dùng là mất mát mức chuỗi (*sequence-level loss*), có dạng cơ bản:

$$\mathrm{Loss} \;=\; -\sum_{\mathbf{y}} \Pr^{t}(\mathbf{y}|\mathbf{c}, \mathbf{z}) \log \Pr^{s}_{\theta}(\mathbf{y}|\mathbf{c}', \mathbf{z}). \tag{9.15}$$

Nhưng hàm này không khả thi về mặt tính toán vì nó đòi hỏi lấy tổng trên một số lượng lớn theo cấp số mũ các chuỗi đầu ra khả dĩ. Một biến thể của phương pháp này là huấn luyện mô hình trò bằng các đầu ra do mô hình thầy sinh ra. Với mỗi mẫu, ta dùng mô hình thầy để tạo ra một đầu ra $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \log \Pr^{t}(\mathbf{y}|\mathbf{c}, \mathbf{z})$. Sau đó ta coi $\hat{\mathbf{y}}$ là đích để học, và hàm mất mát được cho bởi

$$\mathrm{Loss} \;=\; -\log \Pr^{s}_{\theta}(\hat{\mathbf{y}}|\mathbf{c}', \mathbf{z}). \tag{9.16}$$

Ngoài ra, ta có thể cực tiểu hóa khoảng cách giữa các phân phối xác suất do hai mô hình sinh ra [Askell và cộng sự, 2021]. Ví dụ, hàm mất mát có thể được định nghĩa là độ phân kỳ KL (*KL divergence*) giữa hai phân phối đầu ra:

$$\mathrm{Loss} \;=\; \mathrm{D}_{\mathrm{KL}}(\mathrm{P}^{t} \,\|\, \mathrm{P}^{s}_{\theta}), \tag{9.17}$$

trong đó

$$\mathrm{P}^{t} = \Pr^{t}(\cdot|\mathbf{c}, \mathbf{z}), \tag{9.18}$$

$$\mathrm{P}^{s}_{\theta} = \Pr^{s}_{\theta}(\cdot|\mathbf{c}', \mathbf{z}). \tag{9.19}$$

Chú giải của người dịch: Thầy dạy trò bằng "chỉ dẫn rút gọn"

Mô hình thầy được đọc chỉ dẫn đầy đủ $\mathbf{c}$ (ví dụ "Dịch câu sau từ tiếng Anh sang tiếng Trung"), còn mô hình trò chỉ được đọc chỉ dẫn rút gọn $\mathbf{c}'$ ("Dịch!"), nhưng phải học cách trả lời *giống thầy*. Phương trình (9.16): thầy viết ra bản dịch tốt nhất $\hat{\mathbf{y}}$, trò được huấn luyện để gán xác suất cao cho đúng bản dịch đó (mất mát $-\log$ xác suất càng nhỏ khi xác suất càng gần 1). Phương trình (9.17): thay vì chỉ bắt chước một đáp án, trò bắt chước cả "mức độ tin tưởng" của thầy vào mọi đáp án có thể. Độ phân kỳ KL đo hai phân phối xác suất khác nhau bao nhiêu: bằng 0 khi chúng trùng nhau và càng lớn khi chúng càng khác nhau. Ví dụ, thầy cho hai đáp án xác suất $(0.7; 0.3)$, trò cho $(0.7; 0.3)$ thì $\mathrm{KL} = 0$; trò cho $(0.5; 0.5)$ thì $\mathrm{KL} = 0.7 \ln \frac{0.7}{0.5} + 0.3 \ln \frac{0.3}{0.5} \approx 0.08 > 0$. Phương trình (9.15) không tính được vì số câu trả lời $\mathbf{y}$ khả dĩ là khổng lồ: với bộ từ vựng 50 000 token và câu dài 20 token, có tới $50\,000^{20}$ chuỗi.

Việc đơn giản hóa prompt thông qua tinh chỉnh hoặc chưng cất không nhất thiết tạo ra prompt mềm tường minh theo nghĩa thông thường. Tuy nhiên, các cách tiếp cận này có chung một trực giác: hành vi prompting đặc thù cho nhiệm vụ có thể dần dần được hấp thụ vào các biểu diễn liên tục gọn nhẹ bên trong mô hình.

Mặc dù ta mới chỉ giới hạn ở việc chưng cất tri thức cho chỉ dẫn, các cách tiếp cận bàn ở đây mang tính tổng quát. Bằng cách học từ đầu ra của mô hình thầy, tri thức trong prompting có thể được chưng cất vào tham số của mô hình trò. Do đó, mô hình đã chưng cất có thể được xem như mã hóa ngầm hành vi prompting đặc thù cho nhiệm vụ trong tham số của nó. Phương pháp này có thể được áp dụng cho nhiều bài toán khác trong học prompt, chẳng hạn nén ngữ cảnh dài và học prompt mềm như những thành phần cụ thể của LLM.

2. Học prompt mềm cho tinh chỉnh hiệu quả tham số

Tinh chỉnh toàn bộ (*full fine-tuning*), tức cập nhật mọi tham số của mô hình, là một phương pháp phổ biến để thích nghi LLM với các nhiệm vụ hạ nguồn. Mặc dù tinh chỉnh ít tốn kém về tính toán hơn nhiều so với tiền huấn luyện, áp dụng nó trong thực tế vẫn tốn kém. Vấn đề này thúc đẩy sự phát triển của các phương pháp tinh chỉnh hiệu quả tham số (*parameter-efficient fine-tuning*), nhằm giảm thiểu số tham số cần được cập nhật.

Một cách tiếp cận, gọi là **tinh chỉnh tiền tố** (*prefix fine-tuning*), gắn thêm một chuỗi các vectơ huấn luyện được, hay tiền tố (*prefix*), vào trước đầu vào của mỗi tầng Transformer [Li và Liang, 2021]. Các tiền tố này có thể được coi là các prompt mềm đóng vai trò ngữ cảnh bổ sung để dẫn dắt hành vi của mô hình trong các nhiệm vụ cụ thể. Trong quá trình tinh chỉnh, ta chỉ cần học các tiền tố để nhúng tri thức đặc thù cho nhiệm vụ. Vì vậy, phương pháp này hiệu quả vì nó chỉ thay đổi một phần nhỏ của mô hình thay vì điều chỉnh toàn bộ tham số mô hình.

Cụ thể, gọi đầu vào của một tầng ở độ sâu l là $\mathbf{H}^l = \mathbf{h}_0^l \mathbf{h}_1^l \cdots \mathbf{h}_m^l$. Đầu ra của tầng có thể được biểu diễn là:

$$\mathbf{H}^{l+1} \;=\; \text{Layer}(\mathbf{H}^l). \tag{9.20}$$

Trong tinh chỉnh tiền tố, ta mở rộng chuỗi $\mathbf{h}_0^l \mathbf{h}_1^l \cdots \mathbf{h}_m^l$ bằng cách thêm vài vectơ vào đầu, ký hiệu là $\mathbf{p}_0^l \mathbf{p}_1^l \cdots \mathbf{p}_n^l$. Do đó $\mathbf{H}^l$ có thể được viết dưới dạng:

$$\mathbf{H}^l = \underbrace{\mathbf{p}_0^l\, \mathbf{p}_1^l\, \cdots\, \mathbf{p}_n^l}_{\text{huấn luyện được}}\; \underbrace{\mathbf{h}_0^l\, \mathbf{h}_1^l\, \cdots\, \mathbf{h}_m^l}_{\text{đầu ra của tầng trước}}. \tag{9.21}$$

Ta chỉ giữ lại $m+1$ biểu diễn cuối cùng làm các trạng thái ẩn truyền sang tầng tiếp theo:

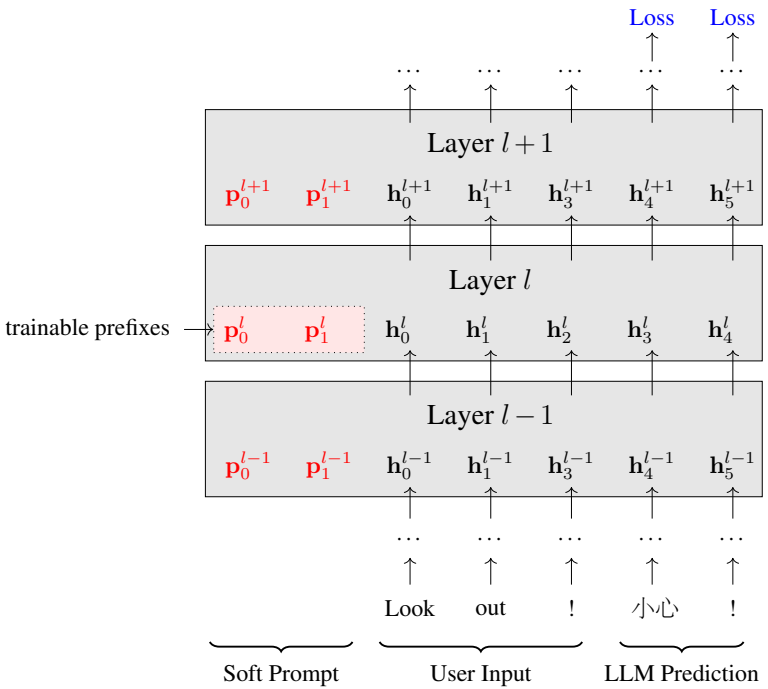
$$\begin{aligned} \overline{\mathbf{H}}^{l+1} &= \text{Layer}(\mathbf{H}^l)[-m-1:] \\ &= \mathbf{h}_0^{l+1} \mathbf{h}_1^{l+1} \cdots \mathbf{h}_m^{l+1}, \end{aligned} \tag{9.22}$$

trong đó $[-m-1:]$ ký hiệu phép cắt lát lấy ra $m+1$ phần tử cuối của một chuỗi. Cho $\overline{\mathbf{H}}^{l+1}$, đầu vào của tầng tiếp theo có thể được biểu diễn cùng dạng với Phương trình (9.21):

$$\mathbf{H}^{l+1} = \mathbf{p}_0^{l+1}\, \mathbf{p}_1^{l+1}\, \cdots\, \mathbf{p}_n^{l+1}\, \overline{\mathbf{H}}^{l+1}$$

$$= \mathbf{p}_0^{l+1}\, \mathbf{p}_1^{l+1} \cdots \mathbf{p}_n^{l+1} \mathbf{h}_0^{l+1} \mathbf{h}_1^{l+1} \cdots \mathbf{h}_m^{l+1}. \tag{9.23}$$

Ở đây mỗi $\mathbf{p}_i \in \mathbb{R}^d$ có thể được xem là một tham số học được. Trong quá trình huấn luyện, $\mathbf{p}_0^l \mathbf{p}_1^l \cdots \mathbf{p}_n^l$ được huấn luyện như thường lệ, còn các tham số của mô hình Transformer ban đầu được giữ cố định.



Hình 9.5: Minh họa tinh chỉnh tiền tố cho một nhiệm vụ dịch (Look out! → 小心!). Ở mỗi tầng, ta thêm hai tiền tố $\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ vào đầu. LLM được huấn luyện để cực tiểu hóa mất mát trên các dự đoán khi cho trước đầu vào. Trong quá trình này, chỉ các tiền tố được tối ưu hóa còn các tham số còn lại được giữ cố định. Vì vậy, mô hình có thể thích nghi với nhiệm vụ chỉ với một số ít tham số huấn luyện được. Khi suy luận, LLM làm việc với các tiền tố đã tối ưu hóa và có thể thực hiện nhiệm vụ mà không cần prompt cứng tường minh.

Hình 9.5 minh họa tinh chỉnh tiền tố cho một nhiệm vụ dịch. Ở đây, chỉ các vectơ tiền tố $\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ được cập nhật thông qua gradient lan truyền từ tầng đầu ra (tức bản dịch tiếng Trung). Bằng cách điều chỉnh các vectơ này cho nhiệm vụ dịch, mô hình thích nghi tương ứng. Điều này khiến

$\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ đóng vai trò như các prompt "kích hoạt" LLM thực hiện nhiệm vụ mà không cần prompt đầu vào tường minh như "Dịch câu sau từ tiếng Anh sang tiếng Trung". Khi kiểm tra, ta gắn $\mathbf{p}_0^l$ và $\mathbf{p}_1^l$ đã tối ưu hóa vào trước tầng, và LLM sẽ dịch câu đầu vào. Lưu ý rằng tinh chỉnh tiền tố đưa vào thêm $L \times n \times d$ tham số, trong đó L là số tầng, n là số tiền tố, và d là số chiều của mỗi tiền tố. Tuy nhiên, con số này nhỏ hơn rất nhiều so với tổng số tham số của LLM, khiến quá trình tinh chỉnh rất hiệu quả.

Chú giải của người dịch: Đọc Hình 9.5 và ước lượng số tham số

Hình vẽ ba tầng liên tiếp ($l-1$, l, $l+1$) của Transformer. Ở mỗi tầng, bên trái có hai ô màu đỏ $\mathbf{p}_0, \mathbf{p}_1$ là *tiền tố huấn luyện được* (*trainable prefixes*) — phần *Soft Prompt*; tiếp theo là các vectơ $\mathbf{h}$ của câu người dùng "Look out!" (*Coi chừng!*) — phần *User Input*; cuối cùng là bản dịch "小心!" do LLM dự đoán (*LLM Prediction*), nơi tính mất mát (*Loss*). Chỉ số trên l là "thuộc tầng thứ l", không phải số mũ. Phép cắt $[-m-1:]$ trong Phương trình (9.22) nghĩa là bỏ đi các vị trí của tiền tố ở đầu ra, chỉ giữ $m+1$ vị trí ứng với token thật, rồi tầng sau lại gắn bộ tiền tố riêng của nó. Ví dụ số: một LLM có $L = 32$ tầng, mỗi vectơ $d = 4096$ chiều, dùng $n = 10$ tiền tố mỗi tầng thì chỉ thêm $32 \times 10 \times 4096 \approx 1{,}3$ triệu tham số — rất nhỏ so với khoảng 7 tỉ tham số của cả mô hình (chưa tới 0,02%).

Mặc dù tinh chỉnh tiền tố đơn giản, nó vẫn đòi hỏi phải sửa đổi LLM. Thay vào đó, tách prompt mềm khỏi LLM cho phép ta giữ nguyên kiến trúc mô hình ban đầu, giúp việc triển khai trên nhiều nhiệm vụ khác nhau hiệu quả hơn mà không cần điều chỉnh mô hình lõi. Một phương pháp như vậy là tinh chỉnh prompt (*prompt tuning*) [Lester và cộng sự, 2021]. Giống như tinh chỉnh tiền tố, tinh chỉnh prompt đưa vào các vectơ huấn luyện được để LLM có thể thích nghi với nhiệm vụ cho trước bằng cách điều chỉnh các vectơ này. Tuy nhiên, tinh chỉnh prompt khác ở chỗ nó chỉ thay đổi tầng nhúng.

Nhớ lại rằng trong LLM, mỗi token đầu vào z_i được biểu diễn bằng một vectơ nhúng $\mathbf{e}_i$. Các vectơ nhúng này thường được học thông qua một mô hình nhúng token, rồi được dùng làm đầu vào thực sự cho LLM, thay cho các token được biểu diễn bằng ký hiệu. Trong tinh chỉnh prompt, một số vectơ nhúng giả (*pseudo embedding*) $\mathbf{p}_0 \cdots \mathbf{p}_n$ được thêm vào đầu chuỗi vectơ nhúng token. Vì vậy đầu vào thực sự của LLM có thể được

biểu diễn là:

$$\underbrace{\mathbf{p}_0\,\mathbf{p}_1\,\cdots\,\mathbf{p}_n}_{\text{huấn luyện được}}\;\underbrace{\mathbf{e}_0\,\mathbf{e}_1\,\cdots\,\mathbf{e}_m}_{\text{vectơ nhúng token}}. \tag{9.24}$$

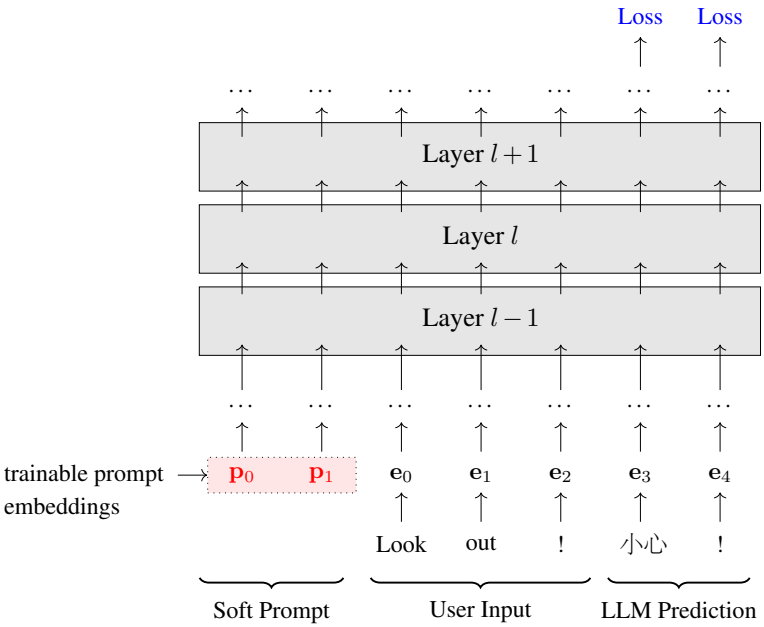
Lưu ý rằng một vectơ nhúng giả không cần tương ứng với bất kỳ token nào trong ngôn ngữ tự nhiên. Thay vào đó, các vectơ nhúng này có thể được xem là "vectơ nhúng prompt mềm" dùng để điều kiện hóa LLM. Bằng cách huấn luyện các vectơ nhúng prompt mềm trên dữ liệu đặc thù cho nhiệm vụ, chúng học cách tương tác một cách thích nghi với các vectơ nhúng token $\mathbf{e}_0 \cdots \mathbf{e}_m$ và dẫn dắt hành vi của LLM. Vì tinh chỉnh prompt không thay đổi các tham số nền tảng của LLM đã tiền huấn luyện, nó được coi là một phương pháp tinh chỉnh nhẹ và hiệu quả, giúp cải thiện hiệu năng trên nhiệm vụ cụ thể trong khi vẫn duy trì khả năng tổng quát hóa của mô hình. Xem Hình 9.6 để thấy minh họa về tinh chỉnh prompt.

Chú giải của người dịch: Tinh chỉnh tiền tố và tinh chỉnh prompt khác nhau ở đâu?

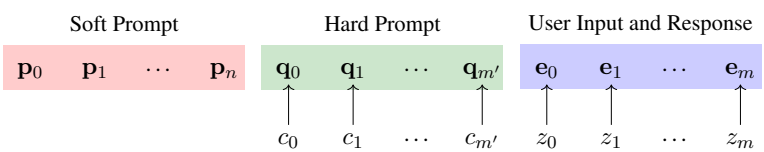
Cả hai đều "đóng băng" LLM và chỉ học vài vectơ prompt mềm. Tinh chỉnh tiền tố (Hình 9.5) chèn vectơ tiền tố riêng vào *mọi* tầng. Tinh chỉnh prompt (Hình 9.6) chỉ chèn vài vectơ $\mathbf{p}_0, \mathbf{p}_1$ (nhãn *trainable prompt embeddings* — vectơ nhúng prompt huấn luyện được) ở *tầng nhúng đầu vào*, ngay cạnh vectơ nhúng của các token thật; các tầng phía trên giữ nguyên. Vì thế với tinh chỉnh prompt, ta có thể dùng chung một LLM cho nhiều nhiệm vụ, chỉ cần đổi vài vectơ $\mathbf{p}$ — giống như dùng chung một chiếc máy ảnh nhưng thay "bộ lọc" cho từng kiểu chụp.

Vì $\mathbf{p}_0\,\mathbf{p}_1\,\cdots\,\mathbf{p}_n$ tự nó là một chuỗi, ta có thể dùng các mô hình chuỗi để biểu diễn nó tốt hơn. Ví dụ, một mô hình Transformer có thể mã hóa chuỗi này, và biểu diễn thu được sau đó có thể được dùng làm đầu vào cho LLM. Nói cách khác, ta có thể xây dựng một mô hình bổ sung để mã hóa prompt mềm. Một cách khác để cải thiện prompting là kết hợp prompt mềm và prompt cứng, qua đó tận dụng ưu điểm của cả hai loại [Liu và cộng sự, 2023c]. Trong chuỗi vectơ nhúng, ta có thể sắp xếp hoặc xen kẽ các prompt này. Điều này dẫn đến các mẫu prompt khác nhau. Ví dụ, xem Hình 9.7 để thấy một mẫu đơn giản kết hợp cả hai loại prompt.

Ở đây ta đã xem xét các phương pháp chèn prompt mềm vào LLM. Nhưng ta bỏ qua chi tiết huấn luyện các prompt mềm này và giả định rằng bạn đọc đã quen với quá trình học có giám sát tiêu chuẩn, tức là cực đại



Hình 9.6: Minh họa tinh chỉnh prompt cho một nhiệm vụ dịch (Look out! → 小心!). Thay vì dùng các prompt văn bản cố định, prompt mềm là các vectơ nhúng học được, được thêm vào đầu chuỗi vectơ nhúng. Trong quá trình tinh chỉnh, chỉ các vectơ nhúng prompt này được tối ưu hóa để thích nghi LLM với nhiệm vụ cho trước một cách hiệu quả. Sau khi đã tối ưu hóa, các vectơ nhúng prompt được dùng để chỉ dẫn LLM thực hiện nhiệm vụ khi có dữ liệu mới.



Hình 9.7: Một mẫu prompt kết hợp prompt mềm (*Soft Prompt*), prompt cứng (*Hard Prompt*) và đầu vào của người dùng (*User Input and Response* — đầu vào của người dùng và câu trả lời). Ở đây, $c_0 \cdots c_{m'}$ ký hiệu prompt cứng và $\mathbf{q}_0 \cdots \mathbf{q}_{m'}$ ký hiệu chuỗi vectơ nhúng tương ứng.

hóa hàm hợp lý (*likelihood*) của đầu ra đúng khi cho trước đầu vào mô hình. Thực ra, việc học prompt mềm có thể liên hệ với nhiều vấn đề trong tinh chỉnh LLM. Ví dụ, nếu coi nó là một bài toán nén ngữ cảnh, ta có thể áp dụng các phương pháp chưng cất tri thức đã mô tả ở trên. Trong công trình của Mu và cộng sự [2024], prompt được nén và biểu diễn thành một vài token giả (*pseudo token*), được gắn vào mỗi chuỗi đầu vào. Các vectơ nhúng của những token giả này được tối ưu hóa để bắt chước dự đoán của một mô hình được prompt theo cách tiêu chuẩn. Nói cách khác, tri thức prompting được chưng cất từ một mô hình thầy vào các token giả.

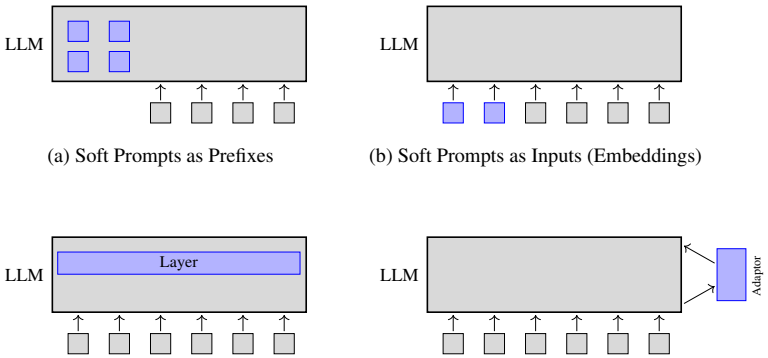
Nói rộng ra, nhiều phương pháp tinh chỉnh hiệu quả tham số có thể được coi là học một dạng prompt mềm nào đó [Lialin và cộng sự, 2023]. Khi ta tinh chỉnh một phần của LLM cho một nhiệm vụ, về bản chất quá trình này có thể được xem như việc "tiêm" thông tin prompting liên quan đến nhiệm vụ vào phần cụ thể đó của mô hình. Một cách tiếp cận được dùng rộng rãi khác cho tinh chỉnh hiệu quả tham số là thêm một tầng thích nghi (*adaptor layer*) vào giữa các tầng sẵn có của mô hình. Cách tiếp cận này cho phép ta chỉ tinh chỉnh tầng thích nghi trên các nhiệm vụ cụ thể mà không thay đổi kiến trúc nền tảng hay huấn luyện lại toàn bộ mô hình. Theo nghĩa này, các tầng thích nghi có thể được xem như prompt mềm mã hóa thông tin prompting và thông tin liên quan đến nhiệm vụ, tương tác với LLM ban đầu để giúp nó thích nghi. Tóm lại, Hình 9.8 so sánh các phương pháp khác nhau để sử dụng prompt mềm trong LLM.

3. Học prompt mềm bằng nén

Một cách tiếp cận khác để học prompt mềm là phát biểu nó thành một bài toán nén. Làm ví dụ đơn giản, xét bài toán xấp xỉ một ngữ cảnh dài bằng một biểu diễn liên tục [Wingate và cộng sự, 2022]. Giả sử ta có một đầu vào của người dùng $\mathbf{z}$ và ngữ cảnh của nó là $\mathbf{c}$ (chẳng hạn các chỉ dẫn và ví dụ minh họa dài). Giờ ta muốn học một biểu diễn nén của ngữ cảnh, ký hiệu là σ, sao cho dự đoán có điều kiện trên $\mathbf{z}$ và σ càng gần càng tốt với dự đoán có điều kiện trên $\mathbf{z}$ và $\mathbf{c}$. Mục tiêu này có thể được phát biểu một cách không chặt chẽ là

$$\hat{\sigma} \;=\; \arg\min_{\sigma} s(\hat{\mathbf{y}}, \hat{\mathbf{y}}_{\sigma}), \tag{9.25}$$

trong đó $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{c}, \mathbf{z})$ và $\hat{\mathbf{y}}_{\sigma} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\sigma, \mathbf{z})$ lần lượt là dự đoán của LLM khi cho ngữ cảnh đầy đủ và ngữ cảnh đã nén. Hàm



(a) Soft Prompts as Prefixes

(b) Soft Prompts as Inputs (Embeddings)

(c) Fine-tuning Parts of the Model

(d) Fine-tuning the Adaptor

Hình 9.8: Minh họa việc sử dụng prompt mềm trong LLM. Ở đây các prompt mềm điều chỉnh được được vẽ màu xanh, còn các thành phần có tham số cố định trong quá trình tinh chỉnh được vẽ màu xám. Trong hình con (a) (*Soft Prompts as Prefixes* — prompt mềm làm tiền tố), prompt mềm là các tiền tố gắn vào mỗi tầng của LLM. Trong hình con (b) (*Soft Prompts as Inputs (Embeddings)* — prompt mềm làm đầu vào, dạng vectơ nhúng), prompt mềm được dùng làm vectơ nhúng đầu vào cho LLM. Trong các hình con (c) (*Fine-tuning Parts of the Model* — tinh chỉnh một phần mô hình, ở đây là một tầng, *Layer*) và (d) (*Fine-tuning the Adaptor* — tinh chỉnh bộ thích nghi, *Adaptor*), prompt mềm được hiểu theo nghĩa rộng là các thành phần của mô hình được tinh chỉnh để thích nghi với nhiệm vụ.

$s(\cdot, \cdot)$ đo mức độ khác biệt giữa hai dự đoán.

Một khung tổng quát để đạt được điều này là chưng cất tri thức, trong đó $\hat{\mathbf{y}}$ và $\hat{\mathbf{y}}_\sigma$ lần lượt có thể được xem là dự đoán của mô hình thầy và mô hình trò. Cách hình thức hóa này nối thảo luận của ta với bài toán chưng cất ngữ cảnh đã bàn ở trên. Mục tiêu huấn luyện có thể được định nghĩa tương tự như Phương trình (9.16) và (9.17). Chẳng hạn, một mục tiêu hợp lý cực đại (*maximum likelihood*) đơn giản được cho bởi

$$\hat{\sigma} \;=\; \arg\max_{\sigma} \log \Pr(\hat{\mathbf{y}} | \sigma, \mathbf{z}). \tag{9.26}$$

Ngoài ra, ta có thể cực tiểu hóa độ phân kỳ KL giữa các phân phối đầu ra, cho ta

$$\hat{\sigma} \;=\; \arg\min_{\sigma} \mathrm{D}_{\mathrm{KL}}\big(\Pr(\cdot | \mathbf{c}, \mathbf{z}) \parallel \Pr(\cdot | \sigma, \mathbf{z}) \big). \tag{9.27}$$

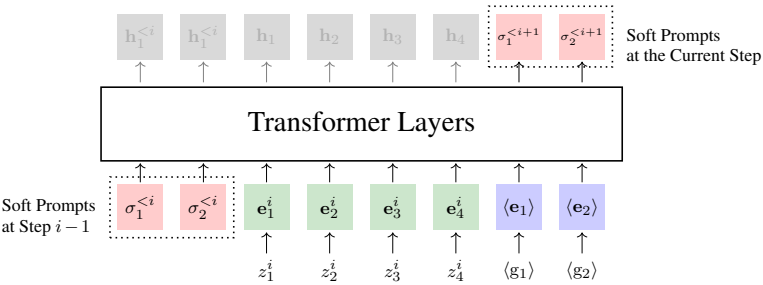
Khác biệt so với các mô hình trong Phương trình (9.16) và (9.17) là ở đây ngữ cảnh đã nén được biểu diễn dưới dạng các vectơ giá trị thực (gọi là **vectơ nhúng prompt** (*prompt embedding*)), chứ không phải các token thông thường. Bằng cách áp dụng các phương pháp trên, ta chưng cất ngữ cảnh từ chuỗi token $\mathbf{c}$ vào các vectơ nhúng σ. Lưu ý rằng mô hình thầy $\Pr(\cdot | \mathbf{c}, \mathbf{z})$ và mô hình trò $\Pr(\cdot | \sigma, \mathbf{z})$ có thể không có cùng kiến trúc hay cùng thiết lập mô hình. Trong thực tế, ta thường mong mô hình thầy mạnh hơn, còn mô hình trò nhỏ hơn và hiệu quả hơn.

Chú giải của người dịch: Nén ngữ cảnh giống "tóm tắt bài học"

Ngữ cảnh $\mathbf{c}$ có thể dài hàng nghìn token (chỉ dẫn + nhiều ví dụ). Ta muốn thay nó bằng σ — chỉ vài vectơ số — mà mô hình vẫn trả lời như khi đọc đủ $\mathbf{c}$. Giống như thay vì mang cả cuốn sách vào phòng thi, ta mang một "tờ tóm tắt" nhỏ nhưng vẫn làm bài tốt như nhau. Điểm khác với Phương trình (9.16)–(9.17) là ở đó ta học *tham số θ của mô hình trò*, còn ở đây ta học chính *các vectơ σ* (mô hình giữ nguyên).

Mặc dù nén toàn bộ ngữ cảnh thành các biểu diễn liên tục là một cách tiếp cận đơn giản để học prompt mềm, nó đòi hỏi một mô hình thầy có thể xử lý các chuỗi đầu vào dài. Tuy nhiên, trong nhiều trường hợp, ngữ cảnh dài đến mức việc áp dụng một LLM quá tốn kém hoặc không khả thi. Mô hình hóa các chuỗi đầu vào dài có thể được xếp vào họ rộng các phương pháp hiệu quả cho LLM ngữ cảnh dài. Nhiều kỹ thuật đã được phát triển để giải quyết vấn đề này. Ví dụ, có thể dùng một bộ nhớ đệm KV (*KV*

cache) kích thước cố định để lưu thông tin quá khứ ở mỗi bước trong quá trình lập luận. Các kiến trúc Transformer hiệu quả và LLM ngữ cảnh dài đã được bàn rộng rãi trong cuốn sách này. Để có thảo luận chi tiết hơn về các chủ đề này, bạn đọc quan tâm có thể tham khảo Chương 6 và 8.



Hình 9.9: Minh họa việc nén một đoạn ngữ cảnh thành prompt mềm ($\kappa = 2$ và $m_i = 4$). Đầu vào của LLM gồm các prompt mềm từ bước trước ($\sigma_1^{<i}$ và $\sigma_2^{<i}$), các token của đoạn (z_1, z_2, z_3 và z_4), và các token tóm tắt ($\langle \mathrm{g}_1 \rangle$ và $\langle \mathrm{g}_2 \rangle$). Với các đầu vào này, LLM hoạt động như thường lệ. Sau đó ta trích các đầu ra ở tầng Transformer cuối cùng tương ứng với các token tóm tắt. Các đầu ra này có thể được xem là các prompt mềm tích lũy được cho tới đoạn này.

Cũng có những phương pháp được thiết kế riêng để nén các chuỗi dài thành prompt mềm. Ở đây ta xét phương pháp của Chevalier và cộng sự [2023] làm ví dụ. Ý tưởng cơ bản là học dần prompt mềm bằng cách nén chuỗi đầu vào thành một biểu diễn có kích thước cố định. Để đơn giản, ta dùng $\mathbf{z}$ để ký hiệu toàn bộ chuỗi đầu vào cần nén. Cho một chuỗi đầu vào dài, trước hết ta chia nó thành một số đoạn $\{\mathbf{z}^1, \cdots, \mathbf{z}^K\}$. Sau đó ta xử lý lần lượt các đoạn này, mỗi lần sinh ra một biểu diễn của phần đầu vào đã xử lý tới lúc đó, ký hiệu là $\sigma^{<i+1}$. Để làm việc này, ta đưa vào một vài token tóm tắt (*summary token*) $\langle \mathrm{g}_1 \rangle, \cdots, \langle \mathrm{g}_\kappa \rangle$. Ở mỗi bước, ta lấy một đoạn $\mathbf{z}^i = z_1^i \cdots z_{m_i}^i$, cùng với biểu diễn nén trước đó $\sigma^{<i}$ và các token tóm tắt $\langle \mathrm{g}_1 \rangle, \cdots, \langle \mathrm{g}_\kappa \rangle$ làm đầu vào, và dùng một LLM để tạo ra chuỗi biểu diễn ẩn tương ứng ở tầng Transformer cuối cùng. Một ví dụ của quá trình này được minh họa trong Hình 9.9.

Ở đây $\sigma^{<i}$ về bản chất là một bộ nhớ. Mô hình hoạt động theo kiểu RNN. Mỗi lần ta lấy một đoạn và cập nhật bộ nhớ này bằng cách mã hóa cả trạng thái bộ nhớ trước đó lẫn đoạn đó. Vì vậy, $\sigma^{<i}$ được tạo ra ở đoạn cuối cùng là một biểu diễn của toàn bộ chuỗi ngữ cảnh. Mô hình Transformer

dùng để học các biểu diễn này có thể là một LLM tiêu chuẩn, nhưng ta cần tinh chỉnh nó để thích nghi với nhiệm vụ biểu diễn ngữ cảnh này.

Chú giải của người dịch: Đọc Hình 9.9: "cuốn chiếu" từng đoạn

Giả sử văn bản dài 4000 token, chia thành $K = 1000$ đoạn mỗi đoạn $m_i = 4$ token. Ở bước i, LLM đọc: 2 vectơ "ghi nhớ" từ bước trước ($\sigma_1^{<i}, \sigma_2^{<i}$, ô đỏ bên trái, nhãn *Soft Prompts at Step* $i-1$), 4 token của đoạn thứ i (ô xanh lá), và 2 token tóm tắt đặc biệt $\langle g_1 \rangle, \langle g_2 \rangle$ (ô tím). Hai đầu ra ở vị trí token tóm tắt (ô đỏ phía trên, nhãn *Soft Prompts at the Current Step*) trở thành "ghi nhớ" mới $\sigma^{<i+1}$. Như vậy dù văn bản dài bao nhiêu, "bộ nhớ" luôn chỉ gồm $\kappa = 2$ vectơ — giống một học sinh đọc từng trang sách và sau mỗi trang chỉ cập nhật lại hai dòng ghi chú tóm tắt. Đây chính là cách mạng RNN mang trạng thái ẩn từ bước này sang bước sau.

Lưu ý rằng ở đây ta đơn giản coi *prompt* và *ngữ cảnh* là những thuật ngữ tương tự, mặc dù chúng không giống nhau. Dù ta dùng thuật ngữ *prompt* theo nghĩa rộng để bao gồm cả thông tin ngữ cảnh cung cấp cho mô hình, ta thường có thể xem nó như một loại ngữ cảnh. Từ góc nhìn này, các phương pháp bàn ở đây có thể được áp dụng cho các bài toán nén văn bản nói chung.

9.3.3 Rút gọn độ dài prompt

Mặc dù prompt mềm cung cấp các biểu diễn ẩn dày đặc, chúng nhìn chung không diễn giải trực tiếp được. Việc thiếu khả năng diễn giải này khiến người dùng khó hiểu prompt ảnh hưởng thế nào tới đầu ra của LLM. Hơn nữa, việc sửa đổi prompt mềm thường đòi hỏi tinh chỉnh bổ sung, điều này hạn chế tính hữu dụng của chúng trong các môi trường biến động, nơi prompt thay đổi thường xuyên.

Một cách tiếp cận khác để cải thiện hiệu quả của prompt là đơn giản hóa chính văn bản prompt. Khác với prompt mềm, prompt văn bản đã đơn giản hóa vẫn diễn giải được và có thể được người dùng sửa đổi trực tiếp. Ví dụ, dưới đây là một prompt để trả lời các câu hỏi về y tế và tài chính.

Nhiệm vụ là xây dựng một mô hình ngôn ngữ có khả năng hiểu và trả lời các câu hỏi của người dùng trong nhiều lĩnh vực khác nhau, đặc biệt chú trọng vào y tế và tài chính. Xét đến phạm vi rộng của các câu hỏi có thể có, từ những

chi tiết cụ thể của chẩn đoán y khoa đến những sắc thái của các quy định tài chính, mô hình phải đảm bảo hiểu một cách toàn diện và trả lời chính xác.
Câu hỏi:
Đâu là những thực hành tốt nhất khi sử dụng trí tuệ nhân tạo trong chẩn đoán các bệnh tim mạch?

___

Ta có thể đơn giản hóa phần mô tả nhiệm vụ bằng cách xóa các phần không quan trọng.

> *Nhiệm vụ là xây dựng một mô hình ngôn ngữ có khả năng hiểu và trả lời các câu hỏi của người dùng ~~trong nhiều lĩnh vực khác nhau, đặc biệt chú trọng vào~~ về y tế và tài chính. ~~Xét đến phạm vi rộng của các câu hỏi có thể có, từ những chi tiết cụ thể của chẩn đoán y khoa đến những sắc thái của các quy định tài chính,~~ Mô hình phải đảm bảo hiểu một cách toàn diện và trả lời chính xác.*

Ta cũng có thể diễn đạt lại nó thành một văn bản ngắn hơn.

> *Nhiệm vụ là xây dựng một mô hình ngôn ngữ tập trung vào y tế và tài chính, có khả năng hiểu và trả lời chính xác nhiều loại câu hỏi của người dùng.*

Bài toán này có thể được xem là một nhiệm vụ NLP tiêu chuẩn — đơn giản hóa văn bản (*text simplification*). Vì vậy, các phương pháp dùng cho đơn giản hóa văn bản nhìn chung áp dụng được và không chỉ giới hạn ở việc đơn giản hóa prompt. Có nhiều cách để đạt được điều này. Một cách đơn giản là định nghĩa một số quy tắc kinh nghiệm (*heuristics*) và xác định các từ thừa có thể loại bỏ mà không làm mất thông tin thiết yếu. Ví dụ, ta có thể xem xét từng token trong một chuỗi về mức độ đóng góp của nó vào nghĩa tổng thể và loại bỏ những token đóng góp rất ít [Li và cộng sự, 2023c; Jiang và cộng sự, 2023b]. Một phương pháp khác là phát biểu bài toán thành một nhiệm vụ chuỗi-sang-chuỗi (*sequence-to-sequence*). Cho dữ liệu có nhãn về đơn giản hóa văn bản, ta có thể huấn luyện một mô hình bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) để biến đổi mỗi văn bản đầu vào thành dạng đơn giản hóa của nó. Ngoài ra, vì nhiều LLM đã được tinh chỉnh và căn chỉnh để thực hiện các nhiệm vụ đơn giản hóa văn

bản, dùng các mô hình này để đơn giản hóa prompt là việc đơn giản. Ví dụ, ta có thể prompt một LLM đơn giản hóa một prompt trong khi thỏa mãn các ràng buộc như giới hạn độ dài.

Chú giải của người dịch: Vì sao prompt ngắn lại có lợi?

Chi phí tính toán (và cả tiền phải trả khi dùng LLM qua API) tăng theo số token đầu vào; với cơ chế tự chú ý, khối lượng tính toán còn tăng nhanh hơn tỷ lệ thuận với độ dài. Nếu một prompt 60 token được rút còn 25 token mà câu trả lời vẫn tốt như cũ, và prompt được dùng một triệu lần mỗi ngày, ta tiết kiệm được 35 triệu token xử lý mỗi ngày. Khác với prompt mềm, prompt rút gọn vẫn là chữ nên con người đọc và sửa được.

9.4 Tóm tắt

Trong chương này, chúng ta đã bàn về nhiều chủ đề liên quan đến prompting LLM. Thảo luận của chúng ta tập trung chủ yếu vào hai khía cạnh:

- Làm thế nào để thiết kế các prompt cơ bản nhằm dẫn dắt dự đoán của LLM, và tinh chỉnh các prompt này để giải quyết vấn đề hiệu quả hơn (cả về chất lượng lẫn chi phí)?
- Làm thế nào để tự động hóa việc thiết kế và biểu diễn prompt?

Lời giải cho các vấn đề này bao gồm cả những thiết kế prompt tổng quát lẫn các kỹ thuật nâng cao hơn, như CoT và học prompt, vốn đã được khám phá rộng rãi trong các nghiên cứu gần đây.

Trong NLP, prompting có thể được xem là một công nghệ phát triển song hành cùng LLM, và theo một nghĩa nào đó, nó đã mở cánh cửa cho việc ứng dụng thực tế các mô hình này trong một phạm vi lĩnh vực bài toán rộng đến ấn tượng. Thực ra, nếu mở rộng khái niệm prompt ở một mức độ nào đó, ta có thể truy nguyên nó về những ngày đầu của học máy và NLP. Ví dụ, nhiều hệ thống NLP dùng các đặc trưng và khuôn mẫu thủ công để "prompt" các nhiệm vụ cụ thể. Hãy tưởng tượng việc xây dựng một đặc trưng cho biết một văn bản là trang trọng hay thân mật. Ta có thể đưa đặc trưng này vào một hệ thống dịch máy (*machine translation*) để bản dịch được điều kiện hóa theo loại của văn bản đầu vào.

Việc sử dụng rộng rãi khái niệm prompt hiện đại bắt đầu cùng với sự trỗi dậy của các mô hình tiền huấn luyện lớn trong lĩnh vực NLP. Ban đầu, các mô hình này, như BERT, được thích nghi với các nhiệm vụ hạ nguồn cụ thể chủ yếu thông qua tinh chỉnh. Tuy nhiên, các nhà nghiên cứu sớm phát hiện ra rằng bằng cách thiết kế các "prompt" cụ thể — thêm một số từ hoặc câu nhất định vào đầu vào — các mô hình có thể được kích hoạt để đáp ứng các nhiệm vụ cụ thể mà không cần tinh chỉnh nhiều. Điều này thúc đẩy cộng đồng NLP phát triển và áp dụng các mô hình nền tảng (*foundation model*) phổ quát có thể được prompt để giải quyết nhiều nhiệm vụ khác nhau mà không thay đổi kiến trúc nền tảng và quy trình tiền huấn luyện.

Các cách tiếp cận prompting lúc đầu được thử nghiệm với các mô hình nhỏ hơn và sau đó đã thể hiện những năng lực ấn tượng với các mô hình lớn như GPT-3, vốn có thể sinh văn bản chất lượng cao để đáp lại các prompt đơn giản trong nhiều nhiệm vụ. Khi công nghệ prompting phát triển, thiết kế prompt nổi lên như một lĩnh vực nghiên cứu then chốt. Như đã bàn trong chương này, nói rộng ra nó liên quan đến việc thiết kế các prompt hiệu quả để tối đa hóa hiệu năng mô hình, bao gồm cả prompt thủ công lẫn prompt sinh tự động. Các nghiên cứu gần đây hơn đã khám phá cách nâng cao hiệu quả của prompting thông qua các kỹ thuật như học few-shot, học zero-shot và suy luận CoT, giúp LLM hoạt động hiệu quả trong nhiều tình huống khác nhau. Một thảo luận tổng quát về prompting có thể rất rộng, và chúng tôi không thể đề cập hết mọi chi tiết trong chương này. Về các kỹ thuật prompting nâng cao hơn, bạn đọc có thể tham khảo các bài tổng quan gần đây. Các chủ đề bao gồm học trong ngữ cảnh [Li, 2023; Dong và cộng sự, 2022], CoT [Chu và cộng sự, 2023; Yu và cộng sự, 2023b; Zhang và cộng sự, 2023a], prompting hiệu quả [Chang và cộng sự, 2024a], và thiết kế prompt nói chung [Liu và cộng sự, 2023d; Chen và cộng sự, 2023a].

Lưu ý rằng mặc dù lý tưởng nhất là ta muốn xây dựng các phương pháp prompting tổng quát mà không cần điều chỉnh kiến trúc và tham số mô hình, kết quả của prompting nhìn chung phụ thuộc nhiều vào chất lượng và kích thước của LLM cho trước. Với những mô hình mạnh hơn, như các LLM trực tuyến thương mại, các prompt đơn giản có thể đủ để chỉ dẫn mô hình thực hiện nhiệm vụ đúng. Trong trường hợp này, thiết kế prompt tương đối dễ, dù việc tinh chỉnh prompt cẩn thận vẫn cần thiết. Ngược lại,

với những mô hình kém năng lực hơn hoặc có quy mô nhỏ hơn, ta có thể cần thiết kế prompt cẩn thận để đạt được kết quả mong muốn. Trong nhiều trường hợp, tinh chỉnh vẫn cần thiết để thích nghi mô hình với các chiến lược prompting phức tạp.

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.